OpenShift Container Platform 4.22

Authentication and authorization

Configuring user authentication and access controls for users and services

Last Updated: 2026-08-06

OpenShift Container Platform 4.22 Authentication and authorization

Configuring user authentication and access controls for users and services

Legal Notice

Abstract

This document provides instructions for defining identity providers in OpenShift Container Platform. It also discusses how to configure role-based access control to secure the cluster.

Table of Contents

CHAPTER 1. OVERVIEW OF AUTHENTICATION AND AUTHORIZATION

Learn about authentication and authorization in OpenShift Container Platform, and how you can control access to your cluster.

1.1. GLOSSARY OF COMMON TERMS FOR OPENSHIFT CONTAINER PLATFORM AUTHENTICATION AND AUTHORIZATION

This glossary defines common terms that are used in OpenShift Container Platform authentication and authorization.

authentication

An authentication determines access to an OpenShift Container Platform cluster and ensures only authenticated users access the OpenShift Container Platform cluster.

authorization

Authorization determines whether the identified user has permissions to perform the requested action.

bearer token

Bearer token is used to authenticate to API with the header **Authorization: Bearer <token>**.

Cloud Credential Operator

The Cloud Credential Operator (CCO) manages cloud provider credentials as custom resource definitions (CRDs).

config map

A config map provides a way to inject configuration data into the pods. You can reference the data stored in a config map in a volume of type **ConfigMap**. Applications running in a pod can use this data.

containers

Lightweight and executable images that consist of software and all its dependencies. Because containers virtualize the operating system, you can run containers in a data center, public or private cloud, or your local host.

Custom Resource (CR)

A CR is an extension of the Kubernetes API.

group

A group is a set of users. A group is useful for granting permissions to multiple users one time.

HTPasswd

HTPasswd updates the files that store usernames and password for authentication of HTTP users.

Keystone

Keystone is an Red Hat OpenStack Platform (RHOSP) project that provides identity, token, catalog, and policy services.

Lightweight directory access protocol (LDAP)

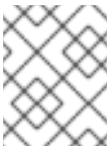
LDAP is a protocol that queries user information.

manual mode

In manual mode, a user manages cloud credentials instead of the Cloud Credential Operator (CCO).

mint mode

In mint mode, the Cloud Credential Operator (CCO) uses the provided administrator-level cloud credential to create new credentials for components in the cluster with only the specific permissions that are required.



NOTE

Mint mode is the default and the preferred setting for the CCO to use on the platforms for which it is supported.

namespace

A namespace isolates specific system resources that are visible to all processes. Inside a namespace, only processes that are members of that namespace can see those resources.

node

A node is a worker machine in the OpenShift Container Platform cluster. A node is either a virtual machine (VM) or a physical machine.

OAuth client

OAuth client is used to get a bearer token.

OAuth server

The OpenShift Container Platform control plane includes a built-in OAuth server that determines the user's identity from the configured identity provider and creates an access token.

OpenID Connect

The OpenID Connect is a protocol to authenticate the users to use single sign-on (SSO) to access sites that use OpenID Providers.

passthrough mode

In passthrough mode, the Cloud Credential Operator (CCO) passes the provided cloud credential to the components that request cloud credentials.

pod

A pod is the smallest logical unit in Kubernetes. A pod consists of one or more containers to run in a worker node.

regular users

Users that are created automatically in the cluster upon first login or via the API.

request header

A request header is an HTTP header that is used to provide information about HTTP request context, so that the server can track the response of the request.

role-based access control (RBAC)

A key security control to ensure that cluster users and workloads have access to only the resources required to execute their roles.

service accounts

Service accounts are used by the cluster components or applications.

system users

Users that are created automatically when the cluster is installed.

users

Users is an entity that can make requests to API.

1.2. ABOUT AUTHENTICATION IN OPENSHIFT CONTAINER PLATFORM

To control access to an OpenShift Container Platform cluster, a cluster administrator can configure user authentication and ensure only approved users access the cluster.

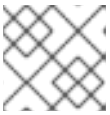
To interact with an OpenShift Container Platform cluster, users must first authenticate to the OpenShift Container Platform API in some way. You can authenticate by providing an OAuth access token or an X.509 client certificate in your requests to the OpenShift Container Platform API.

NOTE

If you do not present a valid access token or certificate, your request is unauthenticated and you receive an HTTP 401 error.

An administrator can configure authentication through the following tasks:

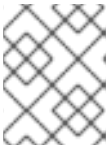
- Configuring an identity provider: You can define any supported identity provider in OpenShift Container Platform and add it to your cluster.
- Configuring the internal OAuth server: The OpenShift Container Platform control plane includes a built-in OAuth server that determines the user's identity from the configured identity provider and creates an access token. You can configure the token duration and inactivity timeout, and customize the internal OAuth server URL.

  NOTE

  Users can view and manage OAuth tokens owned by them .

- Registering an OAuth client: OpenShift Container Platform includes several default OAuth clients. You can register and configure additional OAuth clients .

  NOTE

  When users send a request for an OAuth token, they must specify either a default or custom OAuth client that receives and uses the token.

- Managing cloud provider credentials using the Cloud Credentials Operator: Cluster components use cloud provider credentials to get permissions required to perform cluster-related tasks.
- Impersonating a system admin user: You can grant cluster administrator permissions to a user by impersonating a system admin user .

1.3. ABOUT AUTHORIZATION IN OPENSHIFT CONTAINER PLATFORM

Authorization involves determining whether the identified user has permissions to perform the requested action.

Administrators can define permissions and assign them to users using the RBAC objects, such as rules, roles, and bindings. To understand how authorization works in OpenShift Container Platform, see Evaluating authorization.

You can also control access to an OpenShift Container Platform cluster through projects and namespaces.

Along with controlling user access to a cluster, you can also control the actions a pod can perform and the resources it can access using security context constraints (SCCs) .

You can manage authorization for OpenShift Container Platform through the following tasks:

- Viewing local and cluster roles and bindings.
- Creating a local role and assigning it to a user or group.
- Creating a cluster role and assigning it to a user or group: OpenShift Container Platform includes a set of default cluster roles. You can create additional cluster roles and add them to a user or group.
- Creating a cluster-admin user: By default, your cluster has only one cluster administrator called **kubeadmin**. You can create another cluster administrator. Before creating a cluster administrator, ensure that you have configured an identity provider.

NOTE

After creating the cluster admin user, delete the existing kubeadmin user to improve cluster security.

- Creating service accounts: Service accounts provide a flexible way to control API access without sharing a regular user's credentials. A user can create and use a service account in applications and also as an OAuth client.
- Scoping tokens: A scoped token is a token that identifies as a specific user who can perform only specific operations. You can create scoped tokens to delegate some of your permissions to another user or a service account.
- Syncing LDAP groups: You can manage user groups in one place by syncing the groups stored in an LDAP server with the OpenShift Container Platform user groups.

CHAPTER 2. UNDERSTANDING AUTHENTICATION

To interact with OpenShift Container Platform, log in so the authentication layer can verify your identity. The authorization layer then uses your identity to determine which actions and resources you can access.

As an administrator, you can configure authentication for OpenShift Container Platform.

2.1. USERS

A user in OpenShift Container Platform is an entity that makes API requests and can be granted permissions through role assignments. Users include regular users, system users for infrastructure components, and service accounts associated with projects.

Several types of users can exist:

User type	Description
Regular users	This is the way most interactive OpenShift Container Platform users are represented. Regular users are created automatically in the system upon first login or can be created via the API. Regular users are represented with the **User** object. Examples: **joe alice**
System users	Many of these are created automatically when the infrastructure is defined, mainly for the purpose of enabling the infrastructure to interact with the API securely. They include a cluster administrator (with access to everything), a per-node user, users for use by routers and registries, and various others. Finally, there is an **anonymous** system user that is used by default for unauthenticated requests. Examples: **system:admin system:openshift-registry system:node:node1.example.com**
Service accounts	These are special system users associated with projects; some are created automatically when the project is first created, while project administrators can create more for the purpose of defining access to the contents of each project. Service accounts are represented with the **ServiceAccount** object. Examples: **system:serviceaccount:default:deployer system:serviceaccount:foo:builder**

Each user must authenticate in some way to access OpenShift Container Platform. API requests with no authentication or invalid authentication are authenticated as requests by the **anonymous** system user. After authentication, policy determines what the user is authorized to do.

2.2. GROUPS

Groups represent sets of users and simplify authorization management by allowing administrators to grant permissions to multiple users simultaneously rather than individually. OpenShift Container Platform includes both explicitly defined groups and automatically provisioned virtual groups.

In addition to explicitly defined groups, there are also system groups, or *virtual groups*, that are automatically provisioned by the cluster.

The following default virtual groups are most important:

Virtual group	Description
system:authenticated	Automatically associated with all authenticated users.
system:authenticated:oauth	Automatically associated with all users authenticated with an OAuth access token.
system:unauthenticated	Automatically associated with all unauthenticated users.

2.3. API AUTHENTICATION

Requests to the OpenShift Container Platform API are authenticated using OAuth access tokens or X.509 client certificates, with invalid credentials rejected and anonymous requests assigned virtual user and group identities for authorization processing.

OAuth access tokens

- Obtained from the OpenShift Container Platform OAuth server using the ***<namespace_route>*/oauth/authorize** and ***<namespace_route>*/oauth/token** endpoints.
- Sent as an **Authorization: Bearer…** header.
- Sent as a websocket subprotocol header in the form **base64url.bearer.authorization.k8s.io.<base64url-encoded-token>** for websocket requests.

X.509 client certificates

- Requires an HTTPS connection to the API server.
- Verified by the API server against a trusted certificate authority bundle.
- The API server creates and distributes certificates to controllers to authenticate themselves.

Any request with an invalid access token or an invalid certificate is rejected by the authentication layer with a **401** error.

If no access token or certificate is presented, the authentication layer assigns the **system:anonymous** virtual user and the **system:unauthenticated** virtual group to the request. This allows the authorization layer to determine which requests, if any, an anonymous user is allowed to make.

2.3.1. OpenShift Container Platform OAuth server

The OpenShift Container Platform Control Plane includes a built-in OAuth server. Users obtain OAuth access tokens to authenticate themselves to the API.

When a person requests a new OAuth token, the OAuth server uses the configured identity provider to determine the identity of the person making the request.

It then determines what user that identity maps to, creates an access token for that user, and returns the token for use.

2.3.2. OAuth token requests

OpenShift Container Platform automatically creates OAuth clients to handle token requests from different user agents, including browser-based and CLI tools. These clients interact with OAuth endpoints to authenticate users through interactive login flows or WWW-Authenticate challenges.

The following OAuth clients are automatically created when starting the OpenShift Container Platform API:

OAuth client	Usage
openshift-browser-client	Requests tokens at **<namespace_route>/oauth/token/request** with a user-agent that can handle interactive logins. [1]
openshift-challenging-client	Requests tokens with a user-agent that can handle **WWW-Authenticate** challenges.

1. **<namespace_route>** refers to the namespace route. This is found by running the following command:

```
$ oc get route oauth-openshift -n openshift-authentication -o json | jq .spec.host
```

All requests for OAuth tokens involve a request to **<namespace_route>/oauth/authorize**. Most authentication integrations place an authenticating proxy in front of this endpoint, or configure OpenShift Container Platform to validate credentials against a backing identity provider. Requests to **<namespace_route>/oauth/authorize** can come from user-agents that cannot display interactive login pages, such as the CLI. Therefore, OpenShift Container Platform supports authenticating using a **WWW-Authenticate** challenge in addition to interactive login flows.

If an authenticating proxy is placed in front of the **<namespace_route>/oauth/authorize** endpoint, it sends unauthenticated, non-browser user-agents **WWW-Authenticate** challenges rather than displaying an interactive login page or redirecting to an interactive login flow.

NOTE

To prevent cross-site request forgery (CSRF) attacks against browser clients, only send Basic authentication challenges with if a **X-CSRF-Token** header is on the request. Clients that expect to receive Basic **WWW-Authenticate** challenges must set this header to a non-empty value.

If the authenticating proxy cannot support **WWW-Authenticate** challenges, or if OpenShift Container Platform is configured to use an identity provider that does not support WWW-Authenticate challenges, you must use a browser to manually obtain a token from **<namespace_route>/oauth/token/request**.

2.3.2.1. API impersonation

You can configure API requests in OpenShift Container Platform to act as another user. Impersonation allows you to perform actions on behalf of another account without switching credentials.

Additional resources

- User impersonation (Kubernetes documentation)

2.3.2.2. Authentication metrics for Prometheus

You can use Prometheus metrics to monitor login activity and troubleshoot authentication failures.

OpenShift Container Platform captures the following Prometheus metrics that track authentication attempts and outcomes for both the CLI and web console:

- **openshift_auth_basic_password_count** counts the number of **oc login** user name and password attempts.
- **openshift_auth_basic_password_count_result** counts the number of **oc login** user name and password attempts by result, **success** or **error**.
- **openshift_auth_form_password_count** counts the number of web console login attempts.
- **openshift_auth_form_password_count_result** counts the number of web console login attempts by result, **success** or **error**.
- **openshift_auth_password_total** counts the total number of **oc login** and web console login attempts.

CHAPTER 3. CONFIGURING THE INTERNAL OAUTH SERVER

The OpenShift Container Platform Control Plane includes a built-in OAuth server for user authentication. You can configure token duration, inactivity timeouts, and customize the OAuth server URL.

3.1. OPENSHIFT CONTAINER PLATFORM OAUTH SERVER

The OpenShift Container Platform Control Plane includes a built-in OAuth server. Users obtain OAuth access tokens to authenticate themselves to the API.

When a person requests a new OAuth token, the OAuth server uses the configured identity provider to determine the identity of the person making the request.

It then determines what user that identity maps to, creates an access token for that user, and returns the token for use.

3.2. OAUTH TOKEN REQUEST FLOWS AND RESPONSES

The OAuth server supports standard authorization code grant and implicit grant flows, with specific server responses for token requests using the implicit grant flow with WWW-Authenticate challenges.

When requesting an OAuth token using the implicit grant flow (**response_type=token**) with a client_id configured to request **WWW-Authenticate challenges** (like **openshift-challenging-client**), these are the possible server responses from **/oauth/authorize**, and how they should be handled:

Status	Content	Client response
302	**Location** header containing an **access_token** parameter in the URL fragment (RFC 6749 section 4.2.2)	Use the **access_token** value as the OAuth token.
302	**Location** header containing an **error** query parameter (RFC 6749 section 4.1.2.1)	Fail, optionally surfacing the **error** (and optional **error_description**) query values to the user.
302	Other **Location** header	Follow the redirect, and process the result using these rules.
401	**WWW-Authenticate** header present	Respond to challenge if type is recognized (e.g. **Basic**, **Negotiate**, etc), resubmit request, and process the result using these rules.
401	**WWW-Authenticate** header missing	No challenge authentication is possible. Fail and show response body (which might contain links or details on alternate methods to obtain an OAuth token).
Other	Other	Fail, optionally surfacing response body to the user.

Additional resources

- Authorization Code Grant
- Implicit Grant
- Access Token Response
- Error Response

3.3. OPTIONS FOR THE INTERNAL OAUTH SERVER

The internal OAuth server provides configuration options for token duration and grant strategies to control authentication behavior.

3.3.1. OAuth token duration options

The internal OAuth server generates two kinds of tokens:

Token	Description
Access tokens	Longer-lived tokens that grant access to the API.
Authorize codes	Short-lived tokens whose only use is to be exchanged for an access token.

You can configure the default duration for both types of token. If necessary, you can override the duration of the access token by using an **OAuthClient** object definition.

3.3.2. OAuth grant options

When the OAuth server receives token requests for a client to which the user has not previously granted permission, the action that the OAuth server takes is dependent on the OAuth client's grant strategy.

The OAuth client requesting token must provide its own grant strategy.

You can apply the following default methods:

Grant option	Description
auto	Auto-approve the grant and retry the request.
prompt	Prompt the user to approve or deny the grant.

3.4. CONFIGURING THE INTERNAL OAUTH SERVER'S TOKEN DURATION

Configure the internal OAuth server to extend or reduce access token validity beyond the default 24-hour lifetime.

IMPORTANT

By default, tokens are only valid for 24 hours. Existing sessions expire after this time elapses.

If the default time is insufficient, then this can be modified using the following procedure.

Procedure

1. Create a configuration file that contains the token duration options. The following file sets this to 48 hours, twice the default.

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  tokenConfig:
    accessTokenMaxAgeSeconds: 172800
```

where:

spec.tokenConfig.accessTokenMaxAgeSeconds

Specifies the lifetime of access tokens in seconds. The default lifetime is 24 hours, or 86400 seconds. This attribute cannot be negative. If set to zero, the default lifetime is used.

2. Apply the new configuration file:

NOTE

Because you update the existing OAuth server, you must use the **oc apply** command to apply the change.

```
$ oc apply -f </path/to/file.yaml>
```

3. Confirm that the changes are in effect:

```
$ oc describe oauth.config.openshift.io/cluster
```

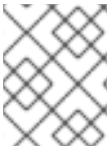
Example output

```
...
Spec:
  Token Config:
    Access Token Max Age Seconds:  172800
...
```

3.5. CONFIGURING TOKEN INACTIVITY TIMEOUT FOR THE INTERNAL OAUTH SERVER

Configure the internal OAuth server to automatically expire tokens after a set period of inactivity, improving security by invalidating idle sessions.

By default, no token inactivity timeout is set.



NOTE

If the token inactivity timeout is also configured in your OAuth client, that value overrides the timeout that is set in the internal OAuth server configuration.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have configured an identity provider (IDP).

Procedure

1. Update the **OAuth** configuration to set a token inactivity timeout.
   a. Edit the **OAuth** object:

```
$ oc edit oauth cluster
```

   Add the **spec.tokenConfig.accessTokenInactivityTimeout** field and set your timeout value:

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
...
spec:
  tokenConfig:
    accessTokenInactivityTimeout: 400s
```

   where:

   spec.tokenConfig.accessTokenInactivityTimeout

   Specifies the token inactivity timeout with appropriate units, for example **400s** for 400 seconds, or **30m** for 30 minutes. The minimum allowed timeout value is **300s**.

   b. Save the file to apply the changes.

2. Check that the OAuth server pods have restarted:

```
$ oc get clusteroperators authentication
```

   Do not continue to the next step until **PROGRESSING** is listed as **False**, as shown in the following output:

   ### Example output

```
NAME             VERSION   AVAILABLE   PROGRESSING   DEGRADED   SINCE
authentication   4.22.0    True        False         False      145m
```

3. Check that a new revision of the Kubernetes API server pods has rolled out. This will take several minutes.

```
$ oc get clusteroperators kube-apiserver
```

Do not continue to the next step until **PROGRESSING** is listed as **False**, as shown in the following output:

Example output

```
NAME             VERSION   AVAILABLE   PROGRESSING   DEGRADED   SINCE
kube-apiserver   4.22.0    True        False         False      145m
```

If **PROGRESSING** is showing **True**, wait a few minutes and try again.

Verification

1. Log in to the cluster with an identity from your IDP.
2. Execute a command and verify that it was successful.
3. Wait longer than the configured timeout without using the identity. In this procedure's example, wait longer than 400 seconds.
4. Try to execute a command from the same identity's session.
   This command should fail because the token should have expired due to inactivity longer than the configured timeout.

   Example output

```
error: You must be logged in to the server (Unauthorized)
```

3.6. CUSTOMIZING THE INTERNAL OAUTH SERVER URL

Customize the internal OAuth server URL to use a custom hostname and TLS certificate by configuring the cluster Ingress component routes.

WARNING

If you update the internal OAuth server URL, you might break trust from components in the cluster that need to communicate with the OpenShift Container Platform OAuth server to retrieve OAuth access tokens. Components that need to trust the OAuth server will need to include the proper CA bundle when calling OAuth endpoints. For example:

```
$ oc login -u <username> -p <password> --certificate-authority=<path_to_ca.crt>
```

+ For self-signed certificates, the **ca.crt** file must contain the custom CA certificate, otherwise the login will not succeed.

The Cluster Authentication Operator publishes the OAuth server's serving certificate in the **oauth-serving-cert** config map in the **openshift-config-managed** namespace. You can find the certificate in the **data.ca-bundle.crt** key of the config map.

Prerequisites

- You have logged in to the cluster as a user with administrative privileges.
- You have created a secret in the **openshift-config** namespace containing the TLS certificate and key. This is required if the domain for the custom hostname suffix does not match the cluster domain suffix. The secret is optional if the suffix matches.

  TIP

  You can create a TLS secret by using the **oc create secret tls** command.

Procedure

1. Edit the cluster **Ingress** configuration:

   ```
   $ oc edit ingress.config.openshift.io cluster
   ```

2. Set the custom hostname and optionally the serving certificate and key:

   ```
   apiVersion: config.openshift.io/v1
   kind: Ingress
   metadata:
     name: cluster
   spec:
     componentRoutes:
       - name: oauth-openshift
         namespace: openshift-authentication
         hostname: <custom_hostname>
         servingCertKeyPairSecret:
           name: <secret_name>
   ```

where:

spec.componentRoutes.hostname

Specifies the custom hostname for the OAuth server.

spec.componentRoutes.servingCertKeyPairSecret.name

Specifies the name of a secret in the **openshift-config** namespace that contains a TLS certificate (**tls.crt**) and key (**tls.key**). This is required if the domain for the custom hostname suffix does not match the cluster domain suffix. The secret is optional if the suffix matches.

3. Save the file to apply the changes.

3.7. OAUTH SERVER METADATA

Applications running in the cluster can query the OAuth 2.0 Authorization Server Metadata endpoint to dynamically discover OAuth server endpoints, supported scopes, and grant types for automated client configuration.

Any application running inside the cluster can issue a **GET** request to ***https://openshift.default.svc/.well-known/oauth-authorization-server*** to fetch the following information:

```
{
  "issuer": "https://<namespace_route>",
  "authorization_endpoint": "https://<namespace_route>/oauth/authorize",
  "token_endpoint": "https://<namespace_route>/oauth/token",
  "scopes_supported": [
    "user:full",
    "user:info",
    "user:check-access",
    "user:list-scoped-projects",
    "user:list-projects"
  ],
  "response_types_supported": [
    "code",
    "token"
  ],
  "grant_types_supported": [
    "authorization_code",
    "implicit"
  ],
  "code_challenge_methods_supported": [
    "plain",
    "S256"
  ]
}
```

+ where:

issuer

Specifies the authorization server's issuer identifier, which is a URL that uses the **https** scheme and has no query or fragment components. This is the location where **.well-known** RFC 5785 resources containing information about the authorization server are published.

authorization_endpoint

Specifies the URL of the authorization server's authorization endpoint.

token_endpoint

Specifies the URL of the authorization server's token endpoint.

scopes_supported

Specifies a JSON array containing a list of the OAuth 2.0 RFC 6749 scope values that this authorization server supports. Note that not all supported scope values are advertised.

response_types_supported

Specifies a JSON array containing a list of the OAuth 2.0 **response_type** values that this authorization server supports. The array values used are the same as those used with the **response_types** parameter defined by OAuth 2.0 Dynamic Client Registration Protocol in RFC 7591.

grant_types_supported

Specifies a JSON array containing a list of the OAuth 2.0 grant type values that this authorization server supports. The array values used are the same as those used with the **grant_types** parameter defined by OAuth 2.0 Dynamic Client Registration Protocol in RFC 7591.

code_challenge_methods_supported

Specifies a JSON array containing a list of PKCE RFC 7636 code challenge methods supported by this authorization server. Code challenge method values are used in the **code_challenge_method** parameter defined in Section 4.3 of RFC 7636. The valid code challenge method values are those registered in the IANA PKCE Code Challenge Methods registry.

Additional resources

- OAuth 2.0 Authorization Server Metadata
- RFC 5785 - Defining Well-Known Uniform Resource Identifiers
- RFC 6749 - The OAuth 2.0 Authorization Framework
- RFC 7591 - OAuth 2.0 Dynamic Client Registration Protocol
- RFC 7636 - Proof Key for Code Exchange by OAuth Public Clients
- RFC 7636 Section 4.3 - Client Creates a Code Challenge
- IANA OAuth Parameters

3.8. TROUBLESHOOTING OAUTH API EVENTS

Use service account event messages to diagnose OAuth configuration issues when the API server returns **unexpected condition** errors that are otherwise difficult to debug.

In some cases the API server returns an **unexpected condition** error message that is difficult to debug without direct access to the API master log. The underlying reason for the error is purposely obscured in order to avoid providing an unauthenticated user with information about the server's state.

A subset of these errors is related to service account OAuth configuration issues. These issues are captured in events that can be viewed by non-administrator users. When encountering an **unexpected condition** server error during OAuth, run **oc get events** to view these events under **ServiceAccount**.

The following example warns of a service account that is missing a proper OAuth redirect URI:

```
$ oc get events | grep ServiceAccount
```

Example output

```
1m 1m 1 proxy ServiceAccount Warning NoSAOAuthRedirectURIs service-account-oauth-client-getter system:serviceaccount:myproject:proxy has no redirectURIs; set serviceaccounts.openshift.io/oauth-redirecturi.<some-value>=<redirect> or create a dynamic URI using serviceaccounts.openshift.io/oauth-redirectreference.<some-value>=<reference>
```

Running **oc describe sa/<service_account_name>** reports any OAuth events associated with the given service account name.

```
$ oc describe sa/proxy | grep -A5 Events
```

Example output

```
Events:
  FirstSeen LastSeen Count From SubObjectPath Type Reason Message
  --------- -------- ----- ---- ------------- -------- ------ -------
  3m 3m 1 service-account-oauth-client-getter Warning NoSAOAuthRedirectURIs system:serviceaccount:myproject:proxy has no redirectURIs; set serviceaccounts.openshift.io/oauth-redirecturi.<some-value>=<redirect> or create a dynamic URI using serviceaccounts.openshift.io/oauth-redirectreference.<some-value>=<reference>
```

The following is a list of the possible event errors:

No redirect URI annotations or an invalid URI is specified

```
Reason Message
NoSAOAuthRedirectURIs system:serviceaccount:myproject:proxy has no redirectURIs; set serviceaccounts.openshift.io/oauth-redirecturi.<some-value>=<redirect> or create a dynamic URI using serviceaccounts.openshift.io/oauth-redirectreference.<some-value>=<reference>
```

Invalid route specified

```
Reason Message
NoSAOAuthRedirectURIs [routes.route.openshift.io "<name>" not found, system:serviceaccount:myproject:proxy has no redirectURIs; set serviceaccounts.openshift.io/oauth-redirecturi.<some-value>=<redirect> or create a dynamic URI using serviceaccounts.openshift.io/oauth-redirectreference.<some-value>=<reference>]
```

Invalid reference type specified

```
Reason Message
NoSAOAuthRedirectURIs [no kind "<name>" is registered for version "v1", system:serviceaccount:myproject:proxy has no redirectURIs; set serviceaccounts.openshift.io/oauth-redirecturi.<some-value>=<redirect> or create a dynamic URI using serviceaccounts.openshift.io/oauth-redirectreference.<some-value>=<reference>]
```

Missing SA tokens

```
Reason                Message
NoSAOAuthTokens       system:serviceaccount:myproject:proxy has no tokens
```

CHAPTER 4. CONFIGURING OAUTH CLIENTS

OpenShift Container Platform includes default OAuth clients for platform authentication. You can register additional OAuth clients to integrate third-party applications and configure token inactivity timeouts to enhance security.

4.1. DEFAULT OAUTH CLIENTS

OpenShift Container Platform automatically creates OAuth clients for browser-based logins, CLI authentication, and challenge-based authentication when the API starts.

The following OAuth clients are created:

OAuth client	Usage
openshift-browser-client	Requests tokens at **<namespace_route>/oauth/token/request** with a user-agent that can handle interactive logins.
openshift-challenging-client	Requests tokens with a user-agent that can handle **WWW-Authenticate** challenges.
openshift-cli-client	Requests tokens by using a local HTTP server fetching an authorization code grant.

where:

<namespace_route>

Specifies the namespace route. Find this value by running the following command:

```
$ oc get route oauth-openshift -n openshift-authentication -o json | jq .spec.host
```

4.2. REGISTERING AN ADDITIONAL OAUTH CLIENT

Register additional OAuth clients to manage authentication for applications that need to interact with your OpenShift Container Platform cluster.

Procedure

- To register additional OAuth clients:

```
$ oc create -f <(echo '
kind: OAuthClient
apiVersion: oauth.openshift.io/v1
metadata:
 name: demo
secret: "..."
redirectURIs:
```

```
 - "http://www.example.com/"
grantMethod: prompt
')
```

where:

metadata.name

Specifies the OAuth client name. This value is used as the **client_id** parameter when making requests to **<namespace_route>/oauth/authorize** and **<namespace_route>/oauth/token**.

secret

Specifies the secret value used as the **client_secret** parameter when making requests to **<namespace_route>/oauth/token**.

redirectURIs

Specifies the list of valid redirect URIs. The **redirect_uri** parameter specified in requests to **<namespace_route>/oauth/authorize** and **<namespace_route>/oauth/token** must be equal to or prefixed by one of these URIs.

grantMethod

Specifies the action to take when this client requests tokens and has not yet been granted access by the user. Use **auto** to automatically approve the grant and retry the request, or **prompt** to prompt the user to approve or deny the grant.

4.3. CONFIGURING TOKEN INACTIVITY TIMEOUT FOR AN OAUTH CLIENT

Configure OAuth clients to expire tokens after a set period of inactivity, improving security by automatically invalidating idle sessions.

By default, no token inactivity timeout is set.

NOTE

If the token inactivity timeout is also configured in the internal OAuth server configuration, the timeout that is set in the OAuth client overrides that value.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have configured an identity provider (IDP).

Procedure

- Update the **OAuthClient** configuration to set a token inactivity timeout.
  a. Edit the **OAuthClient** object:

```
$ oc edit oauthclient <oauth_client>
```

Replace **<oauth_client>** with the OAuth client to configure, for example, **console**.

Add the **accessTokenInactivityTimeoutSeconds** field and set your timeout value:

```
apiVersion: oauth.openshift.io/v1
grantMethod: auto
kind: OAuthClient
metadata:
...
accessTokenInactivityTimeoutSeconds: 600
```

where:

accessTokenInactivityTimeoutSeconds

Specifies the token inactivity timeout in seconds. The minimum allowed value is **300**.

b. Save the file to apply the changes.

Verification

1. Log in to the cluster with an identity from your IDP. Be sure to use the OAuth client that you just configured.
2. Perform an action and verify that it was successful.
3. Wait longer than the configured timeout without using the identity. In this procedure's example, wait longer than 600 seconds.
4. Try to perform an action from the same identity's session.
   This attempt should fail because the token should have expired due to inactivity longer than the configured timeout.

4.4. ADDITIONAL RESOURCES

- OAuthClient [oauth.openshift.io/v1]

CHAPTER 5. MANAGING USER-OWNED OAUTH ACCESS TOKENS

Review and manage your user-owned OAuth access tokens to monitor active sessions, verify token scopes, and revoke tokens that are no longer needed.

5.1. LISTING USER-OWNED OAUTH ACCESS TOKENS

List your user-owned OAuth access tokens to review active sessions, check token expiration, and identify tokens associated with specific OAuth clients.

Token names are not sensitive and cannot be used to log in.

Procedure

- List all user-owned OAuth access tokens:

```
$ oc get useroauthaccesstokens
```

Example output

```
NAME      CLIENT NAME                CREATED              EXPIRES
REDIRECT URI                                           SCOPES
<token1>  openshift-challenging-client  2021-01-11T19:25:35Z  2021-01-12 19:25:35
+0000 UTC  https://oauth-openshift.apps.example.com/oauth/token/implicit      user:full
<token2>  openshift-browser-client      2021-01-11T19:27:06Z  2021-01-12 19:27:06 +0000
UTC  https://oauth-openshift.apps.example.com/oauth/token/display      user:full
<token3>  console                  2021-01-11T19:26:29Z  2021-01-12 19:26:29 +0000 UTC
https://console-openshift-console.apps.example.com/auth/callback  user:full
```

- List user-owned OAuth access tokens for a particular OAuth client:

```
$ oc get useroauthaccesstokens --field-selector=clientName="console"
```

Example output

```
NAME      CLIENT NAME                CREATED              EXPIRES
REDIRECT URI                                           SCOPES
<token3>  console                  2021-01-11T19:26:29Z  2021-01-12 19:26:29 +0000 UTC
https://console-openshift-console.apps.example.com/auth/callback  user:full
```

5.2. VIEWING THE DETAILS OF A USER-OWNED OAUTH ACCESS TOKEN

View details of a user-owned OAuth access token to identify the associated client application, check expiration and inactivity timeouts, verify scopes, and see other information fields.

Procedure

- Describe the details of a user-owned OAuth access token:

```
$ oc describe useroauthaccesstokens <token_name>
```

Example output

```
Name:                         <token_name>
Namespace:
Labels:                       <none>
Annotations:                  <none>
API Version:                  oauth.openshift.io/v1
Authorize Token:              sha256~Ksckkug-9Fg_RWn_AUysPoIg-_HqmFI9zUL_CgD8wr8
Client Name:                  openshift-browser-client
Expires In:                   86400
Inactivity Timeout Seconds:  317
Kind:                         UserOAuthAccessToken
Metadata:
  Creation Timestamp:  2021-01-11T19:27:06Z
  Managed Fields:
    API Version:  oauth.openshift.io/v1
    Fields Type:  FieldsV1
    fieldsV1:
      f:authorizeToken:
      f:clientName:
      f:expiresIn:
      f:redirectURI:
      f:scopes:
      f:userName:
      f:userUID:
    Manager:         oauth-server
    Operation:       Update
    Time:            2021-01-11T19:27:06Z
  Resource Version:  30535
  Self Link:         /apis/oauth.openshift.io/v1/useroauthaccesstokens/<token_name>
  UID:               f9d00b67-ab65-489b-8080-e427fa3c6181
Redirect URI:        https://oauth-openshift.apps.example.com/oauth/token/display
Scopes:
  user:full
User Name:  <user_name>
User UID:   82356ab0-95f9-4fb3-9bc0-10f1d6a6a345
Events:     <none>
```

where:

Name

Specifies the token name, which is the sha256 hash of the token. Token names are not sensitive and cannot be used to log in.

Client Name

Specifies the client name, which describes where the token originated from.

Expires In

Specifies the value in seconds from the creation time before this token expires.

Inactivity Timeout Seconds

If there is a token inactivity timeout set for the OAuth server, this specifies the value in seconds from the creation time before this token can no longer be used.

Scopes

Specifies the scopes for this token.

User Name

Specifies the user name associated with this token.

5.3. DELETING USER-OWNED OAUTH ACCESS TOKENS

You can use the following procedure to delete any user-owned OAuth tokens that are no longer needed.

The **oc logout** command only invalidates the OAuth token for the active session. Deleting an OAuth access token logs out the user from all sessions that use the token.

Procedure

- Delete the user-owned OAuth access token:

```
$ oc delete useroauthaccesstokens <token_name>
```

Example output

```
useroauthaccesstoken.oauth.openshift.io "<token_name>" deleted
```

5.4. ADDING UNAUTHENTICATED GROUPS TO CLUSTER ROLES

Grant unauthenticated users access to specific cluster roles to enable features that require cluster access without authentication, such as external webhooks or automated token management.

You can add unauthenticated users to the following cluster roles:

- **system:scope-impersonation**
- **system:webhook**
- **system:oauth-token-deleter**
- **self-access-reviewer**

IMPORTANT

Always verify compliance with your organization's security standards when modifying unauthenticated access.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Create a YAML file named **add-<cluster_role>-unauth.yaml** and add the following content:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 annotations:
   rbac.authorization.kubernetes.io/autoupdate: "true"
 name: <cluster_role>access-unauthenticated
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: <cluster_role>
subjects:
 - apiGroup: rbac.authorization.k8s.io
   kind: Group
   name: system:unauthenticated
```

2. Apply the configuration by running the following command:

```
$ oc apply -f add-<cluster_role>.yaml
```

CHAPTER 6. UNDERSTANDING IDENTITY PROVIDER CONFIGURATION

As an administrator, you can configure OAuth to specify an identity provider after you install your cluster. Developers and administrators obtain OAuth access tokens to authenticate themselves to the API.

The OpenShift Container Platform master includes a built-in OAuth server.

6.1. IDENTITY PROVIDERS IN OPENSHIFT CONTAINER PLATFORM

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.

NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

6.2. SUPPORTED IDENTITY PROVIDERS

You can configure the following types of identity providers:

Identity provider	Description
htpasswd	Configure the **htpasswd** identity provider to validate user names and passwords against a flat file generated using **htpasswd**.
Keystone	Configure the **keystone** identity provider to integrate your OpenShift Container Platform cluster with Keystone to enable shared authentication with an OpenStack Keystone v3 server configured to store users in an internal database.
LDAP	Configure the **ldap** identity provider to validate user names and passwords against an LDAPv3 server, using simple bind authentication.
Basic authentication	Configure a **basic-authentication** identity provider for users to log in to OpenShift Container Platform with credentials validated against a remote identity provider. Basic authentication is a generic backend integration mechanism.
Request header	Configure a **request-header** identity provider to identify users from request header values, such as **X-Remote-User**. It is typically used in combination with an authenticating proxy, which sets the request header value.
GitHub or GitHub Enterprise	Configure a **github** identity provider to validate user names and passwords against GitHub or GitHub Enterprise's OAuth authentication server.
GitLab	Configure a **gitlab** identity provider to use GitLab.com or any other GitLab instance as an identity provider.

Identity provider	Description
Google	Configure a **google** identity provider using Google's OpenID Connect integration.
OpenID Connect	Configure an **oidc** identity provider to integrate with an OpenID Connect identity provider using an Authorization Code Flow.

Once an identity provider has been defined, you can use RBAC to define and apply permissions .

6.3. REMOVING THE KUBEADMIN USER

After you define an identity provider and create a new **cluster-admin** user, you can remove the **kubeadmin** to improve cluster security.

WARNING

If you follow this procedure before another user is a **cluster-admin**, then OpenShift Container Platform must be reinstalled. It is not possible to undo this command.

Prerequisites

- You must have configured at least one identity provider.
- You must have added the **cluster-admin** role to a user.
- You must be logged in as an administrator.

Procedure

- Remove the **kubeadmin** secrets:

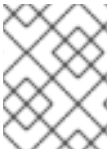
```
$ oc delete secrets kubeadmin -n kube-system
```

6.4. IDENTITY PROVIDER PARAMETERS

The following parameters are common to all identity providers:

Parameter	Description
name	The provider name is prefixed to provider user names to form an identity name.

Parameter	Description
mappingMethod	Defines how new identities are mapped to users when they log in. Enter one of the following values: **claim** The default value. Provisions a user with the identity's preferred user name. Fails if a user with that user name is already mapped to another identity. **lookup** Looks up an existing identity, user identity mapping, and user, but does not automatically provision users or identities. This allows cluster administrators to set up identities and users manually, or using an external process. Using this method requires you to manually provision users. **add** Provisions a user with the identity's preferred user name. If a user with that user name already exists, the identity is mapped to the existing user, adding to any existing identity mappings for the user. Required when multiple identity providers are configured that identify the same set of users and map to the same user names.



NOTE

When adding or changing identity providers, you can map identities from the new provider to existing users by setting the **mappingMethod** parameter to **add**.

6.5. SAMPLE IDENTITY PROVIDER CR

You can use a custom resource (CR) to see the parameters and default values that you use to configure an identity provider.

The following example uses the htpasswd identity provider.

Sample identity provider CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: my_identity_provider
    mappingMethod: claim
    type: HTPasswd
    htpasswd:
      fileData:
        name: htpass-secret
```

where:

spec.identityProviders.name

Specifies the provider name, which is prefixed to provider user names to form an identity name.

spec.identityProviders.mappingMethod

Specifies how mappings are established between this provider's identities and **User** objects.

spec.identityProviders.htpasswd.fileData.name

Specifies an existing secret containing a file generated using **htpasswd**.

6.6. MANUALLY PROVISIONING A USER WHEN USING THE LOOKUP MAPPING METHOD

You can manually provision users when the **lookup** mapping method is enabled. The **lookup** method disables automatic identity-to-user mapping during login, requiring manual provisioning of each user after configuring the identity provider.

Prerequisites

- You have installed the OpenShift CLI (**oc**).

Procedure

1. Create an OpenShift Container Platform user:

```
$ oc create user <username>
```

2. Create an OpenShift Container Platform identity:

```
$ oc create identity <identity_provider>:<identity_provider_user_id>
```

Where **<identity_provider_user_id>** is a name that uniquely represents the user in the identity provider.

3. Create a user identity mapping for the created user and identity:

```
$ oc create useridentitymapping <identity_provider>:<identity_provider_user_id>
<username>
```

Additional resources

- How to create user, identity and map user and identity in LDAP authentication for **mappingMethod** as **lookup** inside the OAuth manifest
- How to create user, identity and map user and identity in OIDC authentication for **mappingMethod** as **lookup**

CHAPTER 7. CONFIGURING IDENTITY PROVIDERS

7.1. CONFIGURING AN HTPASSWD IDENTITY PROVIDER

Configure the **htpasswd** identity provider to allow users to log in to OpenShift Container Platform with credentials from an htpasswd file.

To define an htpasswd identity provider, perform the following tasks:

1. Create an **htpasswd** file to store the user and password information.
2. Create a secret to represent the **htpasswd** file.
3. Define an htpasswd identity provider resource that references the secret.
4. Apply the resource to the default OAuth configuration to add the identity provider.

7.1.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.

NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

7.1.2. About htpasswd authentication

Using htpasswd authentication in OpenShift Container Platform allows you to identify users based on an htpasswd file. An htpasswd file is a flat file that contains the user name and hashed password for each user. You can use the **htpasswd** utility to create this file.

WARNING

Do not use htpasswd authentication in OpenShift Container Platform for production environments. Use htpasswd authentication only for development environments.

7.1.3. Creating the htpasswd file

See one of the following sections for instructions about how to create the htpasswd file:

- Creating an htpasswd file using Linux
- Creating an htpasswd file using Windows

7.1.3.1. Creating an htpasswd file using Linux

To use the htpasswd identity provider, you must generate a flat file that contains the user names and passwords for your cluster by using **htpasswd**.

Prerequisites

- Have access to the **htpasswd** utility. On Red Hat Enterprise Linux this is available by installing the **httpd-tools** package.

Procedure

1. Create or update your flat file with a user name and hashed password:

```
$ htpasswd -c -B -b </path/to/users.htpasswd> <username> <password>
```

The command generates a hashed version of the password.

For example:

```
$ htpasswd -c -B -b users.htpasswd <username> <password>
```

Example output

```
Adding password for user user1
```

2. Continue to add or update credentials to the file:

```
$ htpasswd -B -b </path/to/users.htpasswd> <user_name> <password>
```

7.1.3.2. Creating an htpasswd file using Windows

To use the htpasswd identity provider, you must generate a flat file that contains the user names and passwords for your cluster by using **htpasswd**.

Prerequisites

- Have access to **htpasswd.exe**. This file is included in the **\bin** directory of many Apache httpd distributions.

Procedure

1. Create or update your flat file with a user name and hashed password:

```
> htpasswd.exe -c -B -b <\path\to\users.htpasswd> <username> <password>
```

The command generates a hashed version of the password.

For example:

```
> htpasswd.exe -c -B -b users.htpasswd <username> <password>
```

Example output

```
Adding password for user user1
```

2. Continue to add or update credentials to the file:

```
> htpasswd.exe -b <\path\to\users.htpasswd> <username> <password>
```

7.1.4. Creating the htpasswd secret

To use the htpasswd identity provider, you must define a secret that contains the htpasswd user file.

Prerequisites

- Create an htpasswd file.

Procedure

- Create a **Secret** object that contains the htpasswd users file:

```
$ oc create secret generic htpass-secret --from-file=htpasswd=<path_to_users.htpasswd> -n
openshift-config 1
```

1 The secret key containing the users file for the **--from-file** argument must be named **htpasswd**, as shown in the above command.

TIP

You can alternatively apply the following YAML to create the secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: htpass-secret
  namespace: openshift-config
type: Opaque
data:
  htpasswd: <base64_encoded_htpasswd_file_contents>
```

7.1.5. Sample htpasswd CR

The following custom resource (CR) shows the parameters and acceptable values for an htpasswd identity provider.

htpasswd CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: my_htpasswd_provider 1
```

```
  mappingMethod: claim 2
  type: HTPasswd
  htpasswd:
    fileData:
      name: htpass-secret 3
```

1. This provider name is prefixed to provider user names to form an identity name.
2. Controls how mappings are established between this provider's identities and **User** objects.
3. An existing secret containing a file generated using **htpasswd**.

Additional resources

- See Identity provider parameters for information on parameters, such as **mappingMethod**, that are common to all identity providers.

7.1.6. Adding an identity provider to your cluster

Apply the identity provider custom resource (CR) to your cluster so users can authenticate with the configured identity provider.

Prerequisites

- You installed an OpenShift Container Platform cluster.
- You defined the CR for your identity provider.
- You are logged in as an administrator.

Procedure

1. Apply the defined CR by running the following command:

```
$ oc apply -f </path/to/CR>
```

NOTE

If a CR does not exist, **oc apply** creates a new CR and might trigger the following warning: **Warning: oc apply should be used on resources created by either oc create --save-config or oc apply**. In this case you can safely ignore this warning.

2. Log in to the cluster with credentials from the configured identity provider by running the following command. Enter the password when prompted:

```
$ oc login -u <username>
```

3. Confirm that the user logged in successfully and that the username displays by running the following command:

```
$ oc whoami
```

-

7.1.7. Updating users for an htpasswd identity provider

You can add or remove users from an existing htpasswd identity provider.

Prerequisites

- You have created a **Secret** object that contains the htpasswd user file. This procedure assumes that it is named **htpass-secret**.
- You have configured an htpasswd identity provider. This procedure assumes that it is named **my_htpasswd_provider**.
- You have access to the **htpasswd** utility. On Red Hat Enterprise Linux this is available by installing the **httpd-tools** package.
- You have cluster administrator privileges.

Procedure

1. Retrieve the htpasswd file from the **htpass-secret Secret** object and save the file to your file system:

```
$ oc get secret htpass-secret -ojsonpath={.data.htpasswd} -n openshift-config | base64 --
decode > users.htpasswd
```

2. Add or remove users from the **users.htpasswd** file.
   - To add a new user:

```
$ htpasswd -bB users.htpasswd <username> <password>
```

     Example output

```
Adding password for user <username>
```

   - To remove an existing user:

```
$ htpasswd -D users.htpasswd <username>
```

     Example output

```
Deleting password for user <username>
```

3. Replace the **htpass-secret Secret** object with the updated users in the **users.htpasswd** file:

```
$ oc create secret generic htpass-secret --from-file=htpasswd=users.htpasswd --dry-
run=client -o yaml -n openshift-config | oc replace -f -
```

TIP

You can alternatively apply the following YAML to replace the secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: htpass-secret
  namespace: openshift-config
type: Opaque
data:
  htpasswd: <base64_encoded_htpasswd_file_contents>
```

4. If you removed one or more users, you must additionally remove existing resources for each user.

   a. Delete the **User** object:

   ```
   $ oc delete user <username>
   ```

   Example output

   ```
   user.user.openshift.io "<username>" deleted
   ```

   Be sure to remove the user, otherwise the user can continue using their token as long as it has not expired.

   b. Delete the **Identity** object for the user:

   ```
   $ oc delete identity my_htpasswd_provider:<username>
   ```

   Example output

   ```
   identity.user.openshift.io "my_htpasswd_provider:<username>" deleted
   ```

7.1.8. Configuring identity providers using the web console

Configure your identity provider (IDP) through the web console instead of the CLI.

Prerequisites

- You must be logged in to the web console as a cluster administrator.

Procedure

1. Navigate to **Administration → Cluster Settings**.
2. Under the **Configuration** tab, click **OAuth**.
3. Under the **Identity Providers** section, select your identity provider from the **Add** drop-down menu.

NOTE

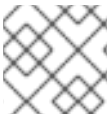
You can specify multiple IDPs through the web console without overwriting existing IDPs.

7.2. CONFIGURING A KEYSTONE IDENTITY PROVIDER

Configure the **keystone** identity provider to integrate your OpenShift Container Platform cluster with Keystone to enable shared authentication with an OpenStack Keystone v3 server configured to store users in an internal database. This configuration allows users to log in to OpenShift Container Platform with their Keystone credentials.

7.2.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.



NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

7.2.2. About Keystone authentication

Keystone is an OpenStack project that provides identity, token, catalog, and policy services.

You can configure the integration with Keystone so that the new OpenShift Container Platform users are based on either the Keystone user names or unique Keystone IDs. With both methods, users log in by entering their Keystone user name and password. Basing the OpenShift Container Platform users on the Keystone ID is more secure because if you delete a Keystone user and create a new Keystone user with that user name, the new user might have access to the old user's resources.

7.2.3. Creating the secret

You can create a TLS **Secret** object in the **openshift-config** namespace by using the **oc** CLI or by applying a YAML file to store client certificates and keys that identity providers require for secure communication.

Procedure

1. Create a **Secret** object that contains the key and certificate by running the following command:

```
$ oc create secret tls <secret_name> --key=key.pem --cert=cert.pem -n openshift-config
```

2. Optional: Apply the following YAML to create the secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: <secret_name>
  namespace: openshift-config
type: kubernetes.io/tls
```

```
data:
  tls.crt: <base64_encoded_cert>
  tls.key: <base64_encoded_key>
```

7.2.4. Creating a 'ConfigMap'

Create a **ConfigMap** object in the **openshift-config** namespace to store the certificate authority bundle that identity providers use to validate secure connections to the remote authentication service.

Procedure

1. Define an OpenShift Container Platform **ConfigMap** object containing the certificate authority by running the following command:

```
$ oc create configmap ca-config-map --from-file=ca.crt=/path/to/ca -n openshift-config
```

2. Optional: Apply the following YAML to create the config map:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ca-config-map
  namespace: openshift-config
data:
  ca.crt: |
    <CA_certificate_PEM>
```

The certificate authority must be stored in the **ca.crt** key of the **ConfigMap** object.

7.2.5. Sample Keystone CR

The following custom resource (CR) shows the parameters and acceptable values for a Keystone identity provider.

Keystone CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: keystoneidp 1
    mappingMethod: claim 2
    type: Keystone
    keystone:
      domainName: default 3
      url: https://keystone.example.com:5000 4
      ca: 5
        name: ca-config-map
      tlsClientCert: 6
```

```
    name: client-cert-secret
  tlsClientKey: 7
    name: client-key-secret
```

1. This provider name is prefixed to provider user names to form an identity name.
2. Controls how mappings are established between this provider's identities and **User** objects.
3. Keystone domain name. In Keystone, usernames are domain-specific. Only a single domain is supported.
4. The URL to use to connect to the Keystone server (required). This must use https.
5. Optional: Reference to an OpenShift Container Platform **ConfigMap** object containing the PEM-encoded certificate authority bundle to use in validating server certificates for the configured URL.
6. Optional: Reference to an OpenShift Container Platform **Secret** object containing the client certificate to present when making requests to the configured URL.
7. Reference to an OpenShift Container Platform **Secret** object containing the key for the client certificate. Required if **tlsClientCert** is specified.

Additional resources

- See Identity provider parameters for information on parameters, such as **mappingMethod**, that are common to all identity providers.

7.2.6. Adding an identity provider to your cluster

Apply the identity provider custom resource (CR) to your cluster so users can authenticate with the configured identity provider.

Prerequisites

- You installed an OpenShift Container Platform cluster.
- You defined the CR for your identity provider.
- You are logged in as an administrator.

Procedure

1. Apply the defined CR by running the following command:

```
$ oc apply -f </path/to/CR>
```

NOTE

If a CR does not exist, **oc apply** creates a new CR and might trigger the following warning: **Warning: oc apply should be used on resources created by either oc create --save-config or oc apply**. In this case you can safely ignore this warning.

2. Log in to the cluster with credentials from the configured identity provider by running the following command. Enter the password when prompted:

```
$ oc login -u <username>
```

3. Confirm that the user logged in successfully and that the username displays by running the following command:

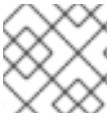
```
$ oc whoami
```

7.3. CONFIGURING AN LDAP IDENTITY PROVIDER

Configure the **ldap** identity provider to validate user names and passwords against an LDAPv3 server, using simple bind authentication.

7.3.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.



NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

7.3.2. About LDAP authentication

During authentication, the LDAP directory is searched for an entry that matches the provided user name. If a single unique match is found, a simple bind is attempted using the distinguished name (DN) of the entry plus the provided password.

These are the steps taken:

1. Generate a search filter by combining the attribute and filter in the configured **url** with the user-provided user name.
2. Search the directory using the generated filter. If the search does not return exactly one entry, deny access.
3. Attempt to bind to the LDAP server using the DN of the entry retrieved from the search, and the user-provided password.
4. If the bind is unsuccessful, deny access.
5. If the bind is successful, build an identity using the configured attributes as the identity, email address, display name, and preferred user name.

The configured **url** is an RFC 2255 URL, which specifies the LDAP host and search parameters to use. The syntax of the URL is:

```
ldap://host:port/basedn?attribute?scope?filter
```

For this URL:

URL component	Description
ldap	For regular LDAP, use the string **ldap**. For secure LDAP (LDAPS), use **ldaps** instead.
host:port	The name and port of the LDAP server. Defaults to **localhost:389** for ldap and **localhost:636** for LDAPS.
basedn	The DN of the branch of the directory where all searches should start from. At the very least, this must be the top of your directory tree, but it could also specify a subtree in the directory.
attribute	The attribute to search for. Although RFC 2255 allows a comma-separated list of attributes, only the first attribute will be used, no matter how many are provided. If no attributes are provided, the default is to use **uid**. It is recommended to choose an attribute that will be unique across all entries in the subtree you will be using.
scope	The scope of the search. Can be either **one** or **sub**. If the scope is not provided, the default is to use a scope of **sub**.
filter	A valid LDAP search filter. If not provided, defaults to **(objectClass=*)**

When doing searches, the attribute, filter, and provided user name are combined to create a search filter that looks like:

```
(&(<filter>)(<attribute>=<username>))
```

For example, consider a URL of:

```
ldap://ldap.example.com/o=Acme?cn?sub?(enabled=true)
```

When a client attempts to connect using a user name of **bob**, the resulting search filter will be **(&(enabled=true)(cn=bob))**.

If the LDAP directory requires authentication to search, specify a **bindDN** and **bindPassword** to use to perform the entry search.

7.3.3. Creating the LDAP secret

To use the identity provider, you must define an OpenShift Container Platform **Secret** object that contains the **bindPassword** field.

Procedure

- Create a **Secret** object that contains the **bindPassword** field:

```
$ oc create secret generic ldap-secret --from-literal=bindPassword=<secret> -n openshift-config
```
(1)

(1) The secret key containing the bindPassword for the **--from-literal** argument must be called **bindPassword**.

TIP

You can alternatively apply the following YAML to create the secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: ldap-secret
  namespace: openshift-config
type: Opaque
data:
  bindPassword: <base64_encoded_bind_password>
```

7.3.4. Creating a 'ConfigMap'

Create a **ConfigMap** object in the **openshift-config** namespace to store the certificate authority bundle that identity providers use to validate secure connections to the remote authentication service.

Procedure

1. Define an OpenShift Container Platform **ConfigMap** object containing the certificate authority by running the following command:

```
$ oc create configmap ca-config-map --from-file=ca.crt=/path/to/ca -n openshift-config
```

2. Optional: Apply the following YAML to create the config map:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ca-config-map
  namespace: openshift-config
data:
  ca.crt: |
    <CA_certificate_PEM>
```

The certificate authority must be stored in the **ca.crt** key of the **ConfigMap** object.

7.3.5. Sample LDAP CR

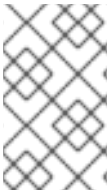
The following custom resource (CR) shows the parameters and acceptable values for an LDAP identity provider.

LDAP CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: ldapidp 1
    mappingMethod: claim 2
```

```
type: LDAP
ldap:
  attributes:
    id: 3
    - dn
    email: 4
    - mail
    name: 5
    - cn
    preferredUsername: 6
    - uid
  bindDN: "" 7
  bindPassword: 8
    name: ldap-secret
  ca: 9
    name: ca-config-map
  insecure: false 10
  url: "ldaps://ldaps.example.com/ou=users,dc=acme,dc=com?uid" 11
```

1. This provider name is prefixed to the returned user ID to form an identity name.
2. Controls how mappings are established between this provider's identities and **User** objects.
3. List of attributes to use as the identity. First non-empty attribute is used. At least one attribute is required. If none of the listed attribute have a value, authentication fails. Defined attributes are retrieved as raw, allowing for binary values to be used.
4. List of attributes to use as the email address. First non-empty attribute is used.
5. List of attributes to use as the display name. First non-empty attribute is used.
6. List of attributes to use as the preferred user name when provisioning a user for this identity. First non-empty attribute is used.
7. Optional DN to use to bind during the search phase. Must be set if **bindPassword** is defined.
8. Optional reference to an OpenShift Container Platform **Secret** object containing the bind password. Must be set if **bindDN** is defined.
9. Optional: Reference to an OpenShift Container Platform **ConfigMap** object containing the PEM-encoded certificate authority bundle to use in validating server certificates for the configured URL. Only used when **insecure** is **false**.
10. When **true**, no TLS connection is made to the server. When **false**, **ldaps://** URLs connect using TLS, and **ldap://** URLs are upgraded to TLS. This must be set to **false** when **ldaps://** URLs are in use, as these URLs always attempt to connect using TLS.
11. An RFC 2255 URL which specifies the LDAP host and search parameters to use.



NOTE

To whitelist users for an LDAP integration, use the **lookup** mapping method. Before a login from LDAP would be allowed, a cluster administrator must create an **Identity** object and a **User** object for each LDAP user.

Additional resources

- See Identity provider parameters for information on parameters, such as **mappingMethod**, that are common to all identity providers.

7.3.6. Adding an identity provider to your cluster

Apply the identity provider custom resource (CR) to your cluster so users can authenticate with the configured identity provider.

Prerequisites

- You installed an OpenShift Container Platform cluster.
- You defined the CR for your identity provider.
- You are logged in as an administrator.

Procedure

1. Apply the defined CR by running the following command:

```
$ oc apply -f </path/to/CR>
```

NOTE

If a CR does not exist, **oc apply** creates a new CR and might trigger the following warning: **Warning: oc apply should be used on resources created by either oc create --save-config or oc apply**. In this case you can safely ignore this warning.

2. Log in to the cluster with credentials from the configured identity provider by running the following command. Enter the password when prompted:

```
$ oc login -u <username>
```

3. Confirm that the user logged in successfully and that the username displays by running the following command:

```
$ oc whoami
```

7.4. CONFIGURING A BASIC AUTHENTICATION IDENTITY PROVIDER

Configure the **basic-authentication** identity provider. Users can log in to OpenShift Container Platform with credentials validated against a remote authentication service, without maintaining a separate user store in the cluster.

7.4.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.

NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

7.4.2. About basic authentication

Configure basic authentication to validate user credentials against a remote service over HTTP. Use this integration when you need a flexible back-end for username and password login in OpenShift Container Platform.

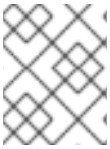
Basic authentication is a generic back-end integration mechanism that allows users to log in to OpenShift Container Platform with credentials validated against a remote identity provider.

Because basic authentication is generic, you can use this identity provider for advanced authentication configurations.

IMPORTANT

Basic authentication must use an HTTPS connection to the remote server to prevent potential snooping of the user ID and password and man-in-the-middle attacks.

With basic authentication configured, users send their username and password to OpenShift Container Platform during login. OpenShift Container Platform validates those credentials against a remote server. OpenShift Container Platform makes a server-to-server request, passing the credentials as a basic authentication header.



NOTE

This only works for username and password login mechanisms, and OpenShift Container Platform must be able to make network requests to the remote authentication server.

Usernames and passwords are validated against a remote URL that is protected by basic authentication and returns JSON.

A **401** response indicates failed authentication.

A non-**200** status, or the presence of a non-empty "error" key, indicates an error:

```
{"error":"Error message"}
```

A **200** status with a **sub** (subject) key indicates success:

```
{"sub":"userid"}
```

where:

userid

Specifies a value that is unique to the authenticated user and must not be modified.

A successful response can optionally provide additional data, such as:

- A display name using the **name** key. For example:

```
{"sub":"userid", "name": "User Name", ...}
```

- An email address using the **email** key. For example:

```
{"sub":"userid", "email":"user@example.com", ...}
```

- A preferred username using the **preferred_username** key. This is useful when the unique, unchangeable subject is a database key or UID, and a more human-readable name exists. This is used as a hint when provisioning the OpenShift Container Platform user for the authenticated identity. For example:

```
{"sub":"014fbff9a07c", "preferred_username":"bob", ...}
```

7.4.3. Creating the secret

You can create a TLS **Secret** object in the **openshift-config** namespace by using the **oc** CLI or by applying a YAML file to store client certificates and keys that identity providers require for secure communication.

Procedure

1. Create a **Secret** object that contains the key and certificate by running the following command:

```
$ oc create secret tls <secret_name> --key=key.pem --cert=cert.pem -n openshift-config
```

2. Optional: Apply the following YAML to create the secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: <secret_name>
  namespace: openshift-config
type: kubernetes.io/tls
data:
  tls.crt: <base64_encoded_cert>
  tls.key: <base64_encoded_key>
```

7.4.4. Creating a 'ConfigMap'

Create a **ConfigMap** object in the **openshift-config** namespace to store the certificate authority bundle that identity providers use to validate secure connections to the remote authentication service.

Procedure

1. Define an OpenShift Container Platform **ConfigMap** object containing the certificate authority by running the following command:

```
$ oc create configmap ca-config-map --from-file=ca.crt=/path/to/ca -n openshift-config
```

2. Optional: Apply the following YAML to create the config map:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ca-config-map
  namespace: openshift-config
data:
  ca.crt: |
    <CA_certificate_PEM>
```

The certificate authority must be stored in the **ca.crt** key of the **ConfigMap** object.

7.4.5. Sample basic authentication custom resource

You can configure basic authentication for your cluster by applying an **OAuth** custom resource (CR) with a **BasicAuth** identity provider. Use this sample to review provider parameters and acceptable values before you connect to your remote authentication server.

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: basicidp
    mappingMethod: claim
    type: BasicAuth
    basicAuth:
      url: https://www.example.com/remote-idp
      ca:
        name: ca-config-map
      tlsClientCert:
        name: client-cert-secret
      tlsClientKey:
        name: client-key-secret
```

where:

spec.identityProviders.name

Specifies that the provider name is prefixed to the returned user ID to form an identity name.

spec.identityProviders.mappingMethod

Specifies how mappings are established between the identities of this provider and **User** objects.

spec.identityProviders.basicAuth.url

Specifies the URL that accepts credentials in Basic authentication headers.

spec.identityProviders.basicAuth.ca

Optional: Specifies a reference to an OpenShift Container Platform **ConfigMap** object containing the Privacy-Enhanced Mail (PEM)-encoded certificate authority bundle to use in validating server certificates for the configured URL.

spec.identityProviders.basicAuth.tlsClientCert

Optional: Specifies a reference to an OpenShift Container Platform **Secret** object containing the client certificate to present when making requests to the configured URL.

spec.identityProviders.basicAuth.tlsClientKey

Specifies a reference to an OpenShift Container Platform **Secret** object containing the key for the client certificate. Required if **tlsClientCert** is specified.

Additional resources

- Identity provider parameters

7.4.6. Adding an identity provider to your cluster

Apply the identity provider custom resource (CR) to your cluster so users can authenticate with the configured identity provider.

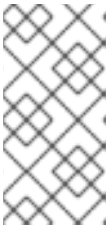
Prerequisites

- You installed an OpenShift Container Platform cluster.
- You defined the CR for your identity provider.
- You are logged in as an administrator.

Procedure

1. Apply the defined CR by running the following command:

```
$ oc apply -f </path/to/CR>
```



NOTE

If a CR does not exist, **oc apply** creates a new CR and might trigger the following warning: **Warning: oc apply should be used on resources created by either oc create --save-config or oc apply**. In this case you can safely ignore this warning.

2. Log in to the cluster with credentials from the configured identity provider by running the following command. Enter the password when prompted:

```
$ oc login -u <username>
```

3. Confirm that the user logged in successfully and that the username displays by running the following command:

```
$ oc whoami
```

7.4.7. Example Apache HTTPD configuration for basic identity providers

You can use CGI scripting in Apache HTTPD to configure a remote authentication server that returns JSON responses for basic identity providers in OpenShift Container Platform.

The following is an example of an Apache **VirtualHost** configuration file.

```
<VirtualHost *:443>
  # CGI Scripts in here
  DocumentRoot /var/www/cgi-bin

  # SSL Directives
  SSLEngine on
  SSLCipherSuite PROFILE=SYSTEM
  SSLProxyCipherSuite PROFILE=SYSTEM
  SSLCertificateFile /etc/pki/tls/certs/localhost.crt
  SSLCertificateKeyFile /etc/pki/tls/private/localhost.key

  # Configure HTTPD to execute scripts
  ScriptAlias /basic /var/www/cgi-bin

  # Handles a failed login attempt
  ErrorDocument 401 /basic/fail.cgi

  # Handles authentication
  <Location /basic/login.cgi>
    AuthType Basic
    AuthName "Please Log In"
    AuthBasicProvider file
    AuthUserFile /etc/httpd/conf/passwords
    Require valid-user
  </Location>
</VirtualHost>
```

The following is an example of a **login.cgi** CGI script file.

```
#!/bin/bash
echo "Content-Type: application/json"
echo ""
echo '{"sub":"userid", "name":"'$REMOTE_USER'"}'
exit 0
```

The following is an example of a **fail.cgi** CGI script file.

```
#!/bin/bash
echo "Content-Type: application/json"
echo ""
echo '{"error": "Login failure"}'
exit 0
```

7.4.7.1. File requirements

These are the requirements for the files you create on an Apache HTTPD web server:

- The **login.cgi** and **fail.cgi** CGI script files must be executable. Use the **chmod +x** command on both files.

- If SELinux is enabled, the **login.cgi** and **fail.cgi** CGI script files must have proper SELinux security contexts. Run the **restorecon -RFv /var/www/cgi-bin** command, or ensure that the context is the **httpd_sys_script_exec_t** SELinux type by using the **ls -laZ** command.
- The **login.cgi** CGI script file runs only when the user successfully logs in according to the **Require** and **Auth** Apache configuration directives.
- The **fail.cgi** CGI script file runs when the user fails to log in and returns an **HTTP 401** HTTP status code.

7.4.8. Troubleshooting basic authentication

Troubleshoot basic authentication by testing backend connectivity and verifying JSON login responses when users cannot authenticate in OpenShift Container Platform.

The most common issue relates to network connectivity to the backend server. To debug connectivity, run **curl** commands on a control plane node.

Procedure

1. To test successful and unsuccessful logins, replace the **<user>** and **<password>** in the following example command with valid or invalid credentials:

```
$ curl --cacert /path/to/ca.crt --cert /path/to/client.crt --key /path/to/client.key -u <user>:
<password> -v https://www.example.com/remote-idp
```

2. Review successful login responses.
   A **200** status with a **sub** (subject) key indicates success:

```
{"sub":"userid"}
```

   The subject must be unique to the authenticated user and must not be modified.

   A successful response can optionally provide additional data, such as:

   - A display name using the **name** key:

```
{"sub":"userid", "name": "User Name", ...}
```

   - An email address using the **email** key:

```
{"sub":"userid", "email":"user@example.com", ...}
```

   - A preferred username using the **preferred_username** key:

```
{"sub":"014fbff9a07c", "preferred_username":"bob", ...}
```

   The **preferred_username** key is useful when the unique, unchangeable subject is a database key or UID, and a more human-readable name exists. This is used as a hint when provisioning the OpenShift Container Platform user for the authenticated identity.

3. Review failed login responses.

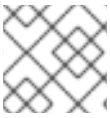
- A **401** response indicates failed authentication.
- A non-**200** status or the presence of a non-empty "error" key indicates an error: **{"error":"Error message"}**

7.5. CONFIGURING A REQUEST HEADER IDENTITY PROVIDER

Configure the **request-header** identity provider to identify users from request header values, such as **X-Remote-User**. It is typically used in combination with an authenticating proxy, which sets the request header value.

7.5.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.



NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

7.5.2. About request header authentication

A request header identity provider identifies users from request header values, such as **X-Remote-User**. It is typically used in combination with an authenticating proxy, which sets the request header value. The request header identity provider cannot be combined with other identity providers that use direct password logins, such as htpasswd, Keystone, LDAP or basic authentication.

NOTE

You can also use the request header identity provider for advanced configurations such as the community-supported SAML authentication. Note that this solution is not supported by Red Hat.

For users to authenticate using this identity provider, they must access **https://*<namespace_route>*/oauth/authorize** (and subpaths) via an authenticating proxy. To accomplish this, configure the OAuth server to redirect unauthenticated requests for OAuth tokens to the proxy endpoint that proxies to **https://*<namespace_route>*/oauth/authorize**.

To redirect unauthenticated requests from clients expecting browser-based login flows:

- Set the **provider.loginURL** parameter to the authenticating proxy URL that will authenticate interactive clients and then proxy the request to **https://*<namespace_route>*/oauth/authorize**.

To redirect unauthenticated requests from clients expecting **WWW-Authenticate** challenges:

- Set the **provider.challengeURL** parameter to the authenticating proxy URL that will authenticate clients expecting **WWW-Authenticate** challenges and then proxy the request to **https://*<namespace_route>*/oauth/authorize**.

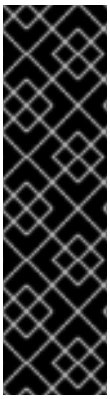
The **provider.challengeURL** and **provider.loginURL** parameters can include the following tokens in the query portion of the URL:

- **${url}** is replaced with the current URL, escaped to be safe in a query parameter.
  For example: **https://www.example.com/sso-login?then=${url}**
- **${query}** is replaced with the current query string, unescaped.
  For example: **https://www.example.com/auth-proxy/oauth/authorize?${query}**

IMPORTANT

As of OpenShift Container Platform 4.1, your proxy must support mutual TLS.

7.5.2.1. SSPI connection support on Microsoft Windows



IMPORTANT

Using SSPI connection support on Microsoft Windows is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

The OpenShift CLI (**oc**) supports the Security Support Provider Interface (SSPI) to allow for SSO flows on Microsft Windows. If you use the request header identity provider with a GSSAPI-enabled proxy to connect an Active Directory server to OpenShift Container Platform, users can automatically authenticate to OpenShift Container Platform by using the **oc** command line interface from a domain-joined Microsoft Windows computer.

7.5.3. Creating a 'ConfigMap'

Create a **ConfigMap** object in the **openshift-config** namespace to store the certificate authority bundle that identity providers use to validate secure connections to the remote authentication service.

Procedure

1. Define an OpenShift Container Platform **ConfigMap** object containing the certificate authority by running the following command:

```
$ oc create configmap ca-config-map --from-file=ca.crt=/path/to/ca -n openshift-config
```

2. Optional: Apply the following YAML to create the config map:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ca-config-map
  namespace: openshift-config
data:
  ca.crt: |
    <CA_certificate_PEM>
```

The certificate authority must be stored in the **ca.crt** key of the **ConfigMap** object.

7.5.4. Sample request header CR

The following custom resource (CR) shows the parameters and acceptable values for a request header identity provider.

Request header CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: requestheaderidp 1
    mappingMethod: claim 2
    type: RequestHeader
    requestHeader:
      challengeURL: "https://www.example.com/challenging-proxy/oauth/authorize?${query}" 3
      loginURL: "https://www.example.com/login-proxy/oauth/authorize?${query}" 4
      ca: 5
        name: ca-config-map
      clientCommonNames: 6
      - my-auth-proxy
      headers: 7
      - X-Remote-User
      - SSO-User
      emailHeaders: 8
      - X-Remote-User-Email
      nameHeaders: 9
      - X-Remote-User-Display-Name
      preferredUsernameHeaders: 10
      - X-Remote-User-Login
```

1. This provider name is prefixed to the user name in the request header to form an identity name.
2. Controls how mappings are established between this provider's identities and **User** objects.
3. Optional: URL to redirect unauthenticated **/oauth/authorize** requests to, that will authenticate browser-based clients and then proxy their request to **https://*<namespace_route>*/oauth/authorize**. The URL that proxies to **https://*<namespace_route>*/oauth/authorize** must end with **/authorize** (with no trailing slash), and also proxy subpaths, in order for OAuth approval flows to work properly. **${url}** is replaced with the current URL, escaped to be safe in a query parameter. **${query}** is replaced with the current query string. If this attribute is not defined, then **loginURL** must be used.
4. Optional: URL to redirect unauthenticated **/oauth/authorize** requests to, that will authenticate clients which expect **WWW-Authenticate** challenges, and then proxy them to **https://*<namespace_route>*/oauth/authorize**. **${url}** is replaced with the current URL, escaped to be safe in a query parameter. **${query}** is replaced with the current query string. If this attribute is not defined, then **challengeURL** must be used.

5 Reference to an OpenShift Container Platform **ConfigMap** object containing a PEM-encoded certificate bundle. Used as a trust anchor to validate the TLS certificates presented by the remote

IMPORTANT

As of OpenShift Container Platform 4.1, the **ca** field is required for this identity provider. This means that your proxy must support mutual TLS.

6 Optional: list of common names (**cn**). If set, a valid client certificate with a Common Name (**cn**) in the specified list must be presented before the request headers are checked for user names. If empty, any Common Name is allowed. Can only be used in combination with **ca**.

7 Header names to check, in order, for the user identity. The first header containing a value is used as the identity. Required, case-insensitive.

8 Header names to check, in order, for an email address. The first header containing a value is used as the email address. Optional, case-insensitive.

9 Header names to check, in order, for a display name. The first header containing a value is used as the display name. Optional, case-insensitive.

10 Header names to check, in order, for a preferred user name, if different than the immutable identity determined from the headers specified in **headers**. The first header containing a value is used as the preferred user name when provisioning. Optional, case-insensitive.

Additional resources

- See Identity provider parameters for information on parameters, such as **mappingMethod**, that are common to all identity providers.

7.5.5. Adding an identity provider to your cluster

Apply the identity provider custom resource (CR) to your cluster so users can authenticate with the configured identity provider.

Prerequisites

- You installed an OpenShift Container Platform cluster.
- You defined the CR for your identity provider.
- You are logged in as an administrator.

Procedure

1. Apply the defined CR by running the following command:

```
$ oc apply -f </path/to/CR>
```

NOTE

If a CR does not exist, **oc apply** creates a new CR and might trigger the following warning: **Warning: oc apply should be used on resources created by either oc create --save-config or oc apply**. In this case you can safely ignore this warning.

2. Log in to the cluster with credentials from the configured identity provider by running the following command. Enter the password when prompted:

```
$ oc login -u <username>
```

3. Confirm that the user logged in successfully and that the username displays by running the following command:

```
$ oc whoami
```

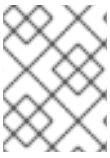
7.5.6. Example Apache authentication configuration using request header

This example configures an Apache authentication proxy for the OpenShift Container Platform using the request header identity provider.

7.5.6.1. Custom proxy configuration

Using the **mod_auth_gssapi** module is a popular way to configure the Apache authentication proxy using the request header identity provider; however, it is not required. Other proxies can easily be used if the following requirements are met:

- Block the **X-Remote-User** header from client requests to prevent spoofing.
- Enforce client certificate authentication in the **RequestHeaderIdentityProvider** configuration.
- Require the **X-Csrf-Token** header be set for all authentication requests using the challenge flow.
- Make sure only the **/oauth/authorize** endpoint and its subpaths are proxied; redirects must be rewritten to allow the backend server to send the client to the correct location.
- The URL that proxies to **https://<namespace_route>/oauth/authorize** must end with **/authorize** with no trailing slash. For example, **https://proxy.example.com/login-proxy/authorize?…** must proxy to **https://<namespace_route>/oauth/authorize?…**.
- Subpaths of the URL that proxies to **https://<namespace_route>/oauth/authorize** must proxy to subpaths of **https://<namespace_route>/oauth/authorize**. For example, **https://proxy.example.com/login-proxy/authorize/approve?…** must proxy to **https://<namespace_route>/oauth/authorize/approve?…**.



NOTE

The **https://<namespace_route>** address is the route to the OAuth server and can be obtained by running **oc get route -n openshift-authentication**.

7.5.6.2. Configuring Apache authentication using request header

This example uses the **mod_auth_gssapi** module to configure an Apache authentication proxy using the request header identity provider.

Prerequisites

- Obtain the **mod_auth_gssapi** module from the Optional channel. You must have the following packages installed on your local machine:
  - **httpd**
  - **mod_ssl**
  - **mod_session**
  - **apr-util-openssl**
  - **mod_auth_gssapi**
- Generate a CA for validating requests that submit the trusted header. Define an OpenShift Container Platform **ConfigMap** object containing the CA. This is done by running:

```
$ oc create configmap ca-config-map --from-file=ca.crt=/path/to/ca -n openshift-config
```
(1)

(1) The CA must be stored in the **ca.crt** key of the **ConfigMap** object.

TIP

You can alternatively apply the following YAML to create the config map:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ca-config-map
  namespace: openshift-config
data:
  ca.crt: |
    <CA_certificate_PEM>
```

- Generate a client certificate for the proxy. You can generate this certificate by using any x509 certificate tooling. The client certificate must be signed by the CA you generated for validating requests that submit the trusted header.
- Create the custom resource (CR) for your identity providers.

Procedure

This proxy uses a client certificate to connect to the OAuth server, which is configured to trust the **X-Remote-User** header.

1. Create the certificate for the Apache configuration. The certificate that you specify as the **SSLProxyMachineCertificateFile** parameter value is the proxy's client certificate that is used to authenticate the proxy to the server. It must use **TLS Web Client Authentication** as the extended key type.

2. Create the Apache configuration. Use the following template to provide your required settings and values:

IMPORTANT

Carefully review the template and customize its contents to fit your environment.

```
LoadModule request_module modules/mod_request.so
LoadModule auth_gssapi_module modules/mod_auth_gssapi.so
# Some Apache configurations might require these modules.
# LoadModule auth_form_module modules/mod_auth_form.so
# LoadModule session_module modules/mod_session.so

# Nothing needs to be served over HTTP.  This virtual host simply redirects to
# HTTPS.
<VirtualHost *:80>
  DocumentRoot /var/www/html
  RewriteEngine              On
  RewriteRule     ^(.*)$     https://%{HTTP_HOST}$1 [R,L]
</VirtualHost>

<VirtualHost *:443>
  # This needs to match the certificates you generated.  See the CN and X509v3
  # Subject Alternative Name in the output of:
  # openssl x509 -text -in /etc/pki/tls/certs/localhost.crt
  ServerName www.example.com

  DocumentRoot /var/www/html
  SSLEngine on
  SSLCertificateFile /etc/pki/tls/certs/localhost.crt
  SSLCertificateKeyFile /etc/pki/tls/private/localhost.key
  SSLCACertificateFile /etc/pki/CA/certs/ca.crt

  SSLProxyEngine on
  SSLProxyCACertificateFile /etc/pki/CA/certs/ca.crt
  # It is critical to enforce client certificates. Otherwise, requests can
  # spoof the X-Remote-User header by accessing the /oauth/authorize endpoint
  # directly.
  SSLProxyMachineCertificateFile /etc/pki/tls/certs/authproxy.pem

  # To use the challenging-proxy, an X-Csrf-Token must be present.
  RewriteCond %{REQUEST_URI} ^/challenging-proxy
  RewriteCond %{HTTP:X-Csrf-Token} ^$ [NC]
  RewriteRule ^.* - [F,L]

  <Location /challenging-proxy/oauth/authorize>
     # Insert your backend server name/ip here.
     ProxyPass https://<namespace_route>/oauth/authorize
     AuthName "SSO Login"
     # For Kerberos
     AuthType GSSAPI
     Require valid-user
     RequestHeader set X-Remote-User %{REMOTE_USER}s

     GssapiCredStore keytab:/etc/httpd/protected/auth-proxy.keytab
```

```
      # Enable the following if you want to allow users to fallback
      # to password based authentication when they do not have a client
      # configured to perform kerberos authentication.
      GssapiBasicAuth On

      # For ldap:
      # AuthBasicProvider ldap
      # AuthLDAPURL "ldap://ldap.example.com:389/ou=People,dc=my-domain,dc=com?uid?
sub?(objectClass=*)"
    </Location>

    <Location /login-proxy/oauth/authorize>
    # Insert your backend server name/ip here.
    ProxyPass https://<namespace_route>/oauth/authorize

      AuthName "SSO Login"
      AuthType GSSAPI
      Require valid-user
      RequestHeader set X-Remote-User %{REMOTE_USER}s env=REMOTE_USER

      GssapiCredStore keytab:/etc/httpd/protected/auth-proxy.keytab
      # Enable the following if you want to allow users to fallback
      # to password based authentication when they do not have a client
      # configured to perform kerberos authentication.
      GssapiBasicAuth On

      ErrorDocument 401 /login.html
    </Location>

</VirtualHost>

RequestHeader unset X-Remote-User
```

NOTE

The **https://<namespace_route>** address is the route to the OAuth server and can be obtained by running **oc get route -n openshift-authentication**.

3. Update the **identityProviders** stanza in the custom resource (CR):

```
identityProviders:
  - name: requestheaderidp
    type: RequestHeader
    requestHeader:
      challengeURL: "https://<namespace_route>/challenging-proxy/oauth/authorize?${query}"
      loginURL: "https://<namespace_route>/login-proxy/oauth/authorize?${query}"
      ca:
        name: ca-config-map
        clientCommonNames:
        - my-auth-proxy
        headers:
        - X-Remote-User
```

4. Verify the configuration.

a. Confirm that you can bypass the proxy by requesting a token by supplying the correct client certificate and header:

```
# curl -L -k -H "X-Remote-User: joe" \
  --cert /etc/pki/tls/certs/authproxy.pem \
  https://<namespace_route>/oauth/token/request
```

b. Confirm that requests that do not supply the client certificate fail by requesting a token without the certificate:

```
# curl -L -k -H "X-Remote-User: joe" \
  https://<namespace_route>/oauth/token/request
```

c. Confirm that the **challengeURL** redirect is active:

```
# curl -k -v -H 'X-Csrf-Token: 1' \
  https://<namespace_route>/oauth/authorize?client_id=openshift-challenging-client&response_type=token
```

Copy the **challengeURL** redirect to use in the next step.

d. Run this command to show a **401** response with a **WWW-Authenticate** basic challenge, a negotiate challenge, or both challenges:

```
# curl -k -v -H 'X-Csrf-Token: 1' \
  <challengeURL_redirect + query>
```

e. Test logging in to the OpenShift CLI (**oc**) with and without using a Kerberos ticket:

i. If you generated a Kerberos ticket by using **kinit**, destroy it:

```
# kdestroy -c cache_name 1
```

1 Make sure to provide the name of your Kerberos cache.

ii. Log in to the **oc** tool by using your Kerberos credentials:

```
# oc login -u <username>
```

Enter your Kerberos password at the prompt.

iii. Log out of the **oc** tool:

```
# oc logout
```

iv. Use your Kerberos credentials to get a ticket:

```
# kinit
```

Enter your Kerberos user name and password at the prompt.

v. Confirm that you can log in to the **oc** tool:

```
# oc login
```

If your configuration is correct, you are logged in without entering separate credentials.

7.6. CONFIGURING A GITHUB OR GITHUB ENTERPRISE IDENTITY PROVIDER

Configure the **github** identity provider to validate user names and passwords against GitHub or GitHub Enterprise's OAuth authentication server. OAuth facilitates a token exchange flow between OpenShift Container Platform and GitHub or GitHub Enterprise.

You can use the GitHub integration to connect to either GitHub or GitHub Enterprise. For GitHub Enterprise integrations, you must provide the **hostname** of your instance and can optionally provide a **ca** certificate bundle to use in requests to the server.

NOTE

The following steps apply to both GitHub and GitHub Enterprise unless noted.

7.6.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.

NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

7.6.2. About GitHub authentication

Configuring GitHub authentication allows users to log in to OpenShift Container Platform with their GitHub credentials. To prevent anyone with any GitHub user ID from logging in to your OpenShift Container Platform cluster, you can restrict access to only those in specific GitHub organizations.

7.6.3. Registering a GitHub application

To use GitHub or GitHub Enterprise as an identity provider, you must register an application to use.

Procedure

1. Register an application on GitHub:
   - For GitHub, click **Settings → Developer settings → OAuth Apps → Register a new OAuth application**.
   - For GitHub Enterprise, go to your GitHub Enterprise home page and then click **Settings → Developer settings → Register a new application**.
2. Enter an application name, for example **My OpenShift Install**.
3. Enter a homepage URL, such as **https://oauth-openshift.apps.<cluster-name>.<cluster-domain>**.

4. Optional: Enter an application description.
5. Enter the authorization callback URL, where the end of the URL contains the identity provider **name**:

```
https://oauth-openshift.apps.<cluster-name>.<cluster-domain>/oauth2callback/<idp-provider-name>
```

For example:

```
https://oauth-openshift.apps.openshift-cluster.example.com/oauth2callback/github
```

6. Click **Register application**. GitHub provides a client ID and a client secret. You need these values to complete the identity provider configuration.

7.6.4. Creating the secret

Create a Secret in the **openshift-config** namespace to store identity provider client secrets, certificates, or keys, so the OAuth custom resource (CR) can reference them.

Procedure

1. Create a **Secret** object containing the client secret by running the following command:

```
$ oc create secret generic <secret_name> --from-literal=clientSecret=<secret> -n openshift-config
```

2. Optional: Apply the following YAML to create the secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: <secret_name>
  namespace: openshift-config
type: Opaque
data:
  clientSecret: <base64_encoded_client_secret>
```

3. Create a **Secret** object from a file by running the following command:

```
$ oc create secret generic <secret_name> --from-file=<path_to_file> -n openshift-config
```

7.6.5. Creating a 'ConfigMap'

Create a **ConfigMap** object in the **openshift-config** namespace to store the certificate authority bundle that identity providers use to validate secure connections to the remote authentication service.

NOTE

This procedure is only required for GitHub Enterprise.

Procedure

1. Define an OpenShift Container Platform **ConfigMap** object containing the certificate authority by running the following command:

```
$ oc create configmap ca-config-map --from-file=ca.crt=/path/to/ca -n openshift-config
```

2. Optional: Apply the following YAML to create the config map:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ca-config-map
  namespace: openshift-config
data:
  ca.crt: |
    <CA_certificate_PEM>
```

The certificate authority must be stored in the **ca.crt** key of the **ConfigMap** object.

7.6.6. Sample GitHub CR

The following custom resource (CR) shows the parameters and acceptable values for a GitHub identity provider.

GitHub CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: githubidp 1
    mappingMethod: claim 2
    type: GitHub
    github:
      ca: 3
        name: ca-config-map
      clientID: {...} 4
      clientSecret: 5
        name: github-secret
      hostname: ... 6
      organizations: 7
      - myorganization1
      - myorganization2
      teams: 8
      - myorganization1/team-a
      - myorganization2/team-b
```

1. This provider name is prefixed to the GitHub numeric user ID to form an identity name. It is also used to build the callback URL.

2. Controls how mappings are established between this provider's identities and **User** objects.

3 Optional: Reference to an OpenShift Container Platform **ConfigMap** object containing the PEM-encoded certificate authority bundle to use in validating server certificates for the configured URL. Only for use in GitHub Enterprise with a non-publicly trusted root certificate.

4 The client ID of a registered GitHub OAuth application. The application must be configured with a callback URL of **https://oauth-openshift.apps.<cluster-name>.<cluster-domain>/oauth2callback/<idp-provider-name>**.

5 Reference to an OpenShift Container Platform **Secret** object containing the client secret issued by GitHub.

6 For GitHub Enterprise, you must provide the hostname of your instance, such as **example.com**. This value must match the GitHub Enterprise **hostname** value in in the **/setup/settings** file and cannot include a port number. If this value is not set, then either **teams** or **organizations** must be defined. For GitHub, omit this parameter.

7 The list of organizations. Either the **organizations** or **teams** field must be set unless the **hostname** field is set, or if **mappingMethod** is set to **lookup**. Cannot be used in combination with the **teams** field.

8 The list of teams. Either the **teams** or **organizations** field must be set unless the **hostname** field is set, or if **mappingMethod** is set to **lookup**. Cannot be used in combination with the **organizations** field.

NOTE

If **organizations** or **teams** is specified, only GitHub users that are members of at least one of the listed organizations will be allowed to log in. If the GitHub OAuth application configured in **clientID** is not owned by the organization, an organization owner must grant third-party access to use this option. This can be done during the first GitHub login by the organization's administrator, or from the GitHub organization settings.

Additional resources

- See Identity provider parameters for information on parameters, such as **mappingMethod**, that are common to all identity providers.

7.6.7. Adding an identity provider to your cluster

Apply the identity provider custom resource (CR) to your cluster so users can authenticate with the configured identity provider.

Prerequisites

- You installed an OpenShift Container Platform cluster.
- You defined the CR for your identity provider.
- You are logged in as an administrator.

Procedure

1. Apply the defined CR by running the following command:

```
$ oc apply -f </path/to/CR>
```

NOTE

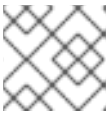
If a CR does not exist, **oc apply** creates a new CR and might trigger the following warning: **Warning: oc apply should be used on resources created by either oc create --save-config or oc apply**. In this case you can safely ignore this warning.

2. Obtain a token from the OAuth server.
   As long as the **kubeadmin** user has been removed, the **oc login** command provides instructions on how to access a web page where you can retrieve the token.

   You can also access this page from the web console by navigating to **(?) Help → Command Line Tools → Copy Login Command**.

3. Log in to the cluster by running the following command and pass in the token to authenticate:

```
$ oc login --token=<token>
```



NOTE

This identity provider does not support logging in with a username and password.

4. Confirm that the user logged in successfully and that the username displays by running the following command:

```
$ oc whoami
```

7.7. CONFIGURING A GITLAB IDENTITY PROVIDER

Configure the **gitlab** identity provider using GitLab.com or any other GitLab instance as an identity provider.

7.7.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.

NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

7.7.2. About GitLab authentication

Configuring GitLab authentication allows users to log in to OpenShift Container Platform with their GitLab credentials.

If you use GitLab version 7.7.0 to 11.0, you connect using the OAuth integration. If you use GitLab version 11.1 or later, you can use OpenID Connect (OIDC) to connect instead of OAuth.

7.7.3. Creating the secret

Create a Secret in the **openshift-config** namespace to store identity provider client secrets, certificates, or keys, so the OAuth custom resource (CR) can reference them.

Procedure

1. Create a **Secret** object containing the client secret by running the following command:

```
$ oc create secret generic <secret_name> --from-literal=clientSecret=<secret> -n openshift-config
```

2. Optional: Apply the following YAML to create the secret:

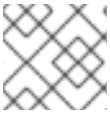
```
apiVersion: v1
kind: Secret
metadata:
  name: <secret_name>
  namespace: openshift-config
type: Opaque
data:
  clientSecret: <base64_encoded_client_secret>
```

3. Create a **Secret** object from a file by running the following command:

```
$ oc create secret generic <secret_name> --from-file=<path_to_file> -n openshift-config
```

7.7.4. Creating a 'ConfigMap'

Create a **ConfigMap** object in the **openshift-config** namespace to store the certificate authority bundle that identity providers use to validate secure connections to the remote authentication service.



NOTE

This procedure is only required for GitHub Enterprise.

Procedure

1. Define an OpenShift Container Platform **ConfigMap** object containing the certificate authority by running the following command:

```
$ oc create configmap ca-config-map --from-file=ca.crt=/path/to/ca -n openshift-config
```

2. Optional: Apply the following YAML to create the config map:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ca-config-map
  namespace: openshift-config
data:
  ca.crt: |
    <CA_certificate_PEM>
```

-

The certificate authority must be stored in the **ca.crt** key of the **ConfigMap** object.

7.7.5. Sample GitLab CR

The following custom resource (CR) shows the parameters and acceptable values for a GitLab identity provider.

GitLab CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: gitlabidp 1
    mappingMethod: claim 2
    type: GitLab
    gitlab:
      clientID: {...} 3
      clientSecret: 4
        name: gitlab-secret
      url: https://gitlab.com 5
      ca: 6
        name: ca-config-map
```

1. This provider name is prefixed to the GitLab numeric user ID to form an identity name. It is also used to build the callback URL.
2. Controls how mappings are established between this provider's identities and **User** objects.
3. The client ID of a registered GitLab OAuth application. The application must be configured with a callback URL of **https://oauth-openshift.apps.<cluster-name>.<cluster-domain>/oauth2callback/<idp-provider-name>**.
4. Reference to an OpenShift Container Platform **Secret** object containing the client secret issued by GitLab.
5. The host URL of a GitLab provider. This could either be **https://gitlab.com/** or any other self hosted instance of GitLab.
6. Optional: Reference to an OpenShift Container Platform **ConfigMap** object containing the PEM-encoded certificate authority bundle to use in validating server certificates for the configured URL.

Additional resources

- See Identity provider parameters for information on parameters, such as **mappingMethod**, that are common to all identity providers.

7.7.6. Adding an identity provider to your cluster

Apply the identity provider custom resource (CR) to your cluster so users can authenticate with the configured identity provider.

Prerequisites

- You installed an OpenShift Container Platform cluster.
- You defined the CR for your identity provider.
- You are logged in as an administrator.

Procedure

1. Apply the defined CR by running the following command:

```
$ oc apply -f </path/to/CR>
```

NOTE

If a CR does not exist, **oc apply** creates a new CR and might trigger the following warning: **Warning: oc apply should be used on resources created by either oc create --save-config or oc apply**. In this case you can safely ignore this warning.

2. Log in to the cluster with credentials from the configured identity provider by running the following command. Enter the password when prompted:

```
$ oc login -u <username>
```

3. Confirm that the user logged in successfully and that the username displays by running the following command:

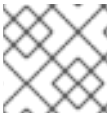
```
$ oc whoami
```

7.8. CONFIGURING A GOOGLE IDENTITY PROVIDER

Configure a Google identity provider so users can authenticate to OpenShift Container Platform with Google accounts. When configured, sign-in is permitted only for Google accounts in that hosted domain.

7.8.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.



NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

7.8.2. Google authentication

By using Google as an identity provider, you can authenticate to your server. You can use the **hostedDomain** configuration attribute to limit authentication to members of a specific hosted domain.

Google authentication uses OpenID Connect through the cluster OAuth server.

NOTE

Using Google as an identity provider requires users to get a token using **<namespace_route>/oauth/token/request** to use with command-line tools.

7.8.3. Creating the secret

Create a Secret in the **openshift-config** namespace to store identity provider client secrets, certificates, or keys, so the OAuth custom resource (CR) can reference them.

Procedure

1. Create a **Secret** object containing the client secret by running the following command:

```
$ oc create secret generic <secret_name> --from-literal=clientSecret=<secret> -n openshift-config
```

2. Optional: Apply the following YAML to create the secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: <secret_name>
  namespace: openshift-config
type: Opaque
data:
  clientSecret: <base64_encoded_client_secret>
```

3. Create a **Secret** object from a file by running the following command:

```
$ oc create secret generic <secret_name> --from-file=<path_to_file> -n openshift-config
```

7.8.4. Sample Google custom resource

Review the custom resource (CR) fields and acceptable values for configuring a Google identity provider in OpenShift Container Platform. Use these definitions to set client credentials and hosted domain restrictions before applying the configuration to the cluster.

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: googleidp
    mappingMethod: claim
    type: Google
    google:
```

```
    clientID: {...}
    clientSecret:
      name: google-secret
    hostedDomain: "example.com"
```

where:

spec.identityProviders.name

Specifies the provider name, which is prefixed to the Google numeric user ID to form an identity name. The provider name is also used to build the redirect URL.

spec.identityProviders.mappingMethod

Specifies how mappings are established between identities from this provider and **User** objects.

spec.identityProviders.google.clientID

Specifies the client ID from the Google Cloud project where you create the OAuth client. The project must be configured with a redirect URI of **https://oauth-openshift.apps.<cluster-name>.<cluster-domain>/oauth2callback/<idp-provider-name>**.

spec.identityProviders.google.clientSecret

Specifies a reference to an OpenShift Container Platform **Secret** object containing the client secret issued by Google.

spec.identityProviders.google.hostedDomain

Specifies a hosted domain used to restrict sign-in accounts. Optional if the **lookup mappingMethod** is used. If empty, any Google account is allowed to authenticate.

Additional resources

- Identity provider parameters

7.8.5. Adding an identity provider to your cluster

Apply the identity provider custom resource (CR) to your cluster so users can authenticate with the configured identity provider.

Prerequisites

- You installed an OpenShift Container Platform cluster.
- You defined the CR for your identity provider.
- You are logged in as an administrator.

Procedure

1. Apply the defined CR by running the following command:

```
$ oc apply -f </path/to/CR>
```

NOTE

If a CR does not exist, **oc apply** creates a new CR and might trigger the following warning: **Warning: oc apply should be used on resources created by either oc create --save-config or oc apply**. In this case you can safely ignore this warning.

2. Obtain a token from the OAuth server.
   As long as the **kubeadmin** user has been removed, the **oc login** command provides instructions on how to access a web page where you can retrieve the token.

   You can also access this page from the web console by navigating to **(?) Help → Command Line Tools → Copy Login Command**.

3. Log in to the cluster by running the following command and pass in the token to authenticate:

```
$ oc login --token=<token>
```

NOTE

This identity provider does not support logging in with a username and password.

4. Confirm that the user logged in successfully and that the username displays by running the following command:

```
$ oc whoami
```

7.8.6. Additional resources

- OpenID Connect (Google Identity documentation)

7.9. CONFIGURING AN OPENID CONNECT IDENTITY PROVIDER

Configure the **oidc** identity provider to integrate with an OpenID Connect identity provider using an Authorization Code Flow.

7.9.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.

NOTE

OpenShift Container Platform usernames containing /, **:**, and **%** are not supported.

7.9.2. About OpenID Connect authentication

The Authentication Operator in OpenShift Container Platform requires that the configured OpenID Connect identity provider implements the OpenID Connect Discovery specification.

NOTE

ID Token and **UserInfo** decryptions are not supported.

By default, the **openid** scope is requested. If required, extra scopes can be specified in the **extraScopes** field.

Claims are read from the JWT **id_token** returned from the OpenID identity provider and, if specified, from the JSON returned by the **UserInfo** URL.

At least one claim must be configured to use as the user's identity. The standard identity claim is **sub**.

You can also indicate which claims to use as the user's preferred user name, display name, and email address. If multiple claims are specified, the first one with a non-empty value is used. The following table lists the standard claims:

Claim	Description
sub	Short for "subject identifier." The remote identity for the user at the issuer.
preferred_username	The preferred user name when provisioning a user. A shorthand name that the user wants to be referred to as, such as **janedoe**. Typically a value that corresponding to the user's login or username in the authentication system, such as username or email.
email	Email address.
name	Display name.

See the OpenID claims documentation for more information.

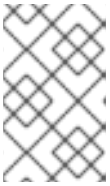
NOTE

Unless your OpenID Connect identity provider supports the resource owner password credentials (ROPC) grant flow, users must get a token from **<namespace_route>/oauth/token/request** to use with command-line tools.

7.9.3. Supported OIDC providers

Red Hat tests and supports specific OpenID Connect (OIDC) providers with OpenShift Container Platform. The following OpenID Connect (OIDC) providers are tested and supported with OpenShift Container Platform. Using an OIDC provider that is not on the following list might work with OpenShift Container Platform, but the provider was not tested by Red Hat and therefore is not supported by Red Hat.

- Active Directory Federation Services for Windows Server



NOTE

Currently, it is not supported to use Active Directory Federation Services for Windows Server with OpenShift Container Platform when custom claims are used.

- GitLab
- Google
- Keycloak
- Microsoft Entra ID

NOTE

Currently, it is not supported to use Microsoft Entra ID when group names are required to be synced.

- Okta
- Ping Identity
- Red Hat Single Sign-On

7.9.4. Creating the secret

Create a Secret in the **openshift-config** namespace to store identity provider client secrets, certificates, or keys, so the OAuth custom resource (CR) can reference them.

Procedure

1. Create a **Secret** object containing the client secret by running the following command:

```
$ oc create secret generic <secret_name> --from-literal=clientSecret=<secret> -n openshift-config
```

2. Optional: Apply the following YAML to create the secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: <secret_name>
  namespace: openshift-config
type: Opaque
data:
  clientSecret: <base64_encoded_client_secret>
```

3. Create a **Secret** object from a file by running the following command:

```
$ oc create secret generic <secret_name> --from-file=<path_to_file> -n openshift-config
```

7.9.5. Creating a 'ConfigMap'

Create a **ConfigMap** object in the **openshift-config** namespace to store the certificate authority bundle that identity providers use to validate secure connections to the remote authentication service.

NOTE

This procedure is only required for GitHub Enterprise.

Procedure

1. Define an OpenShift Container Platform **ConfigMap** object containing the certificate authority by running the following command:

```
$ oc create configmap ca-config-map --from-file=ca.crt=/path/to/ca -n openshift-config
```

2. Optional: Apply the following YAML to create the config map:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ca-config-map
  namespace: openshift-config
data:
  ca.crt: |
    <CA_certificate_PEM>
```

The certificate authority must be stored in the **ca.crt** key of the **ConfigMap** object.

7.9.6. Sample OpenID Connect CRs

The following custom resources (CRs) show the parameters and acceptable values for an OpenID Connect identity provider.

If you must specify a custom certificate bundle, extra scopes, extra authorization request parameters, or a **userInfo** URL, use the full OpenID Connect CR.

Standard OpenID Connect CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: oidcidp 1
    mappingMethod: claim 2
    type: OpenID
    openID:
      clientID: ... 3
      clientSecret: 4
        name: idp-secret
      claims: 5
        preferredUsername:
        - preferred_username
```

```
      name:
      - name
      email:
      - email
      groups:
      - groups
    issuer: https://www.idp-issuer.com 6
```

1. This provider name is prefixed to the value of the identity claim to form an identity name. It is also used to build the redirect URL.
2. Controls how mappings are established between this provider's identities and **User** objects.
3. The client ID of a client registered with the OpenID provider. The client must be allowed to redirect to **https://oauth-openshift.apps.<cluster_name>.<cluster_domain>/oauth2callback/<idp_provider_name>**.
4. A reference to an OpenShift Container Platform **Secret** object containing the client secret.
5. The list of claims to use as the identity. The first non-empty claim is used.
6. The Issuer Identifier described in the OpenID spec. Must use **https** without query or fragment component.

Full OpenID Connect CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: oidcidp
    mappingMethod: claim
    type: OpenID
    openID:
      clientID: ...
      clientSecret:
        name: idp-secret
      ca: 1
        name: ca-config-map
      extraScopes: 2
      - email
      - profile
      extraAuthorizeParameters: 3
        include_granted_scopes: "true"
      claims:
        preferredUsername: 4
        - preferred_username
        - email
        name: 5
        - nickname
        - given_name
        - name
```

```
    email: 6
    - custom_email_claim
    - email
    groups: 7
    - groups
  issuer: https://www.idp-issuer.com
```

1. Optional: Reference to an OpenShift Container Platform config map containing the PEM-encoded certificate authority bundle to use in validating server certificates for the configured URL.
2. Optional: The list of scopes to request, in addition to the **openid** scope, during the authorization token request.
3. Optional: A map of extra parameters to add to the authorization token request.
4. The list of claims to use as the preferred user name when provisioning a user for this identity. The first non-empty claim is used.
5. The list of claims to use as the display name. The first non-empty claim is used.
6. The list of claims to use as the email address. The first non-empty claim is used.
7. The list of claims to use to synchronize groups from the OpenID Connect provider to OpenShift Container Platform upon user login. The first non-empty claim is used.

Additional resources

- See Identity provider parameters for information on parameters, such as **mappingMethod**, that are common to all identity providers.

7.9.7. Adding an identity provider to your cluster

Apply the identity provider custom resource (CR) to your cluster so users can authenticate with the configured identity provider.

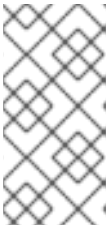
Prerequisites

- You installed an OpenShift Container Platform cluster.
- You defined the CR for your identity provider.
- You are logged in as an administrator.

Procedure

1. Apply the defined CR by running the following command:

```
$ oc apply -f </path/to/CR>
```



NOTE

If a CR does not exist, **oc apply** creates a new CR and might trigger the following warning: **Warning: oc apply should be used on resources created by either oc create --save-config or oc apply**. In this case you can safely ignore this warning.

2. Obtain a token from the OAuth server.
   As long as the **kubeadmin** user has been removed, the **oc login** command provides instructions on how to access a web page where you can retrieve the token.

   You can also access this page from the web console by navigating to **(?) Help → Command Line Tools → Copy Login Command**.

3. Log in to the cluster by running the following command and pass in the token to authenticate:

```
$ oc login --token=<token>
```

NOTE

If your OpenID Connect identity provider supports the resource owner password credentials (ROPC) grant flow, you can log in with a username and password. You might need to take steps to enable the ROPC grant flow for your identity provider.

After the OIDC identity provider is configured in OpenShift Container Platform, you can log in by using the following command, which prompts for your username and password:

```
$ oc login -u <identity_provider_username> --server=
<api_server_url_and_port>
```

4. Confirm that the user logged in successfully and that the username displays by running the following command:

```
$ oc whoami
```

7.9.8. Configuring identity providers using the web console

Configure your identity provider (IDP) through the web console instead of the CLI.

Prerequisites

- You must be logged in to the web console as a cluster administrator.

Procedure

1. Navigate to **Administration → Cluster Settings**.
2. Under the **Configuration** tab, click **OAuth**.
3. Under the **Identity Providers** section, select your identity provider from the **Add** drop-down menu.

NOTE

You can specify multiple IDPs through the web console without overwriting existing IDPs.

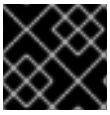
CHAPTER 8. ENABLING DIRECT AUTHENTICATION WITH AN EXTERNAL OIDC IDENTITY PROVIDER

Configure OpenShift Container Platform to use an external OpenID Connect (OIDC) identity provider directly for token-based authentication, replacing the built-in OAuth server with your organization's existing identity infrastructure.

8.1. ABOUT DIRECT AUTHENTICATION WITH AN EXTERNAL OIDC IDENTITY PROVIDER

You can enable direct integration with an external OpenID Connect (OIDC) identity provider to issue tokens for authentication. This bypasses the built-in OAuth server and uses the external identity provider directly.

By integrating directly with an external OIDC provider, you can leverage the advanced capabilities of your preferred OIDC provider instead of being limited by the capabilities of the built-in OAuth server. Your organization can manage users and groups from a single interface, while also streamlining authentication across multiple clusters and in hybrid environments. You can also integrate with existing tools and solutions.



IMPORTANT

Currently, you may configure only one OIDC provider for direct authentication.

After switching to direct authentication, existing authentication configuration is not guaranteed to be preserved. Before enabling direct authentication, back up any existing user, group, oauthclient, or identity provider configuration in case you need to revert back to using the built-in OAuth server for authentication.

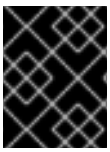
Before replacing the built-in OAuth server with an external provider, ensure that you have access to a long-lived method of logging in with cluster administrator permissions, such as one of the following:

- a certificate-based user **kubeconfig** file, such as the one generated by the installation program
- a long-lived service account token **kubeconfig** file
- a certificate-based service account **kubeconfig** file

If there are any issues with the external identity provider, you need one of these methods to gain access to the OpenShift Container Platform cluster in an emergency situation.

8.1.1. Disabled OAuth resources

When you enable direct authentication, several OAuth resources are intentionally removed.



IMPORTANT

Ensure that you do not rely on these removed resources before configuring direct authentication.

The following resources are unavailable when direct authentication is configured:

- OpenShift OAuth server and OpenShift OAuth API server

- User and group APIs (***.user.openshift.io**)
- OAuth APIs (***.oauth.openshift.io**)
- OAuth server and client configurations

8.1.2. Direct authentication identity providers

Direct authentication has been tested with multiple OpenID Connect (OIDC) identity providers to help you verify compatibility before configuring your cluster.

The following identity providers have been tested:

- Active Directory Federation Services for Windows Server
- GitLab
- Google
- Keycloak
- Microsoft Entra ID
- Okta
- Ping Identity
- Red Hat Single Sign-On

NOTE

Red Hat does not test all factors associated with third-party identity provider functionality.

Additional resources

- Red Hat third-party support policy

8.2. CONFIGURING AN EXTERNAL OIDC IDENTITY PROVIDER FOR DIRECT AUTHENTICATION

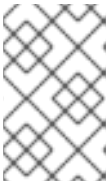
Configure OpenShift Container Platform to use an external OIDC identity provider for direct authentication, enabling users to log in with existing corporate credentials while bypassing the built-in OAuth server for streamlined single sign-on.

Prerequisites

- You have configured your external authentication provider.
  This procedure uses Keycloak as the identity provider and assumes that you have the following clients configured:
  - A confidential client for the web console called **console-test** with the valid redirect URIs set to **https://<openshift_console_route>/auth/callback**

- A public client for the OpenShift CLI (**oc**) called **oc-cli-test** with the valid redirect URIs set to **http://localhost:8080**

- You have access to the **kubeconfig** file generated by the installation program for the cluster.
- You have backed up any existing authentication configuration, in case you need to revert back to using the built-in OAuth server for authentication.
- You have the OpenShift Container Platform web console enabled on the cluster.



NOTE

If the web console is not enabled in your cluster, see "Example OIDC provider configuration for CLI clients only" for an example configuration without the web console client.

Procedure

1. Ensure that you are using the **kubeconfig** file generated by the installation program, or another long-lived method of logging in as a cluster administrator.
2. Create a secret that allows you to authenticate with the web console by running the following command:

```
$ oc create secret generic console-secret \
    --from-literal=clientSecret=<secret_value> \//
    -n openshift-config
```

   Replace **<secret_value>** with the value of the secret for the **console-test** client in your identity provider.

3. Optional: Create a config map that contains the provider's certificate authority bundle by running the following command:

```
$ oc create configmap keycloak-oidc-ca --from-file=ca-bundle.crt=my-directory/ca-bundle.crt
\
    -n openshift-config
```

   Specify the path to your provider's **ca-bundle.crt** file.

4. Edit the authentication configuration by running the following command:

```
$ oc edit authentication.config/cluster
```

5. Update the authentication configuration by setting the **type** field to **OIDC**, configuring the **oidcProviders** field for your provider, and setting the **webhookTokenAuthenticator** field to **null**:

```
apiVersion: config.openshift.io/v1
kind: Authentication
metadata:
# ...
spec:
  type: OIDC
```

```
    webhookTokenAuthenticator: null
    oidcProviders:
    - claimMappings:
        extra:
        - key: example.com/role
          valueExpression: claims.?role.orValue("unknown")
        groups:
          claim: groups
          prefix: 'oidc-groups-test:'
        uid:
          claim: "sub"
        username:
          claim: email
          prefixPolicy: Prefix
          prefix:
            prefixString: 'oidc-user-test:'
      issuer:
        audiences:
        - console-test
        - oc-cli-test
        issuerCertificateAuthority:
          name: keycloak-oidc-ca
        issuerURL: https://keycloak-keycloak.apps.example.com/realms/master
      name: 'keycloak-oidc-server'
      oidcClients:
      - clientID: oc-cli-test
        componentName: cli
        componentNamespace: openshift-console
      - clientID: console-test
        clientSecret:
          name: console-secret
        componentName: console
        componentNamespace: openshift-console
        extraScopes:
          - email
          - profile
```

where:

spec.type

Specifies the authentication type. Must be set to **OIDC** to indicate to use an external OIDC identity provider.

spec.webhookTokenAuthenticator

Specifies the webhook token authenticator configuration. Must be set to **null** when **type** is set to **OIDC**.

spec.oidcProviders

Specifies the OIDC provider configuration. Currently, only one OIDC provider configuration is allowed.

spec.oidcProviders.claimMappings.extra

Specifies the mappings used to construct the extra attributes for the cluster identity. This field is optional.

spec.oidcProviders.claimMappings.groups.claim

Specifies the name of the claim to construct group names for the cluster identity.

spec.oidcProviders.claimMappings.uid

Specifies the claim mapping used to construct the UID for the cluster identity. This field is optional.

spec.oidcProviders.claimMappings.username.claim

Specifies the name of the claim to construct usernames for the cluster identity.

spec.oidcProviders.issuer.audiences

Specifies the list of audiences that this authentication provider issues tokens for.

spec.oidcProviders.issuer.issuerCertificateAuthority.name

Specifies the name of the config map that contains the **ca-bundle.crt** key. If unset, system trust is used instead.

spec.oidcProviders.issuer.issuerURL

Specifies the URL for the token issuer.

spec.oidcProviders.name

Specifies the name for external OIDC provider.

spec.oidcProviders.oidcClients.clientID

Specifies the client ID that your provider uses. Configure separate entries for the OpenShift CLI (**oc**) and the OpenShift Container Platform web console.

spec.oidcProviders.oidcClients.clientSecret.name

Specifies the name of the secret that stores the secret value for the console client.

spec.oidcProviders.oidcClients.extraScopes

Specifies the extra scopes to request. Some providers, such as GitLab, might require extra scopes in order to log in through the web console properly.

6. Exit and save the changes to apply the new configuration.

7. Wait for the cluster to roll out new revisions to all nodes.

   a. Check the Kubernetes API server Operator status by running the following command:

   ```
   $ oc get co kube-apiserver
   ```

   Example output

   ```
   NAME             VERSION   AVAILABLE   PROGRESSING   DEGRADED   SINCE   MESSAGE
   kube-apiserver   4.22.0    True        True          False      85m     NodeInstallerProgressing: 2 node are at revision 8; 1 node is at revision 10
   ```

   The message in the preceding example shows that one node has progressed to the new revision and two nodes have not yet updated. It can take 20 minutes or more to roll out the new revision to all nodes, depending on the size of your cluster.

   b. To troubleshoot any issues, you can also check the Cluster Authentication Operator and **kube-apiserver** pod logs for errors.

Verification

1. Verify that you can log in to the OpenShift CLI (**oc**) by authenticating with your identity provider:

   a. Log in by running the following command:

   ```
   $ oc login --exec-plugin=oc-oidc \
     --issuer-url=https://keycloak-keycloak.apps.example.com/realms/master \
     --client-id=oc-cli-test \
     --extra-scopes=email --callback-port=8080 \
     --oidc-certificate-authority my-directory/ca-bundle.crt
   ```

   where:

   --exec-plugin

   Specifies the exec plugin type. Only a value of **oc-oidc** is allowed.

   --issuer-url

   Specifies the issuer URL for your identity provider.

   --client-id

   Specifies the client ID for the OpenShift CLI (**oc**).

   --oidc-certificate-authority

   Specifies the path to the **ca-bundle.crt** file on your local machine.

   Example output

   ```
   Please visit the following URL in your browser: http://localhost:8080
   ```

   b. Open http://localhost:8080 in a browser.

   c. Authenticate with credentials from your identity provider.
   After successfully authenticating, you should see a message similar to the following output in your terminal:

   ```
   Logged into "https://api.my-cluster.example.com:6443" as "oidc-user-
   test:user1@example.com" from an external oidc issuer.
   ```

2. Verify that you can log in to the OpenShift Container Platform web console by authenticating with your identity provider:

   a. Open the web console URL for your cluster in a browser.
   You are redirected to your identity provider to log in.

   b. Authenticate with credentials from your identity provider.
   Verify that you logged in successfully and are redirected to the OpenShift Container Platform web console.

Additional resources

- Example OIDC provider configuration for CLI clients only
- Configuring advanced direct authentication fields

8.2.1. OIDC provider configuration parameters

Configure OIDC providers for external authentication by using these parameters to map JWT token claims to cluster identities, validate authentication tokens, and enable platform components to authenticate with identity providers.

The following table lists all available OIDC provider parameters for direct authentication:

Table 8.1. oidcProviders configuration

Parameter	Description
claimMappings	Configures the rules to be used by the Kubernetes API server for translating claims in a JSON web token (JWT), issued by the identity provider, to a cluster identity.
claimMappings.extra	An optional field for configuring the mappings used to construct the extra attribute for the cluster identity. When omitted, no extra attributes will be present on the cluster identity. Key values for extra mappings must be unique. A maximum of 32 extra attribute mappings can be provided.
claimMappings.extra.key	A required field that specifies the string to use as the extra attribute key. The following restrictions apply: • Key must be a domain-prefix path (e.g **example.org/foo**). • Key must not exceed 510 characters in length. • Key must contain the / character, separating the domain and path characters. • Key must not be empty. • The domain portion of the key (string of characters before the /) must be a valid RFC1123 subdomain. • It must not exceed 253 characters in length. • It must start and end with an alphanumeric character. • It must only contain lower case alphanumeric characters and **-** or **.**. • It must not use the reserved domains, or be subdomains of, **kubernetes.io**, **k8s.io**, and **openshift.io**. • The path portion of the key (string of characters after the /) must not be empty and must consist of at least one alphanumeric character, percent-encoded octets, **-**, **.**, **_**, **~**, **!**, **$**, **&**, **'**, **(**, **)**, *****, **+**, **,**, **;**, **=**, and **:**. • Domain portion of the key must not exceed 256 characters in length.

Parameter	Description
claimMappings.extra.valueExpression	A required field to specify the CEL expression to extract the extra attribute value from claims of a JWT token. The **valueExpression** field must produce a string or string array value. The following restrictions apply: • CEL expressions that return "", [], and null are treated as the extra mapping not being present. • Empty string values within an array are filtered out. For example, [**one**, **`**, **`three**] becomes [**one**, **three**]. • CEL expressions have access to the token claims through a CEL variable, **claims**. • **claims** is a map of claim names to claim values. For example, the **sub** claim value can be accessed as**claims.sub**. • Nested claims can be accessed using dot notation (**claims.foo.bar**). • The **valueExpression** value must not exceed 1024 characters in length. • The **valueExpression** value must not be empty.
claimMappings.groups	Configures how the groups of a cluster identity should be constructed from the claims in a JWT token issued by the identity provider. When referencing a claim, if the claim is present in the JWT token, its value must be a comma-separated list of groups.
claimMappings.groups.claim	Optional parameter. JWT token claim used for groups mapping. Set either **claim** or **expression**, not both. Length: 1-256 characters.
claimMappings.groups.expression	Optional parameter (Technology Preview). CEL expression that produces a string or string array from JWT token claims. Access claims by using the **claims** variable (for example, **claims.groups** or **claims.foo.bar** for nested claims). Set either **claim** or **expression**, not both. Length: 1-1024 characters.
claimMappings.groups.prefix	Configures the prefix that is applied to the cluster identity attribute during the process of mapping JWT claims to cluster identity attributes.
claimMappings.uid	An optional field for configuring the claim mapping used to construct the UID for the cluster identity. When omitted, this means the user has no opinion and the platform is left to choose a default, which is subject to change over time. The current default is to use the **sub** claim.

Parameter	Description
claimMappings.uid.claim	An optional field for specifying the JWT token claim that is used in the mapping. The value of this claim will be assigned to the field in which this mapping is associated. To specify the claim, use a single string value for **uid.claim**. You must set either **claim** or **expression**. Do not specify **claim** when **expression** is set. The value of **claim** must be at least 1 character and must not exceed 256 characters in length.
claimMappings.uid.expression	An optional field for specifying a CEL expression that produces a string value from JWT token claims. When using **uid.expression** the expression must result in a single string value. CEL expressions have access to the token claims through a CEL variable, **claims**. The **claims** variable is a map of claim names to claim values. For example, you can access the **sub** claim value as **claims.sub**. Nested claims can be accessed using dot notation for example, **claims.foo.bar**. You must set either **claim** or **expression**. Do not specify **expression** when **claim** is set. The value of **expression** must be at least 1 character and must not exceed 1024 characters in length.
claimMappings.username	Configures how the username of a cluster identity should be constructed from the claims in a JWT token issued by the identity provider.
claimMappings.username.claim	Optional parameter. JWT token claim used for username mapping. Set either **claim** or **expression**, not both. Length: 1-256 characters.
claimMappings.username.expression	Optional parameter (Technology Preview). CEL expression that produces a string value from JWT token claims. Must result in a single string. Access claims by using the **claims** variable (for example, **claims.email** or **claims.foo.bar** for nested claims). Set either **claim** or **expression**, not both. Length: 1-1024 characters.
claimMappings.username.prefix	Configures the prefix that should be prepended to the value of the JWT claim. Must be set when **prefixPolicy** is set to **Prefix** and must be unset otherwise.
claimMappings.username.prefix.prefixString	Configures the prefix that is applied to the cluster identity username attribute during the process of mapping JWT claims to cluster identity attributes. Must not be an empty string (**""**).

Parameter	Description
claimMappings.username.prefixPolicy	Configures how a prefix should be applied to the value of the JWT claim specified in the **claim** field. Allowed values are **Prefix**, **NoPrefix**, and omitted (not provided or an empty string). When set to **Prefix**, the value specified in the prefix field is prepended to the value of the JWT claim. The prefix field must be set when **prefixPolicy** is **Prefix**. When set to **NoPrefix**, no prefix is prepended to the value of the JWT claim. When omitted, this means no opinion and the platform is left to choose any prefixes that are applied which is subject to change over time. Currently, the platform prepends **{issuerURL}#** to the value of the JWT claim when the claim is not **email**.
claimValidationRules	Configures the rules to be used by the Kubernetes API server for validating the claims in a JWT token issued by the identity provider. Validation rules are joined by an **AND** operation.
claimValidationRules.cel	Optional parameter (Technology Preview). Required when **type** is **CEL**. Contains **expression** (CEL expression to evaluate) and **message** (error text).
claimValidationRules.cel.expression	Technology Preview. CEL expression that validates token claims. Must evaluate to **true** for authentication to succeed. Access claims by using the **claims** variable using dot notation (for example, **claims.sub** or **claims.foo.bar**). Constraints: 1-1024 characters.
claimValidationRules.cel.message	Technology Preview. Error message displayed when validation fails. Constraints: 1-256 characters.
claimValidationRules.requiredClaim	Configures the required claim and value that the Kubernetes API server uses to validate if an incoming JWT is valid for this identity provider. Required when **type** is set to **RequiredClaim**.
claimValidationRules.requiredClaim.claim	Configures the name of the required claim. When taken from the JWT claims, the claim must be a string value. Must not be an empty string (**""**).
claimValidationRules.requiredClaim.requiredValue	Configures the value that **claim** must have when taken from the incoming JWT claims. If the value in the JWT claims does not match, the token is rejected for authentication. Must not be an empty string (**""**).

Parameter	Description
claimValidationRules.type	Validation rule type. Allowed values: **RequiredClaim** and **CEL**. • **RequiredClaim** - Validates that the JWT contains the required claim with the required value • **CEL** (Technology Preview) - Validates the JWT against a CEL expression
issuer	A required field that configures how the platform interacts with the identity provider and how tokens issued from the identity provider are evaluated by the Kubernetes API server.
issuer.audiences	A required field that configures the acceptable audiences the JWT token, issued by the identity provider, must be issued to. At least one of the entries must match the **aud** claim in the JWT token. Must contain at least one entry and must not exceed 10 entries.
issuer.discoveryURL	Optional parameter (Technology Preview). Custom OIDC discovery endpoint URL. Must be a valid HTTPS URL and differ from **issuer.issuerURL**. When not specified, OpenShift Container Platform constructs the discovery URL by using the standard OIDC format: **{issuerURL}/.well-known/openid-configuration**.
issuer.issuerCertificateAuthority	Configures the certificate authority, used by the Kubernetes API server, to validate the connection to the identity provider when fetching discovery information. When not specified, the system trust is used. When specified, it must reference a config map in the **openshift-config** namespace containing the PEM-encoded CA certificates under the **ca-bundle.crt** key in the **data** field of the config map.
issuer.issuerCertificateAuthority.name	The name of the referenced config map.
issuer.issuerURL	Configures the URL used to issue tokens by the identity provider. The Kubernetes API server determines how authentication tokens should be handled by matching the **iss** claim in the JWT to the issuerURL of configured identity providers. This field is required and must use the **https://** scheme.
name	A required field that configures the unique human-readable identifier associated with the identity provider. It is used to distinguish between multiple identity providers and has no impact on token validation or authentication mechanics. Must not be an empty string (**""**).
oidcClients	Configures how on-cluster, platform clients should request tokens from the identity provider. Must not exceed 20 entries and entries must have unique namespace/name pairs.

Parameter	Description
oidcClients.clientID	Configures the client identifier, from the identity provider, that the platform component uses for authentication requests made to the identity provider. The identity provider must accept this identifier for platform components to be able to use the identity provider as an authentication mode. Must not be an empty string (**""**).
oidcClients.clientSecret	Configures the client secret used by the platform component when making authentication requests to the identity provider. When not specified, no client secret is used when making authentication requests to the identity provider. When specified, it references a secret in the **openshift-config** namespace that contains the client secret in the **clientSecret** key of the **.data** field. The client secret is used when making authentication requests to the identity provider. Public clients do not require a client secret, but private clients do require a client secret to work with the identity provider.
oidcClients.clientSecret.name	The name of the referenced secret.
oidcClients.componentName	Specifies the name of the platform component being configured to use the identity provider as an authentication mode. It is used in combination with **componentNamespace** as a unique identifier. Must not be an empty string (**""**) and must not exceed 256 characters in length.
oidcClients.componentNamespace	Specifies the namespace in which the platform component being configured to use the identity provider as an authentication mode is running. It is used in combination with **componentName** as a unique identifier. Must not be an empty string (**""**) and must not exceed 63 characters in length.
oidcClients.extraScopes	Configures the extra scopes that should be requested by the platform component when making authentication requests to the identity provider. This is useful if you have configured claim mappings that require specific scopes to be requested beyond the standard OIDC scopes. When omitted, no additional scopes are requested.
userValidationRules	Optional parameter (Technology Preview). Validation rules for user objects created from authenticated tokens. All rules must pass (AND operation). Each rule contains **expression** (must evaluate to **true**) and **message** (error text). Access user by using the **user** variable: **user.username** (string), **user.groups** (array), **user.uid** (string), **user.extra** (map).

Parameter	Description
userValidationRules[].expression	Required. CEL expression that validates the user object. Must evaluate to **true** for authentication to succeed. Constraints: 1-1024 characters, boolean result.
userValidationRules[].message	Required. Error message displayed when validation fails.

8.2.2. Example OIDC provider configuration for CLI clients only

In OpenShift Container Platform clusters where the web console is disabled, you can configure direct authentication with an external OIDC provider for a CLI client only. In these cases, users must authenticate with the cluster directly through the OpenShift CLI (**oc**) instead of through the web console.

The following example OIDC provider configuration shows how to configure a CLI client without defining a web console client:

OIDC provider configuration with only a CLI client

```
apiVersion: config.openshift.io/v1
kind: Authentication
metadata:
# ...
spec:
  type: OIDC
  webhookTokenAuthenticator: null
  oidcProviders:
  - claimMappings:
      groups:
        claim: groups
        prefix: 'oidc-groups-test:'
      username:
        claim: email
        prefixPolicy: Prefix
        prefix:
          prefixString: 'oidc-user-test:'
    issuer:
      audiences:
      - my-cli-client-id
      issuerURL: my-issuer-url
    name: my-oidc-provider-name
```

8.3. DISABLING DIRECT AUTHENTICATION

Disable direct authentication to revert your cluster back to using the built-in OpenShift Container Platform OAuth server for authentication when external OIDC integration is no longer needed.

Prerequisites

- You have access to the **kubeconfig** file generated by the installation program for the cluster.

Procedure

1. Ensure that you are using the **kubeconfig** file generated by the installation program, or another long-lived method of logging in as a cluster administrator.

2. Update the authentication configuration to use the built-in OpenShift Container Platform OAuth server by running the following command:

```
$ oc patch authentication.config/cluster --type=merge -p='
spec:
  type: ""
  oidcProviders: null
'
```

   where:

   spec.type

   Specifies the authentication type. Set to **""** to use the built-in OpenShift Container Platform OAuth server. A value of **IntegratedOAuth** is also equivalent.

   spec.oidcProviders

   Specifies the OIDC provider configuration. Set to **null** to remove the external OIDC provider configuration.

3. Wait for the cluster to roll out new revisions to all nodes.

   a. Check the Kubernetes API server Operator status by running the following command:

```
$ oc get co kube-apiserver
```

   Example output

```
NAME             VERSION   AVAILABLE   PROGRESSING   DEGRADED   SINCE   MESSAGE
kube-apiserver   4.22.0    True        True          False      85m     NodeInstallerProgressing: 2 node are at revision 12; 1 node is at revision 14
```

   The message in the preceding example shows that one node has progressed to the new revision and two nodes have not yet updated. It can take 20 minutes or more to roll out the new revision to all nodes, depending on the size of your cluster.

   b. To troubleshoot any issues, you can also check the Cluster Authentication Operator and **kube-apiserver** pod logs for errors.

4. If necessary, restore any existing authentication configuration.

Verification

- Verify that you can successfully log in to the OpenShift Container Platform web console and OpenShift CLI (**oc**).

CHAPTER 9. CONFIGURING ADVANCED DIRECT AUTHENTICATION FIELDS

You can configure advanced direct authentication fields in the **authentications.config.openshift.io** custom resource definition (CRD) to enable enhanced OIDC configurations, security enforcement, and flexible token validation for standalone and hosted control plane (HCP) clusters.

IMPORTANT

Advanced direct authentication fields is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

9.1. ABOUT ADVANCED DIRECT AUTHENTICATION FIELDS

Advanced direct authentication fields provide flexibility and security for OIDC-based authentication in OpenShift Container Platform. You can configure advanced OIDC settings, implement custom token validation logic, and enforce security policies on usernames and groups.

The following capabilities are available:

Custom OIDC discovery URL

Specify a custom OIDC discovery endpoint when your identity provider does not use the standard discovery URL format. Useful for complex networking setups or non-standard identity providers.

CEL-based claim mapping

Use Common Expression Language (CEL) expressions to construct usernames and groups from JWT token claims with fallback logic. This addresses scenarios where different user types require varying claim mappings.

Claim validation rules

Use CEL expressions to implement advanced token validation logic, such as enforcing maximum token lifetimes, validating multiple claims, or implementing custom security policies.

User validation rules

Enforce security policies on usernames and groups extracted from tokens to prevent privilege escalation by blocking reserved system usernames and group prefixes.

These fields extend the base OIDC authentication configuration introduced in OpenShift Container Platform 4.14. You must first configure an external OIDC identity provider before using these advanced fields.

These fields require the **TechPreviewNoUpgrade** feature set to be enabled. They are available on standalone OpenShift Container Platform clusters and hosted control plane (HCP) environments.

IMPORTANT

These advanced authentication fields are available as a Technology Preview feature. Ensure you have a backup authentication method, such as a certificate-based kubeconfig file, before configuring these fields.

9.2. CONFIGURING A CUSTOM OIDC DISCOVERY URL

Configure a custom OIDC discovery URL when your identity provider does not follow the standard discovery endpoint format.

Prerequisites

- You have configured an external OIDC identity provider for direct authentication.
- You have access to the cluster as a user with the **cluster-admin** role.
- You have access to a long-lived authentication method, such as a certificate-based kubeconfig file.

Procedure

1. Create a YAML file named **authentication-discovery-url.yaml** with your custom discovery URL configuration:

```
apiVersion: config.openshift.io/v1
kind: Authentication
metadata:
  name: cluster
spec:
  type: OIDC
  oidcProviders:
  - name: my-oidc-provider
    issuer:
      issuerURL: https://idp.example.com
      discoveryURL: https://custom-discovery.example.com/.well-known/openid-configuration
      audiences:
      - my-audience
    claimMappings:
      username:
        claim: email
```

where:

issuerURL

Specifies the issuer URL displayed in JWT token **iss** claim.

discoveryURL

Specifies the custom OIDC discovery endpoint URL. Must differ from **issuerURL**, use HTTPS, and must not contain query parameters, fragments, or user info. Maximum length: 2048 characters.

NOTE

Replace the placeholder values (**my-oidc-provider**, **https://idp.example.com**, **https://custom-discovery.example.com/.well-known/openid-configuration**, **my-audience**) with your actual OIDC provider configuration.

2. Apply the configuration:

```
$ oc apply -f authentication-discovery-url.yaml
```

Verification

- Monitor the cluster authentication Operator status to ensure the configuration is applied successfully:

```
$ oc get clusteroperator authentication
```

  The Operator should report **Available=True** and **Degraded=False**.

- Check the cluster authentication Operator logs for any errors:

```
$ oc logs -n openshift-authentication-operator deployments/authentication-operator
```

- Verify the kube-apiserver is using the custom discovery URL by checking the authentication configuration:

```
$ oc get configmap kube-apiserver-to-kubelet-client-ca -n openshift-kube-apiserver -o yaml
```

```
$ oc get authentication.config.openshift.io/cluster -o
jsonpath='{.spec.oidcProviders[0].issuer.discoveryURL}'
```

  The output should display your custom discovery URL.

9.3. CONFIGURING CEL EXPRESSIONS FOR USERNAME AND GROUPS CLAIM MAPPING

You can use Common Expression Language (CEL) expressions to construct usernames and groups from JWT token claims. This provides flexible claim mapping, including fallback logic when specific claims are not present.

Prerequisites

- You have configured an external OIDC identity provider for direct authentication.
- You have access to the cluster as a user with the **cluster-admin** role.
- You have access to a long-lived authentication method, such as a certificate-based kubeconfig file.
- You are familiar with CEL expression syntax.

Procedure

1. Create a YAML file named **authentication-cel-mapping.yaml** with your CEL expression configuration:

IMPORTANT

When using **expression**, do not set the **claim** field. You must use either **claim** or **expression**, but not both. Setting both will result in a validation error. Additionally, when using **expression**, do not set **prefixPolicy** to **Prefix**. Prefix policies are only compatible with **claim**-based mappings.

NOTE

When using the **email** claim in CEL expressions, you must also validate **email_verified** to ensure the email address has been verified by the identity provider.

```
apiVersion: config.openshift.io/v1
kind: Authentication
metadata:
  name: cluster
spec:
  type: OIDC
  oidcProviders:
  - name: my-oidc-provider
    issuer:
      issuerURL: https://idp.example.com
      audiences:
      - my-audience
    claimMappings:
      username:
        expression: 'claims.?upn.orValue(claims.?oid.orValue(claims.sub))'
      groups:
        expression: 'claims.?groups.orValue([])'
# ...
```

where:

username.expression

Specifies the fallback logic for username: **upn** if present, else **oid**, else **sub**.

groups.expression

Specifies that the **groups** claim is used if present, else an empty array.

NOTE

Replace the placeholder values (**my-oidc-provider**, **https://idp.example.com**, **my-audience**) with your actual OIDC provider configuration.

2. Apply the configuration:

```
$ oc apply -f authentication-cel-mapping.yaml
```

Verification

- Verify that the authentication configuration is applied successfully:

```
$ oc get authentication.config.openshift.io/cluster -o yaml
```

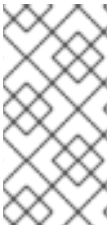
- Authenticate with a user account and verify the username is constructed correctly:

```
$ oc whoami
```

- Monitor the cluster authentication Operator status:

```
$ oc get clusteroperator authentication
```

  The Operator should report **Available=True** and **Degraded=False**.



NOTE

CEL expressions have access to standard CEL string functions (**lowerAscii()**, **upperAscii()**, **contains()**, **startsWith()**, **endsWith()**, **matches()**, **split()**), operators (**+**, **?**, **has()**, ternary **? :**), and the **orValue()** method for optional chaining. See the CEL specification link in Additional resources for the complete function reference.

You can use the following CEL expression patterns for claim mapping:

Use the optional chaining Operator? to safely access claims that might not exist

```
username:
  expression: 'claims.email_verified ? claims.email : claims.sub'
```

Uses **email** if verified, otherwise **sub**. When using the **email** claim, you must also check **email_verified**.

Concatenate multiple claims

```
username:
  expression: 'claims.givenname + "." + claims.surname'
```

Combines given name and surname claims.

Transform claim values

```
username:
  expression: 'claims.email.lowerAscii()'
```

Converts email to lowercase.

Conditional logic for different user types

```
username:
```

```
username:
  expression: 'has(claims.upn) ? claims.upn : claims.oid'
```

Uses **upn** for regular users, **oid** for service principals.

Extract domain from email

```
groups:
  expression: 'claims.?email.orValue("").split("@").size() > 1 ? [claims.email.split("@")[1]] : []'
```

Safely extracts domain from email address for group assignment, returning an empty array if email is missing or malformed.

Combine group sources

```
groups:
  expression: 'claims.?groups.orValue([]) + claims.?roles.orValue([])'
```

Combines **groups** and **roles** claims.

9.4. CONFIGURING CLAIM VALIDATION RULES

Use Common Expression Language (CEL) expressions to define custom validation rules for JWT token claims and enforce advanced security policies such as maximum token lifetimes.

WARNING

All claim validation rules must pass for authentication to succeed. Incorrectly configured validation rules can lock all users out of the cluster. Ensure you complete the following tasks:

- Have a backup authentication method, such as a certificate-based kubeconfig file, before applying these rules
- Test validation rules in a non-production environment first
- Verify your CEL expressions are correct to avoid blocking valid users from accessing the cluster

Prerequisites

- You have configured an external OIDC identity provider for direct authentication.
- You have access to the cluster as a user with the **cluster-admin** role.
- You have access to a long-lived authentication method, such as a certificate-based kubeconfig file.
- You are familiar with CEL expression syntax.

Procedure

1. Create a YAML file named **authentication-claim-validation.yaml** with your claim validation rules:

```
apiVersion: config.openshift.io/v1
kind: Authentication
metadata:
  name: cluster
spec:
  type: OIDC
  oidcProviders:
  - name: my-oidc-provider
    issuer:
      issuerURL: https://idp.example.com
      audiences:
      - my-audience
    claimMappings:
      username:
        claim: email
    claimValidationRules:
    - type: CEL
      cel:
        expression: 'claims.exp - claims.nbf <= 86400'
        message: 'Total token lifetime must not exceed 24 hours'
    - type: CEL
      cel:
        expression: 'has(claims.email) && claims.email_verified &&
claims.email.contains("@example.com")'
        message: 'Email claim must be verified and from example.com domain'
```

where:

claimValidationRules

Specifies an array of validation rules. All must pass for authentication.

type

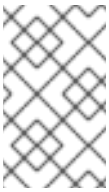
Specifies the validation type. Set to **CEL** for CEL-based validation.

cel.expression

Specifies the CEL expression that must evaluate to **true**.

cel.message

Specifies the error message displayed when validation fails.



NOTE

Replace the placeholder values (**my-oidc-provider**, **https://idp.example.com**, **my-audience**, **@example.com**) with your actual OIDC provider configuration and validation requirements.

2. Apply the configuration:

```
$ oc apply -f authentication-claim-validation.yaml
```

Verification

- Verify that the authentication configuration is applied successfully:

```
$ oc get authentication.config.openshift.io/cluster -o yaml
```

- Authenticate with a token that matches your validation rules to confirm they are enforced correctly.
- Check the cluster authentication Operator logs for validation errors:

```
$ oc logs -n openshift-authentication-operator deployments/authentication-operator
```

When writing CEL expressions for claim validation:

- Access claims using **claims** variable (for example, **claims.sub** or **claims.foo.bar** for nested claims)
- Expressions must evaluate to boolean values
- Use **has()** to check claim existence
- Standard CEL operators and functions available: **&&**, ||, **!**, **contains()**, **startsWith()**, **endsWith()**

Common use cases include:

Enforce maximum token lifetime

```
claimValidationRules:
- type: CEL
  cel:
    expression: 'claims.exp - claims.nbf <= 86400'
    message: 'Token lifetime must not exceed 24 hours'
```

Require specific claim values

```
claimValidationRules:
- type: CEL
  cel:
    expression: 'claims.tenant == "production"'
    message: 'Only production tenant tokens are allowed'
```

Validate email domain

```
claimValidationRules:
- type: CEL
  cel:
    expression: 'has(claims.email) && claims.email_verified && claims.email.endsWith("@trusted-domain.com")'
    message: 'Email must be verified and from trusted-domain.com'
```

When using the **email** claim, you must also check **email_verified** to ensure the email address has been verified by the identity provider.

Combine conditions

```
claimValidationRules:
- type: CEL
  cel:
    expression: 'has(claims.role) && (claims.role == "admin" || claims.role == "developer")'
    message: 'User must have admin or developer role'
```

Troubleshooting

If authentication fails after the configuration is applied

Even if your claim validation rules pass the configuration validation gates, runtime authentication errors can occur. Check the kube-apiserver logs for detailed error messages explaining why authentication failed:

```
$ oc logs -n openshift-kube-apiserver -l app=openshift-kube-apiserver | grep -i auth
```

These log messages will indicate which validation rule failed and include the custom **message** you specified in the CEL expression.

If you incorrectly configure validation rules and lock users out of the cluster

1. Use your certificate-based kubeconfig to authenticate as **cluster-admin**.
2. Edit the Authentication custom resource to remove or fix invalid rules:

   ```
   $ oc edit authentication.config.openshift.io/cluster
   ```

3. Monitor the cluster authentication Operator to confirm it returns to **Available** status:

   ```
   $ oc get clusteroperator authentication
   ```

9.5. CONFIGURING USER VALIDATION RULES

You can define validation rules to enforce security policies on the user object created from an authenticated token. This helps prevent privilege escalation by blocking reserved usernames and group prefixes.

NOTE

User validation rules are evaluated after claim mapping is complete, including all prefix transformations. Your CEL expressions must validate the final username and group names as they will appear in RBAC policies, not the raw claim values from the JWT token.

WARNING

All user validation rules must pass for authentication to succeed. Incorrectly configured validation rules can lock all users out of the cluster. Ensure you:

- Have a backup authentication method, such as a certificate-based kubeconfig file, before applying these rules
- Test validation rules in a non-production environment first
- Verify your CEL expressions correctly validate the final username and groups to avoid blocking valid users or allowing unauthorized access

Prerequisites

- You have configured an external OIDC identity provider for direct authentication.
- You have access to the cluster as a user with the **cluster-admin** role.
- You have access to a long-lived authentication method, such as a certificate-based kubeconfig file.
- You are familiar with CEL expression syntax.

Procedure

1. Create a YAML file named **authentication-user-validation.yaml** with your user validation rules:

```
apiVersion: config.openshift.io/v1
kind: Authentication
metadata:
  name: cluster
spec:
  type: OIDC
  oidcProviders:
  - name: my-oidc-provider
    issuer:
      issuerURL: https://idp.example.com
      audiences:
      - my-audience
    claimMappings:
      username:
        claim: email
      groups:
        claim: groups
    userValidationRules:
    - expression: "!user.username.startsWith('system:')"
      message: 'Username cannot use reserved system: prefix'
    - expression: "!user.groups.exists(g, g.startsWith('system:'))"
      message: 'Groups cannot use reserved system: prefix'
```

where:

userValidationRules

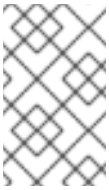
Specifies an array of validation rules. All must pass for authentication.

expression

Specifies the CEL expression that must evaluate to **true**.

message

Specifies the error message displayed when validation fails.



NOTE

Replace the placeholder values (**my-oidc-provider**, **https://idp.example.com**, **my-audience**) with your actual OIDC provider configuration.

2. Apply the configuration:

```
$ oc apply -f authentication-user-validation.yaml
```

Verification

- Verify that the authentication configuration is applied successfully:

```
$ oc get authentication.config.openshift.io/cluster -o yaml
```

- Authenticate with credentials that would create a user matching your validation rules to confirm they are enforced correctly.
- Check the cluster authentication Operator logs for validation errors:

```
$ oc logs -n openshift-authentication-operator deployments/authentication-operator
```

When writing CEL expressions for user validation:

- Access user fields: **user.username**, **user.groups** (array), **user.uid**, **user.extra** (map)
- Expressions must evaluate to boolean values
- Use **startsWith()**, **endsWith()**, **contains()** for string matching
- Use **exists()** for array checks (for example, **user.groups.exists(g, g == "admin")**)

Common use cases include:

Prevent reserved username prefixes

```
userValidationRules:
- expression: "!user.username.startsWith('system:')"
  message: 'Username cannot use reserved system: prefix'
```

Prevent reserved group prefixes

```
userValidationRules:
- expression: "!user.groups.exists(g, g.startsWith('system:'))"
  message: 'Groups cannot use reserved system: prefix'
```

Require username format

```
userValidationRules:
- expression: "user.username.matches('^[a-z0-9]([-a-z0-9]*[a-z0-9])?$')"
  message: 'Username must be a valid DNS subdomain'
```

Validate group membership

```
userValidationRules:
- expression: "user.groups.exists(g, g == 'verified-users')"
  message: 'User must be a member of verified-users group'
```

Combine conditions

```
userValidationRules:
- expression: "!user.username.startsWith('system:') && !user.username.startsWith('kube:')"
  message: 'Username cannot use reserved system: or kube: prefixes'
```

Troubleshooting

If you incorrectly configure validation rules and lock users out of the cluster:

1. Use your certificate-based kubeconfig to authenticate as **cluster-admin**.
2. Edit the Authentication custom resource to remove or fix invalid rules:

   ```
   $ oc edit authentication.config.openshift.io/cluster
   ```

3. Monitor the cluster authentication Operator to confirm it returns to **Available** status:

   ```
   $ oc get clusteroperator authentication
   ```

9.6. ADVANCED AUTHENTICATION FIELD REFERENCE

The following table describes the advanced authentication configuration fields available as Technology Preview in OpenShift Container Platform.

Table 9.1. Advanced oidcProviders configuration fields

Parameter	Description

Parameter	Description

issuer.discoveryURL	Optional parameter. Custom OIDC discovery endpoint URL for retrieving identity provider metadata from a non-standard location.
	
	Requirements:
	
	• Must be a valid HTTPS URL
	• Must differ from **issuer.issuerURL**
	
	When not specified, OpenShift Container Platform constructs the discovery URL by using the standard OIDC format: **{issuerURL}/.well-known/openid-configuration**.
	
	Example:
	
	`issuer:`
	`  issuerURL: https://idp.example.com`
	`  discoveryURL: https://custom-discovery.example.com/.well-known/openid-configuration`

claimValidationRules	Optional parameter. Array of validation rules for JWT token claims using Common Expression Language (CEL) expressions. All rules must evaluate to **true** for authentication to succeed (AND operation).
	
	Each rule has:
	
	• **type**: Set to **CEL** for CEL-based validation
	• **cel**: Object with **expression** (must evaluate to **true**) and **message** (error text)
	
	CEL expressions access claims by using **claims** variable (for example, **claims.sub**).
	
	Example:
	
	`claimValidationRules:`
	`- type: CEL`
	`  cel:`
	`  expression: 'claims.exp - claims.nbf <= 86400'`
	`  message: 'Total token lifetime must not exceed 24 hours'`
	`- type: CEL`
	`  cel:`
	`  expression: 'has(claims.email) && claims.email.contains("@example.com")'`
	`  message: 'Email claim must be present and from example.com domain'`

claimValidationRules[].type	Required. Validation rule type. Set to **CEL** for CEL-based validation. Requires **cel** field.

Parameter	Description
claimValidationRules[].cel	Required when **type** is **CEL**. Contains **expression** (CEL expression to evaluate) and **message** (error text).
claimValidationRules[].cel.expression	Required. CEL expression that validates token claims. Must evaluate to **true** for authentication to succeed. Constraints: 1-1024 characters, must evaluate to boolean. Access claims by using **claims** variable: **claims.sub**, **claims.foo.bar** (nested), **has(claims.email)** (existence check).  NOTE When using the **email** claim in CEL expressions, you must also validate the **email_verified** claim to ensure the email address has been verified by the identity provider. For example: **claims.email_verified && claims.email.endsWith("@example.com")**.
claimValidationRules[].cel.message	Required. Error message displayed when validation fails. Constraints: 1-256 characters.

userValidationRules	Optional parameter. Array of validation rules for user objects by using CEL expressions. All rules must evaluate to **true** for authentication to succeed (AND operation).
	
	Each rule has **expression** (must evaluate to **true**) and **message** (error text).
	
	CEL expressions access user object by using **user** variable: **user.username** (string), **user.groups** (array), **user.uid** (string), **user.extra** (map).
	
	Example:
	
	`userValidationRules:`
	`- expression: "!user.username.startsWith('system:')"`
	`  message: 'Username cannot use reserved system: prefix'`
	`- expression: "!user.groups.exists(g, g.startsWith('system:'))"`
	`  message: 'Groups cannot use reserved system: prefix'`

Parameter	Description
userValidationRules[].expression	Required. CEL expression that validates the user object. Must evaluate to **true** for authentication to succeed. Constraints: 1-1024 characters, must evaluate to boolean. Access user fields: **user.username.startsWith('system:')**, **user.groups.exists(g, g == "admin")**, **user.extra["example.com/role"]**.
userValidationRules[].message	Required. Error message displayed when validation fails. Must not be empty.

Additional resources

- Enabling direct authentication with an external OIDC identity provider
- Common Expression Language (CEL) specification
- Common Expression Language in Kubernetes

CHAPTER 10. USING RBAC TO DEFINE AND APPLY PERMISSIONS

10.1. RBAC OVERVIEW

You can use role-based access control to configure whether users and groups can perform specific actions on cluster or project resources by evaluating roles, rules, and bindings.

Role-based access control (RBAC) objects determine whether a user is allowed to perform a given action within a project.

Cluster administrators can use the cluster roles and bindings to control who has various access levels to the OpenShift Container Platform platform itself and all projects.

Developers can use local roles and bindings to control who has access to their projects. Note that authorization is a separate step from authentication, which is more about determining the identity of who is taking the action.

Authorization is managed using:

Authorization object	Description
Rules	Sets of permitted verbs on a set of objects. For example, whether a user or service account can **create** pods.
Roles	Collections of rules. You can associate, or bind, users and groups to multiple roles.
Bindings	Associations between users and/or groups with a role.

There are two levels of RBAC roles and bindings that control authorization:

RBAC level	Description
Cluster RBAC	Roles and bindings that are applicable across all projects. *Cluster roles* exist cluster-wide, and *cluster role bindings* can reference only cluster roles.
Local RBAC	Roles and bindings that are scoped to a given project. While *local roles* exist only in a single project, local role bindings can reference *both* cluster and local roles.

A cluster role binding is a binding that exists at the cluster level. A role binding exists at the project level. The cluster role *view* must be bound to a user using a local role binding for that user to view the project. Create local roles only if a cluster role does not provide the set of permissions needed for a particular situation.

This two-level hierarchy allows reuse across multiple projects through the cluster roles while allowing customization inside of individual projects through local roles.

During evaluation, both the cluster role bindings and the local role bindings are used. For example:

1. Cluster-wide "allow" rules are checked.
2. Locally-bound "allow" rules are checked.
3. Deny by default.

10.1.1. Default cluster roles

OpenShift Container Platform includes a set of default cluster roles that you can bind to users and groups cluster-wide or locally.

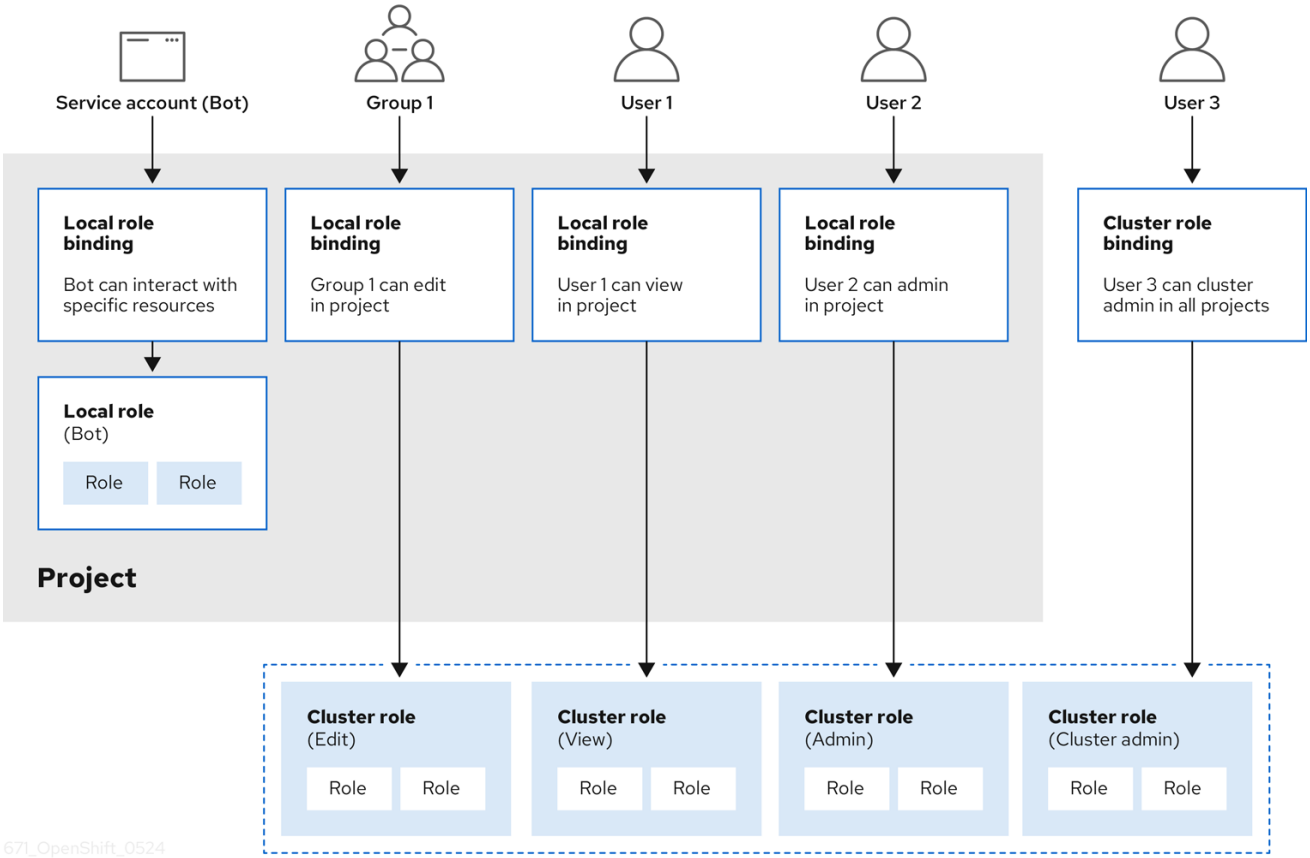
IMPORTANT

It is not recommended to manually modify the default cluster roles. Modifications to these system roles can prevent a cluster from functioning properly.

Default cluster role	Description
admin	A project manager. If used in a local binding, an **admin** has rights to view any resource in the project and modify any resource in the project except for quota.
basic-user	A user that can get basic information about projects and users.
cluster-admin	A super-user that can perform any action in any project. When bound to a user with a local binding, they have full control over quota and every action on every resource in the project.
cluster-status	A user that can get basic cluster status information.
cluster-reader	A user that can get or view most of the objects but cannot modify them.
edit	A user that can modify most objects in a project but does not have the power to view or modify roles or bindings.
self-provisioner	A user that can create their own projects.
view	A user who cannot make any modifications, but can see most objects in a project. They cannot view or modify roles or bindings.

Be mindful of the difference between local and cluster bindings. For example, if you bind the **cluster-admin** role to a user by using a local role binding, it might appear that this user has the privileges of a cluster administrator. This is not the case. Binding the **cluster-admin** to a user in a project grants super administrator privileges for only that project to the user. That user has the permissions of the cluster role **admin**, plus a few additional permissions like the ability to edit rate limits, for that project. This binding can be confusing via the web console UI, which does not list cluster role bindings that are bound to true cluster administrators. However, it does list local role bindings that you can use to locally bind **cluster-admin**.

The relationships between cluster roles, local roles, cluster role bindings, local role bindings, users, groups and service accounts are illustrated below.



WARNING

The **get pods/exec**, **get pods/***, and **get *** rules grant execution privileges when they are applied to a role. Apply the principle of least privilege and assign only the minimal RBAC rights required for users and agents. For more information, see "RBAC rules allow execution privileges".

10.1.2. Evaluating authorization

OpenShift Container Platform evaluates authorization by using:

Identity

The user name and list of groups that the user belongs to.

Action

The action you perform. In most cases, this consists of:

- **Project**: The project you access. A project is a Kubernetes namespace with additional annotations that allows a community of users to organize and manage their content in isolation from other communities.
- **Verb** : The action itself: **get**, **list**, **create**, **update**, **delete**, **deletecollection**, or **watch**.

- **Resource name**: The API endpoint that you access.

Bindings

The full list of bindings, the associations between users or groups with a role.

OpenShift Container Platform evaluates authorization by using the following steps:

1. The identity and the project-scoped action is used to find all bindings that apply to the user or their groups.
2. Bindings are used to locate all the roles that apply.
3. Roles are used to find all the rules that apply.
4. The action is checked against each rule to find a match.
5. If no matching rule is found, the action is then denied by default.

TIP

Remember that users and groups can be associated with, or bound to, multiple roles at the same time.

Project administrators can use the CLI to view local roles and bindings, including a matrix of the verbs and resources each are associated with.

IMPORTANT

The cluster role bound to the project administrator is limited in a project through a local binding. It is not bound cluster-wide like the cluster roles granted to the **cluster-admin** or **system:admin**.

Cluster roles are roles defined at the cluster level but can be bound either at the cluster level or at the project level.

10.1.3. Cluster role aggregation

The default admin, edit, view, and cluster-reader cluster roles support cluster role aggregation, where the cluster rules for each role are dynamically updated as new rules are created. This feature is relevant only if you extend the Kubernetes API by creating custom resources.

Additional resources

- RBAC rules allow execution privileges
- Aggregated ClusterRoles (Kubernetes documentation)

10.2. PROJECTS AND NAMESPACES

You can use projects and namespaces to organize and isolate cluster resources. These resources provide boundaries for access control, policies, quotas, and service accounts.

A Kubernetes *namespace* provides a mechanism to scope resources in a cluster. The Kubernetes documentation has more information on namespaces.

Namespaces provide a unique scope for:

- Named resources to avoid basic naming collisions.
- Delegated management authority to trusted users.
- The ability to limit community resource consumption.

Most objects in the system are scoped by namespace, but some are excepted and have no namespace, including nodes and users.

A *project* is a Kubernetes namespace with additional annotations and is the central vehicle by which access to resources for regular users is managed. A project allows a community of users to organize and manage their content in isolation from other communities. Users must be given access to projects by administrators, or if allowed to create projects, automatically have access to their own projects.

Projects can have a separate **name**, **displayName**, and **description**.

- The mandatory **name** is a unique identifier for the project and is most visible when using the CLI tools or API. The maximum name length is 63 characters.
- The optional **displayName** is how the project is displayed in the web console (defaults to **name**).
- The optional **description** can be a more detailed description of the project and is also visible in the web console.

Each project scopes its own set of:

Object	Description
Objects	Pods, services, replication controllers, etc.
Policies	Rules for which users can or cannot perform actions on objects.
Constraints	Quotas for each kind of object that can be limited.
Service accounts	Service accounts act automatically with designated access to objects in the project.

Cluster administrators can create projects and delegate administrative rights for the project to any member of the user community. Cluster administrators can also allow developers to create their own projects.

Developers and administrators can interact with projects by using the CLI or the web console.

Additional resources

- Kubernetes documentation on namespaces

10.3. DEFAULT PROJECTS

Default projects host critical cluster and infrastructure components. By understanding their purpose, you can avoid making changes that could disrupt essential cluster services.

OpenShift Container Platform includes several default projects, and projects starting with **openshift-** are the most essential to users. These projects host master components that run as pods and other infrastructure components. The pods created in these namespaces that have a critical pod annotation are considered critical, and they have guaranteed admission by kubelet. Pods created for master components in these namespaces are already marked as critical.

IMPORTANT

Do not run workloads in or share access to default projects. Default projects are reserved for running core cluster components.

The following default projects are considered highly privileged: **default**, **kube-public**, **kube-system**, **openshift**, **openshift-infra**, **openshift-node**, and other system-created projects that have the **openshift.io/run-level** label set to **0** or **1**. Functionality that relies on admission plugins, such as pod security admission, security context constraints, cluster resource quotas, and image reference resolution, does not work in highly privileged projects.

Additional resources

- Guaranteed Scheduling For Critical Add-On Pods (Kubernetes documentation)

10.4. VIEWING CLUSTER ROLES AND BINDINGS

You can view cluster roles and bindings by using the **oc** CLI to determine the permissions associated with roles and identify the users, groups, and service accounts assigned to them.

You can use the **oc** CLI to view cluster roles and bindings by using the **oc describe** command.

Prerequisites

- Install the **oc** CLI.
- Obtain permission to view the cluster roles and bindings.

Users with the **cluster-admin** default cluster role bound cluster-wide can perform any action on any resource, including viewing cluster roles and bindings.

Procedure

1. To view the cluster roles and their associated rule sets:

```
$ oc describe clusterrole.rbac
```

Example output

```
Name:         admin
Labels:       kubernetes.io/bootstrapping=rbac-defaults
Annotations:  rbac.authorization.kubernetes.io/autoupdate: true
PolicyRule:
  Resources                                  Non-Resource URLs  Resource Names  Verbs
```

```
---------                                  ----------------  --------------  -----
.packages.apps.redhat.com                  []                []              [* create update
patch delete get list watch]
imagestreams                               []                []              [create delete
deletecollection get list patch update watch create get list watch]
imagestreams.image.openshift.io            []                []              [create delete
deletecollection get list patch update watch create get list watch]
secrets                                    []                []              [create delete deletecollection
get list patch update watch get list watch create delete deletecollection patch update]
buildconfigs/webhooks                      []                []              [create delete
deletecollection get list patch update watch get list watch]
buildconfigs                               []                []              [create delete
deletecollection get list patch update watch get list watch]
buildlogs                                  []                []              [create delete deletecollection
get list patch update watch get list watch]
deploymentconfigs/scale                    []                []              [create delete
deletecollection get list patch update watch get list watch]
deploymentconfigs                          []                []              [create delete
deletecollection get list patch update watch get list watch]
imagestreamimages                          []                []              [create delete
deletecollection get list patch update watch get list watch]
imagestreammappings                        []                []              [create delete
deletecollection get list patch update watch get list watch]
imagestreamtags                            []                []              [create delete
deletecollection get list patch update watch get list watch]
processedtemplates                         []                []              [create delete
deletecollection get list patch update watch get list watch]
routes                                     []                []              [create delete deletecollection
get list patch update watch get list watch]
templateconfigs                            []                []              [create delete
deletecollection get list patch update watch get list watch]
templateinstances                          []                []              [create delete
deletecollection get list patch update watch get list watch]
templates                                  []                []              [create delete
deletecollection get list patch update watch get list watch]
deploymentconfigs.apps.openshift.io/scale  []                []              [create delete
deletecollection get list patch update watch get list watch]
deploymentconfigs.apps.openshift.io        []                []              [create delete
deletecollection get list patch update watch get list watch]
buildconfigs.build.openshift.io/webhooks   []                []              [create delete
deletecollection get list patch update watch get list watch]
buildconfigs.build.openshift.io            []                []              [create delete
deletecollection get list patch update watch get list watch]
buildlogs.build.openshift.io               []                []              [create delete
deletecollection get list patch update watch get list watch]
imagestreamimages.image.openshift.io       []                []              [create delete
deletecollection get list patch update watch get list watch]
imagestreammappings.image.openshift.io     []                []              [create delete
deletecollection get list patch update watch get list watch]
imagestreamtags.image.openshift.io         []                []              [create delete
deletecollection get list patch update watch get list watch]
routes.route.openshift.io                  []                []              [create delete
deletecollection get list patch update watch get list watch]
processedtemplates.template.openshift.io   []                []              [create delete
deletecollection get list patch update watch get list watch]
templateconfigs.template.openshift.io      []                []              [create delete
```

```
deletecollection get list patch update watch get list watch]
  templateinstances.template.openshift.io                []              []              [create delete
deletecollection get list patch update watch get list watch]
  templates.template.openshift.io                        []              []              [create delete
deletecollection get list patch update watch get list watch]
  serviceaccounts                                        []              []              [create delete
deletecollection get list patch update watch impersonate create delete deletecollection patch
update get list watch]
  imagestreams/secrets                                   []              []              [create delete
deletecollection get list patch update watch]
  rolebindings                                           []              []              [create delete
deletecollection get list patch update watch]
  roles                                                  []              []              [create delete deletecollection
get list patch update watch]
  rolebindings.authorization.openshift.io                []              []              [create delete
deletecollection get list patch update watch]
  roles.authorization.openshift.io                       []              []              [create delete
deletecollection get list patch update watch]
  imagestreams.image.openshift.io/secrets                []              []              [create delete
deletecollection get list patch update watch]
  rolebindings.rbac.authorization.k8s.io                 []              []              [create delete
deletecollection get list patch update watch]
  roles.rbac.authorization.k8s.io                        []              []              [create delete
deletecollection get list patch update watch]
  networkpolicies.extensions                             []              []              [create delete
deletecollection patch update create delete deletecollection get list patch update watch get
list watch]
  networkpolicies.networking.k8s.io                      []              []              [create delete
deletecollection patch update create delete deletecollection get list patch update watch get
list watch]
  configmaps                                             []              []              [create delete
deletecollection patch update get list watch]
  endpoints                                              []              []              [create delete
deletecollection patch update get list watch]
  persistentvolumeclaims                                 []              []              [create delete
deletecollection patch update get list watch]
  pods                                                   []              []              [create delete deletecollection
patch update get list watch]
  replicationcontrollers/scale                           []              []              [create delete
deletecollection patch update get list watch]
  replicationcontrollers                                 []              []              [create delete
deletecollection patch update get list watch]
  services                                               []              []              [create delete deletecollection
patch update get list watch]
  daemonsets.apps                                        []              []              [create delete
deletecollection patch update get list watch]
  deployments.apps/scale                                 []              []              [create delete
deletecollection patch update get list watch]
  deployments.apps                                       []              []              [create delete
deletecollection patch update get list watch]
  replicasets.apps/scale                                 []              []              [create delete
deletecollection patch update get list watch]
  replicasets.apps                                       []              []              [create delete
deletecollection patch update get list watch]
  statefulsets.apps/scale                                []              []              [create delete
deletecollection patch update get list watch]
```

```
  statefulsets.apps                                  []                  []              [create delete
deletecollection patch update get list watch]
  horizontalpodautoscalers.autoscaling               []                  []              [create delete
deletecollection patch update get list watch]
  cronjobs.batch                                     []                  []              [create delete
deletecollection patch update get list watch]
  jobs.batch                                         []                  []              [create delete
deletecollection patch update get list watch]
  daemonsets.extensions                              []                  []              [create delete
deletecollection patch update get list watch]
  deployments.extensions/scale                       []                  []              [create delete
deletecollection patch update get list watch]
  deployments.extensions                             []                  []              [create delete
deletecollection patch update get list watch]
  ingresses.extensions                               []                  []              [create delete
deletecollection patch update get list watch]
  replicasets.extensions/scale                       []                  []              [create delete
deletecollection patch update get list watch]
  replicasets.extensions                             []                  []              [create delete
deletecollection patch update get list watch]
  replicationcontrollers.extensions/scale            []                  []              [create delete
deletecollection patch update get list watch]
  poddisruptionbudgets.policy                        []                  []              [create delete
deletecollection patch update get list watch]
  deployments.apps/rollback                          []                  []              [create delete
deletecollection patch update]
  deployments.extensions/rollback                    []                  []              [create delete
deletecollection patch update]
  catalogsources.operators.coreos.com                []                  []              [create update
patch delete get list watch]
  clusterserviceversions.operators.coreos.com        []                  []              [create update
patch delete get list watch]
  installplans.operators.coreos.com                  []                  []              [create update
patch delete get list watch]
  packagemanifests.operators.coreos.com              []                  []              [create update
patch delete get list watch]
  subscriptions.operators.coreos.com                 []                  []              [create update
patch delete get list watch]
  buildconfigs/instantiate                           []                  []              [create]
  buildconfigs/instantiatebinary                     []                  []              [create]
  builds/clone                                       []                  []              [create]
  deploymentconfigrollbacks                          []                  []              [create]
  deploymentconfigs/instantiate                      []                  []              [create]
  deploymentconfigs/rollback                         []                  []              [create]
  imagestreamimports                                 []                  []              [create]
  localresourceaccessreviews                         []                  []              [create]
  localsubjectaccessreviews                          []                  []              [create]
  podsecuritypolicyreviews                           []                  []              [create]
  podsecuritypolicyselfsubjectreviews                []                  []              [create]
  podsecuritypolicysubjectreviews                    []                  []              [create]
  resourceaccessreviews                              []                  []              [create]
  routes/custom-host                                 []                  []              [create]
  subjectaccessreviews                               []                  []              [create]
  subjectrulesreviews                                []                  []              [create]
  deploymentconfigrollbacks.apps.openshift.io        []                  []              [create]
  deploymentconfigs.apps.openshift.io/instantiate    []                  []              [create]
```

```
  deploymentconfigs.apps.openshift.io/rollback                []                []              [create]
  localsubjectaccessreviews.authorization.k8s.io              []                []              [create]
  localresourceaccessreviews.authorization.openshift.io       []                []              [create]
  localsubjectaccessreviews.authorization.openshift.io        []                []              [create]
  resourceaccessreviews.authorization.openshift.io            []                []              [create]
  subjectaccessreviews.authorization.openshift.io             []                []              [create]
  subjectrulesreviews.authorization.openshift.io              []                []              [create]
  buildconfigs.build.openshift.io/instantiate                 []                []              [create]
  buildconfigs.build.openshift.io/instantiatebinary           []                []              [create]
  builds.build.openshift.io/clone                             []                []              [create]
  imagestreamimports.image.openshift.io                       []                []              [create]
  routes.route.openshift.io/custom-host                       []                []              [create]
  podsecuritypolicyreviews.security.openshift.io              []                []              [create]
  podsecuritypolicyselfsubjectreviews.security.openshift.io   []                []              [create]
  podsecuritypolicysubjectreviews.security.openshift.io       []                []              [create]
  jenkins.build.openshift.io                                  []                []              [edit view view admin
edit view]
  builds                                                      []                []              [get create delete
deletecollection get list patch update watch get list watch]
  builds.build.openshift.io                                   []                []              [get create delete
deletecollection get list patch update watch get list watch]
  projects                                                    []                []              [get delete get delete get patch
update]
  projects.project.openshift.io                               []                []              [get delete get delete
get patch update]
  namespaces                                                  []                []              [get get list watch]
  pods/attach                                                 []                []              [get list watch create delete
deletecollection patch update]
  pods/exec                                                   []                []              [get list watch create delete
deletecollection patch update]
  pods/portforward                                            []                []              [get list watch create
delete deletecollection patch update]
  pods/proxy                                                  []                []              [get list watch create delete
deletecollection patch update]
  services/proxy                                              []                []              [get list watch create delete
deletecollection patch update]
  routes/status                                               []                []              [get list watch update]
  routes.route.openshift.io/status                            []                []              [get list watch update]
  appliedclusterresourcequotas                                []                []              [get list watch]
  bindings                                                    []                []              [get list watch]
  builds/log                                                  []                []              [get list watch]
  deploymentconfigs/log                                       []                []              [get list watch]
  deploymentconfigs/status                                    []                []              [get list watch]
  events                                                      []                []              [get list watch]
  imagestreams/status                                         []                []              [get list watch]
  limitranges                                                 []                []              [get list watch]
  namespaces/status                                           []                []              [get list watch]
  pods/log                                                    []                []              [get list watch]
  pods/status                                                 []                []              [get list watch]
  replicationcontrollers/status                               []                []              [get list watch]
  resourcequotas/status                                       []                []              [get list watch]
  resourcequotas                                              []                []              [get list watch]
  resourcequotausages                                         []                []              [get list watch]
  rolebindingrestrictions                                     []                []              [get list watch]
  deploymentconfigs.apps.openshift.io/log                     []                []              [get list watch]
  deploymentconfigs.apps.openshift.io/status                  []                []              [get list watch]
```

```
  controllerrevisions.apps                                   []                 []              [get list watch]
  rolebindingrestrictions.authorization.openshift.io         []                 []              [get list watch]
  builds.build.openshift.io/log                              []                 []              [get list watch]
  imagestreams.image.openshift.io/status                     []                 []              [get list watch]
  appliedclusterresourcequotas.quota.openshift.io            []                 []              [get list watch]
  imagestreams/layers                                        []                 []              [get update get]
  imagestreams.image.openshift.io/layers                     []                 []              [get update get]
  builds/details                                             []                 []              [update]
  builds.build.openshift.io/details                          []                 []              [update]


Name:         basic-user
Labels:       <none>
Annotations:  openshift.io/description: A user that can get basic information about projects.
              rbac.authorization.kubernetes.io/autoupdate: true
PolicyRule:
 Resources                                          Non-Resource URLs  Resource Names  Verbs
  ---------                                         -----------------  --------------  -----
  selfsubjectrulesreviews                           []                 []              [create]
  selfsubjectaccessreviews.authorization.k8s.io     []                 []              [create]
  selfsubjectrulesreviews.authorization.openshift.io  []               []              [create]
  clusterroles.rbac.authorization.k8s.io            []                 []              [get list watch]
  clusterroles                                      []                 []              [get list]
  clusterroles.authorization.openshift.io           []                 []              [get list]
  storageclasses.storage.k8s.io                     []                 []              [get list]
  users                                             []                 [~]             [get]
  users.user.openshift.io                           []                 [~]             [get]
  projects                                          []                 []              [list watch]
  projects.project.openshift.io                     []                 []              [list watch]
  projectrequests                                   []                 []              [list]
  projectrequests.project.openshift.io              []                 []              [list]

Name:         cluster-admin
Labels:       kubernetes.io/bootstrapping=rbac-defaults
Annotations:  rbac.authorization.kubernetes.io/autoupdate: true
PolicyRule:
Resources  Non-Resource URLs  Resource Names  Verbs
---------  -----------------  --------------  -----
*.*        []                 []              [*]
           [*]                []              [*]

...
```

2. To view the current set of cluster role bindings, which shows the users and groups that are bound to various roles:

```
$ oc describe clusterrolebinding.rbac
```

Example output

```
Name:         alertmanager-main
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
```

```
  Name:  alertmanager-main
Subjects:
  Kind           Name              Namespace
  ----           ----              ---------
  ServiceAccount  alertmanager-main  openshift-monitoring


Name:         basic-users
Labels:       <none>
Annotations:  rbac.authorization.kubernetes.io/autoupdate: true
Role:
  Kind:  ClusterRole
  Name:  basic-user
Subjects:
  Kind   Name                Namespace
  ----   ----                ---------
  Group  system:authenticated


Name:         cloud-credential-operator-rolebinding
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  cloud-credential-operator-role
Subjects:
  Kind           Name     Namespace
  ----           ----     ---------
  ServiceAccount  default  openshift-cloud-credential-operator


Name:         cluster-admin
Labels:       kubernetes.io/bootstrapping=rbac-defaults
Annotations:  rbac.authorization.kubernetes.io/autoupdate: true
Role:
  Kind:  ClusterRole
  Name:  cluster-admin
Subjects:
  Kind   Name           Namespace
  ----   ----           ---------
  Group  system:masters


Name:         cluster-admins
Labels:       <none>
Annotations:  rbac.authorization.kubernetes.io/autoupdate: true
Role:
  Kind:  ClusterRole
  Name:  cluster-admin
Subjects:
  Kind   Name                 Namespace
  ----   ----                 ---------
  Group  system:cluster-admins
  User   system:admin
```

```
Name:         cluster-api-manager-rolebinding
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  cluster-api-manager-role
Subjects:
  Kind            Name     Namespace
  ----            ----     ---------
  ServiceAccount  default  openshift-machine-api

...
```

10.5. VIEWING LOCAL ROLES AND BINDINGS

You can view local role bindings by using the **oc** CLI to identify the users, groups, and service accounts that have roles within the current project or another project.

You can use the **oc** CLI to view local roles and bindings by using the **oc describe** command.

Prerequisites

- Install the **oc** CLI.
- Obtain permission to view the local roles and bindings:
  - Users with the **cluster-admin** default cluster role bound cluster-wide can perform any action on any resource, including viewing local roles and bindings.
  - Users with the **admin** default cluster role bound locally can view and manage roles and bindings in that project.

Procedure

1. To view the current set of local role bindings, which show the users and groups that are bound to various roles for the current project:

```
$ oc describe rolebinding.rbac
```

2. To view the local role bindings for a different project, add the **-n** flag to the command:

```
$ oc describe rolebinding.rbac -n joe-project
```

Example output

```
Name:         admin
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  admin
Subjects:
  Kind  Name         Namespace
  ----  ----         ---------
```

```
  User  kube:admin


Name:         system:deployers
Labels:       <none>
Annotations:  openshift.io/description:
              Allows deploymentconfigs in this namespace to rollout pods in
              this namespace.  It is auto-managed by a controller; remove
              subjects to disa...
Role:
  Kind:  ClusterRole
  Name:  system:deployer
Subjects:
  Kind           Name      Namespace
  ----           ----      ---------
  ServiceAccount  deployer  joe-project


Name:         system:image-builders
Labels:       <none>
Annotations:  openshift.io/description:
              Allows builds in this namespace to push images to this
              namespace.  It is auto-managed by a controller; remove subjects
              to disable.
Role:
  Kind:  ClusterRole
  Name:  system:image-builder
Subjects:
  Kind           Name     Namespace
  ----           ----     ---------
  ServiceAccount  builder  joe-project


Name:         system:image-pullers
Labels:       <none>
Annotations:  openshift.io/description:
              Allows all pods in this namespace to pull images from this
              namespace.  It is auto-managed by a controller; remove subjects
              to disable.
Role:
  Kind:  ClusterRole
  Name:  system:image-puller
Subjects:
  Kind   Name                         Namespace
  ----   ----                         ---------
  Group  system:serviceaccounts:joe-project
```

10.6. ADDING ROLES TO USERS

To grant a user access within a project, you can bind an appropriate role to the user and verify the resulting role binding.

You can use the **oc adm** administrator CLI to manage the roles and bindings.

Binding, or adding, a role to users or groups gives the user or group the access that is granted by the role. You can add and remove roles to and from users and groups using **oc adm policy** commands.

You can bind any of the default cluster roles to local users or groups in your project.

Procedure

1. Add a role to a user in a specific project:

```
$ oc adm policy add-role-to-user <role> <user> -n <project>
```

For example, you can add the **admin** role to the **alice** user in **joe** project by running:

```
$ oc adm policy add-role-to-user admin alice -n joe
```

TIP

You can alternatively apply the following YAML to add the role to the user:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: admin-0
  namespace: joe
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: alice
```

2. View the local role bindings and verify the addition in the output:

```
$ oc describe rolebinding.rbac -n <project>
```

For example, to view the local role bindings for the **joe** project:

```
$ oc describe rolebinding.rbac -n joe
```

Example output

```
Name:         admin
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  admin
Subjects:
  Kind  Name        Namespace
  ----  ----        ---------
```

```
  User  kube:admin


Name:         admin-0
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  admin
Subjects:
  Kind  Name   Namespace
  ----  ----   ---------
  User  alice


Name:         system:deployers
Labels:       <none>
Annotations:  openshift.io/description:
              Allows deploymentconfigs in this namespace to rollout pods in
              this namespace.  It is auto-managed by a controller; remove
              subjects to disa...
Role:
  Kind:  ClusterRole
  Name:  system:deployer
Subjects:
  Kind           Name      Namespace
  ----           ----      ---------
  ServiceAccount  deployer  joe


Name:         system:image-builders
Labels:       <none>
Annotations:  openshift.io/description:
              Allows builds in this namespace to push images to this
              namespace.  It is auto-managed by a controller; remove subjects
              to disable.
Role:
  Kind:  ClusterRole
  Name:  system:image-builder
Subjects:
  Kind           Name     Namespace
  ----           ----     ---------
  ServiceAccount  builder  joe


Name:         system:image-pullers
Labels:       <none>
Annotations:  openshift.io/description:
              Allows all pods in this namespace to pull images from this
              namespace.  It is auto-managed by a controller; remove subjects
              to disable.
Role:
  Kind:  ClusterRole
  Name:  system:image-puller
Subjects:
```

```
Kind   Name                         Namespace
----   ----                         ---------
Group  system:serviceaccounts:joe
```

The **alice** user has been added to the **admins RoleBinding**.

10.7. CREATING A LOCAL ROLE

You can create a local role and bind it to a user to define custom permissions within a project.

Procedure

1. To create a local role for a project, run the following command:

```
$ oc create role <name> --verb=<verb> --resource=<resource> -n <project>
```

In this command, specify:

- **<name>**, the local role's name
- **<verb>**, a comma-separated list of the verbs to apply to the role
- **<resource>**, the resources that the role applies to
- **<project>**, the project name

For example, to create a local role that allows a user to view pods in the **blue** project, run the following command:

```
$ oc create role podview --verb=get --resource=pod -n blue
```

2. To bind the new role to a user, run the following command:

```
$ oc adm policy add-role-to-user podview user2 --role-namespace=blue -n blue
```

10.8. CREATING A CLUSTER ROLE

To define custom cluster-wide permissions, you can create a cluster role that specifies the verbs and resources users can access.

Procedure

- To create a cluster role, run the following command:

```
$ oc create clusterrole <name> --verb=<verb> --resource=<resource>
```

In this command, specify:

  - **<name>**, the local role's name
  - **<verb>**, a comma-separated list of the verbs to apply to the role
  - **<resource>**, the resources that the role applies to

For example, to create a cluster role that allows a user to view pods, run the following command:

```
$ oc create clusterrole podviewonly --verb=get --resource=pod
```

10.9. LOCAL ROLE BINDING COMMANDS

You can use local role binding commands to review, grant, or remove user and group permissions within the current or a specified project.

When you manage a user or group's associated roles for local role bindings using the following operations, a project may be specified with the **-n** flag. If it is not specified, then the current project is used.

You can use the following commands for local RBAC management.

Table 10.1. Local role binding operations

Command	Description
$ oc adm policy who-can *<verb> <resource>*	Indicates which users can perform an action on a resource.
$ oc adm policy add-role-to-user *<role> <username>*	Binds a specified role to specified users in the current project.
$ oc adm policy remove-role-from-user *<role> <username>*	Removes a given role from specified users in the current project.
$ oc adm policy remove-user *<username>*	Removes specified users and all of their roles in the current project.
$ oc adm policy add-role-to-group *<role> <groupname>*	Binds a given role to specified groups in the current project.
$ oc adm policy remove-role-from-group *<role> <groupname>*	Removes a given role from specified groups in the current project.
$ oc adm policy remove-group *<groupname>*	Removes specified groups and all of their roles in the current project.

10.10. CLUSTER ROLE BINDING COMMANDS

You can use cluster role binding commands to grant or remove roles for users and groups across all projects in the cluster.

You can also manage cluster role bindings using the following operations. The **-n** flag is not used for these operations because cluster role bindings use non-namespaced resources.

Table 10.2. Cluster role binding operations

Command	Description
$ oc adm policy add-cluster-role-to-user *<role> <username>*	Binds a given role to specified users for all projects in the cluster.
$ oc adm policy remove-cluster-role-from-user *<role> <username>*	Removes a given role from specified users for all projects in the cluster.
$ oc adm policy add-cluster-role-to-group *<role> <groupname>*	Binds a given role to specified groups for all projects in the cluster.
$ oc adm policy remove-cluster-role-from-group *<role> <groupname>*	Removes a given role from specified groups for all projects in the cluster.

10.11. CREATING A CLUSTER ADMIN

To grant a user full administrative access to the cluster, you can bind the **cluster-admin** cluster role to that user.

The **cluster-admin** role is required to perform administrator level tasks on the OpenShift Container Platform cluster, such as modifying cluster resources.

Prerequisites

- You must have created a user to define as the cluster admin.

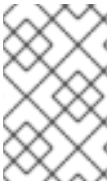
Procedure

- Define the user as a cluster admin:

```
$ oc adm policy add-cluster-role-to-user cluster-admin <user>
```

10.12. CLUSTER ROLE BINDINGS FOR UNAUTHENTICATED GROUPS

Unauthenticated groups do not have default access to cluster roles. As a cluster administrator, you can grant limited unauthenticated access when required, while ensuring that the change complies with organizational security standards.



NOTE

Before OpenShift Container Platform 4.17, unauthenticated groups were allowed access to some cluster roles. Clusters updated from versions before OpenShift Container Platform 4.17 retain this access for unauthenticated groups.

For security reasons OpenShift Container Platform 4.22 does not allow unauthenticated groups to have default access to cluster roles.

There are use cases where it might be necessary to add **system:unauthenticated** to a cluster role.

Cluster administrators can add unauthenticated users to the following cluster roles:

- **system:scope-impersonation**
- **system:webhook**
- **system:oauth-token-deleter**
- **self-access-reviewer**

IMPORTANT

Always verify compliance with your organization's security standards when modifying unauthenticated access.

CHAPTER 11. REMOVING THE KUBEADMIN USER

After installation, OpenShift Container Platform creates a cluster administrator user called **kubeadmin** with the **cluster-admin** role. To improve cluster security, you can remove this user after configuring an identity provider and creating a new **cluster-admin** user.

11.1. THE KUBEADMIN USER

OpenShift Container Platform creates a cluster administrator, **kubeadmin**, after the installation process completes. This user has the **cluster-admin** role automatically applied and is treated as the root user for the cluster.

The password is dynamically generated and unique to your OpenShift Container Platform environment. After the installation completes, the password is provided in the installation program's output. For example:

```
INFO Install complete!
INFO Run 'export KUBECONFIG=<your working directory>/auth/kubeconfig' to manage the cluster with 'oc', the OpenShift CLI.
INFO The cluster is ready when 'oc login -u kubeadmin -p <provided>' succeeds (wait a few minutes).
INFO Access the OpenShift web-console here: https://console-openshift-console.apps.demo1.openshift4-beta-abcorp.com
INFO Login to the console with user: kubeadmin, password: <provided>
```

11.2. REMOVING THE KUBEADMIN USER

After you define an identity provider and create a new **cluster-admin** user, you can remove the **kubeadmin** to improve cluster security.

WARNING

If you follow this procedure before another user is a **cluster-admin**, then OpenShift Container Platform must be reinstalled. It is not possible to undo this command.

Prerequisites

- You must have configured at least one identity provider.
- You must have added the **cluster-admin** role to a user.
- You must be logged in as an administrator.

Procedure

- Remove the **kubeadmin** secrets:

```
$ oc delete secrets kubeadmin -n kube-system
```

CHAPTER 12. UNDERSTANDING AND CREATING SERVICE ACCOUNTS

To control API access without sharing regular user credentials, you can use service accounts.

12.1. SERVICE ACCOUNTS OVERVIEW

You can use OpenShift Container Platform service accounts to allow a OpenShift Container Platform component to directly access the API.

Service accounts are API objects that exist within each project that provide a flexible way to control API access without sharing a regular user's credentials.

When you use the OpenShift Container Platform CLI or web console, your API token authenticates you to the API. You can associate a component with a service account so that they can access the API without using a regular user's credentials.

For example, service accounts can allow:

- Replication controllers to make API calls to create or delete pods
- Applications inside containers to make API calls for discovery purposes
- External applications to make API calls for monitoring or integration purposes

Each service account's user name is derived from its project and name:

```
system:serviceaccount:<project>:<name>
```

Every service account is also a member of two groups:

Group	Description
system:serviceaccounts	Includes all service accounts in the system.
system:serviceaccounts:<project>	Includes all service accounts in the specified project.

12.1.1. Automatically generated image pull secrets

OpenShift Container Platform automatically creates image pull secrets for each service account to integrate the internal image registry with user authentication.

NOTE

Prior to OpenShift Container Platform 4.16, a long-lived service account API token secret was also generated for each service account that was created. Starting with OpenShift Container Platform 4.16, this service account API token secret is no longer created.

After upgrading to 4.22, any existing long-lived service account API token secrets are not deleted and will continue to function. For information about detecting long-lived API tokens that are in use in your cluster or deleting them if they are not needed, see "Long-lived service account API tokens in OpenShift Container Platform (Red Hat Knowledgebase)".

This image pull secret is necessary to integrate the OpenShift image registry into the cluster's user authentication and authorization system.

However, if you do not enable the **ImageRegistry** capability or if you disable the integrated OpenShift image registry in the Cluster Image Registry Operator's configuration, an image pull secret is not generated for each service account.

When the integrated OpenShift image registry is disabled on a cluster that previously had it enabled, the previously generated image pull secrets are deleted automatically.

12.2. CREATING SERVICE ACCOUNTS

You can create a service account in a project and grant it permissions by binding it to a role.

Procedure

1. Optional: To view the service accounts in the current project:

```
$ oc get sa
```

Example output

```
NAME      SECRETS   AGE
builder   1         2d
default   1         2d
deployer  1         2d
```

2. To create a new service account in the current project:

```
$ oc create sa <service_account_name>
```

To create a service account in a different project, specify **-n <project_name>**.

Example output

```
serviceaccount "robot" created
```

TIP

You can alternatively apply the following YAML to create the service account:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: <service_account_name>
  namespace: <current_project>
```

3. Optional: View the secrets for the service account:

```
$ oc describe sa robot
```

Example output

```
Name:                robot
Namespace:           project1
Labels:              <none>
Annotations:         openshift.io/internal-registry-pull-secret-ref: robot-dockercfg-qzbhb
Image pull secrets:  robot-dockercfg-qzbhb
Mountable secrets:   robot-dockercfg-qzbhb
Tokens:              <none>
Events:              <none>
```

12.3. GRANTING ROLES TO SERVICE ACCOUNTS

You can grant roles to service accounts in the same way that you grant roles to a regular user account.

Procedure

1. You can modify the service accounts for the current project. For example, to add the **view** role to the **robot** service account in the **top-secret** project:

```
$ oc policy add-role-to-user view system:serviceaccount:top-secret:robot
```

TIP

You can alternatively apply the following YAML to add the role:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: view
  namespace: top-secret
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view
subjects:
- kind: ServiceAccount
  name: robot
  namespace: top-secret
```

2. You can also grant access to a specific service account in a project. For example, from the project to which the service account belongs, use the **-z** flag and specify the **<service_account_name>**

```
$ oc policy add-role-to-user <role_name> -z <service_account_name>
```

IMPORTANT

If you want to grant access to a specific service account in a project, use the **-z** flag. Using this flag helps prevent typos and ensures that access is granted to only the specified service account.

TIP

You can alternatively apply the following YAML to add the role:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: <rolebinding_name>
  namespace: <current_project_name>
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: <role_name>
subjects:
- kind: ServiceAccount
  name: <service_account_name>
  namespace: <current_project_name>
```

3. To modify a different namespace, you can use the **-n** option to indicate the project namespace it applies to, as shown in the following examples.

   - For example, to allow all service accounts in all projects to view resources in the **my-project** project:

```
$ oc policy add-role-to-group view system:serviceaccounts -n my-project
```

TIP

You can alternatively apply the following YAML to add the role:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: view
  namespace: my-project
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: system:serviceaccounts
```

- To allow all service accounts in the **managers** project to edit resources in the **my-project** project:

```
$ oc policy add-role-to-group edit system:serviceaccounts:managers -n my-project
```

TIP

You can alternatively apply the following YAML to add the role:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: edit
  namespace: my-project
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: edit
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: system:serviceaccounts:managers
```

CHAPTER 13. USING SERVICE ACCOUNTS IN APPLICATIONS

13.1. SERVICE ACCOUNTS OVERVIEW

You can use OpenShift Container Platform service accounts to allow a OpenShift Container Platform component to directly access the API.

Service accounts are API objects that exist within each project that provide a flexible way to control API access without sharing a regular user's credentials.

When you use the OpenShift Container Platform CLI or web console, your API token authenticates you to the API. You can associate a component with a service account so that they can access the API without using a regular user's credentials.

For example, service accounts can allow:

- Replication controllers to make API calls to create or delete pods
- Applications inside containers to make API calls for discovery purposes
- External applications to make API calls for monitoring or integration purposes

Each service account's user name is derived from its project and name:

```
system:serviceaccount:<project>:<name>
```

Every service account is also a member of two groups:

Group	Description
system:serviceaccounts	Includes all service accounts in the system.
system:serviceaccounts:<project>	Includes all service accounts in the specified project.

13.2. DEFAULT SERVICE ACCOUNTS

Your OpenShift Container Platform cluster contains default service accounts for cluster management and generates more service accounts for each project.

13.2.1. Default cluster service accounts

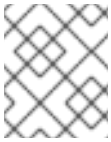
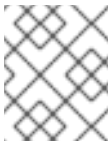
Several infrastructure controllers run using service account credentials. The following service accounts are created in the OpenShift Container Platform infrastructure project (**openshift-infra**) at server start, and given the following roles cluster-wide:

Service account	Description
replication-controller	Assigned the **system:replication-controller** role

Service account	Description
deployment-controller	Assigned the **system:deployment-controller** role
build-controller	Assigned the **system:build-controller** role. Additionally, the **build-controller** service account is included in the privileged security context constraint to create privileged build pods.

13.2.2. Default project service accounts and roles

Three service accounts are automatically created in each project:

Service account	Usage
builder	Used by build pods. It is given the **system:image-builder** role, which allows pushing images to any imagestream in the project using the internal Docker registry.  NOTE The **builder** service account is not created if the **Build** cluster capability is not enabled.
deployer	Used by deployment pods and given the **system:deployer** role, which allows viewing and modifying replication controllers and pods in the project.  NOTE The **deployer** service account is not created if the **DeploymentConfig** cluster capability is not enabled.

Service account	Usage
default	Used to run all other pods unless they specify a different service account.  IMPORTANT Access rights and security privileges tied to the **default** service account apply to every pod in the project that does not specify a different service account. To implement the principle of least privilege and improve auditability, create dedicated service accounts for your workloads instead of using the **default** service account. While most OpenShift Container Platform platform components and Operators use dedicated service accounts, the following dynamic tools continue to use the **default** service account to ensure operational efficiency: ● **oc debug**: Uses the **default** service account to avoid the performance overhead of creating and removing unique service accounts for short-lived troubleshooting sessions. ● **oc adm must-gather**: Uses the **default** service account to collect diagnostic data across the cluster without requiring extensive manual RBAC modifications.

All service accounts in a project are given the **system:image-puller** role, which allows pulling images from any image stream in the project using the internal container image registry.

13.2.3. Automatically generated image pull secrets

OpenShift Container Platform automatically creates image pull secrets for each service account to integrate the internal image registry with user authentication.

NOTE

Prior to OpenShift Container Platform 4.16, a long-lived service account API token secret was also generated for each service account that was created. Starting with OpenShift Container Platform 4.16, this service account API token secret is no longer created.

After upgrading to 4.22, any existing long-lived service account API token secrets are not deleted and will continue to function. For information about detecting long-lived API tokens that are in use in your cluster or deleting them if they are not needed, see "Long-lived service account API tokens in OpenShift Container Platform (Red Hat Knowledgebase)".

This image pull secret is necessary to integrate the OpenShift image registry into the cluster's user authentication and authorization system.

However, if you do not enable the **ImageRegistry** capability or if you disable the integrated OpenShift image registry in the Cluster Image Registry Operator's configuration, an image pull secret is not generated for each service account.

When the integrated OpenShift image registry is disabled on a cluster that previously had it enabled, the previously generated image pull secrets are deleted automatically.

13.3. CREATING SERVICE ACCOUNTS

You can create a service account in a project and grant it permissions by binding it to a role.

Procedure

1. Optional: To view the service accounts in the current project:

```
$ oc get sa
```

Example output

```
NAME      SECRETS   AGE
builder   1         2d
default   1         2d
deployer  1         2d
```

2. To create a new service account in the current project:

```
$ oc create sa <service_account_name>
```

To create a service account in a different project, specify **-n <project_name>**.

Example output

```
serviceaccount "robot" created
```

TIP

You can alternatively apply the following YAML to create the service account:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: <service_account_name>
  namespace: <current_project>
```

3. Optional: View the secrets for the service account:

```
$ oc describe sa robot
```

Example output

```
Name:                robot
Namespace:           project1
Labels:              <none>
Annotations:         openshift.io/internal-registry-pull-secret-ref: robot-dockercfg-qzbhb
Image pull secrets:  robot-dockercfg-qzbhb
Mountable secrets:   robot-dockercfg-qzbhb
Tokens:              <none>
Events:              <none>
```

CHAPTER 14. USING A SERVICE ACCOUNT AS AN OAUTH CLIENT

To authenticate users when restricting access to a specific namespace, you can configure a service account as a constrained OAuth client by using static or dynamic redirect URI annotations.

14.1. ABOUT SERVICE ACCOUNTS AS OAUTH CLIENTS

You can configure a service account to function as a constrained OAuth client that can request a limited subset of scopes.

Service accounts can request only a subset of scopes that allow access to the following basic user information and role-based power inside of the service account's own namespace:

- **user:info**
- **user:check-access**
- **role:<any_role>:<service_account_namespace>**
- **role:<any_role>:<service_account_namespace>:!**

When using a service account as an OAuth client:

- **client_id** is **system:serviceaccount:<service_account_namespace>:<service_account_name>**.
- **client_secret** can be any of the API tokens for that service account. For example:

```
$ oc sa get-token <service_account_name>
```

- To get **WWW-Authenticate** challenges, set an **serviceaccounts.openshift.io/oauth-want-challenges** annotation on the service account to **true**.
- **redirect_uri** must match an annotation on the service account.

Annotation keys in service accounts must have the prefix **serviceaccounts.openshift.io/oauth-redirecturi.** or **serviceaccounts.openshift.io/oauth-redirectreference.** such as:

```
serviceaccounts.openshift.io/oauth-redirecturi.<name>
```

In its simplest form, the annotation can be used to directly specify valid redirect URIs. For example:

```
"serviceaccounts.openshift.io/oauth-redirecturi.first":  "https://example.com"
"serviceaccounts.openshift.io/oauth-redirecturi.second": "https://other.com"
```

The **first** and **second** postfixes in the above example are used to separate the two valid redirect URIs.

In more complex configurations, static redirect URIs may not be enough. For example, perhaps you want all Ingresses for a route to be considered valid. This is where dynamic redirect URIs via the **serviceaccounts.openshift.io/oauth-redirectreference.** prefix come into play.

For example:

```
"serviceaccounts.openshift.io/oauth-redirectreference.first": "
{\"kind\":\"OAuthRedirectReference\",\"apiVersion\":\"v1\",\"reference\":
{\"kind\":\"Route\",\"name\":\"jenkins\"}}"
```

Since the value for this annotation contains serialized JSON data, it is easier to see in an expanded format:

```
{
  "kind": "OAuthRedirectReference",
  "apiVersion": "v1",
  "reference": {
    "kind": "Route",
    "name": "jenkins"
  }
}
```

Now you can see that an **OAuthRedirectReference** allows us to reference the route named **jenkins**. Thus, all Ingresses for that route will now be considered valid. The full specification for an **OAuthRedirectReference** is:

```
{
  "kind": "OAuthRedirectReference",
  "apiVersion": "v1",
  "reference": {
    "kind": ...,
    "name": ...,
    "group": ...
  }
}
```

where:

reference.kind

Specifies the type of the object being referenced. Currently, only **route** is supported.

reference.name

Specifies the name of the object. The object must be in the same namespace as the service account.

reference.group

Specifies the group of the object. Leave this blank, as the group for a route is the empty string.

Both annotation prefixes can be combined to override the data provided by the reference object. For example:

```
"serviceaccounts.openshift.io/oauth-redirecturi.first":  "custompath"
"serviceaccounts.openshift.io/oauth-redirectreference.first": "
{\"kind\":\"OAuthRedirectReference\",\"apiVersion\":\"v1\",\"reference\":
{\"kind\":\"Route\",\"name\":\"jenkins\"}}"
```

The **first** postfix is used to tie the annotations together. Assuming that the **jenkins** route had an Ingress of **https://example.com**, now **https://example.com/custompath** is considered valid, but **https://example.com** is not. The format for partially supplying override data is as follows:

Type	Syntax
Scheme	"\https://"
Hostname	"//website.com"
Port	"//:8000"
Path	"examplepath"

NOTE

Specifying a hostname override will replace the hostname data from the referenced object, which is not likely to be desired behavior.

Any combination of the above syntax can be combined using the following format:

<scheme:>//<hostname><:port>/<path>

The same object can be referenced more than once for more flexibility:

```
"serviceaccounts.openshift.io/oauth-redirecturi.first":  "custompath"
"serviceaccounts.openshift.io/oauth-redirectreference.first": "
{\"kind\":\"OAuthRedirectReference\",\"apiVersion\":\"v1\",\"reference\":
{\"kind\":\"Route\",\"name\":\"jenkins\"}}"
"serviceaccounts.openshift.io/oauth-redirecturi.second":  "//:8000"
"serviceaccounts.openshift.io/oauth-redirectreference.second": "
{\"kind\":\"OAuthRedirectReference\",\"apiVersion\":\"v1\",\"reference\":
{\"kind\":\"Route\",\"name\":\"jenkins\"}}"
```

Assuming that the route named **jenkins** has an Ingress of **https://example.com**, then both **https://example.com:8000** and **https://example.com/custompath** are considered valid.

Static and dynamic annotations can be used at the same time to achieve the desired behavior:

```
"serviceaccounts.openshift.io/oauth-redirectreference.first": "
{\"kind\":\"OAuthRedirectReference\",\"apiVersion\":\"v1\",\"reference\":
{\"kind\":\"Route\",\"name\":\"jenkins\"}}"
"serviceaccounts.openshift.io/oauth-redirecturi.second": "https://other.com"
```

CHAPTER 15. SCOPING TOKENS

You can create scoped tokens to delegate specific permissions to users or service accounts, and configure cluster role bindings for unauthenticated users when required.

15.1. ABOUT SCOPING TOKENS

You can create scoped tokens to delegate some of your permissions to another user or service account. For example, a project administrator might want to delegate the power to create pods.

A scoped token is a token that identifies as a given user but is limited to certain actions by its scope. Only a user with the **cluster-admin** role can create scoped tokens.

Scopes are evaluated by converting the set of scopes for a token into a set of **PolicyRules**. Then, the request is matched against those rules. The request attributes must match at least one of the scope rules to be passed to the "normal" authorizer for further authorization checks.

15.1.1. User scopes

User scopes are focused on getting information about a given user. They are intent-based, so the rules are automatically created for you:

- **user:full** - Allows full read/write access to the API with all of the user's permissions.
- **user:info** - Allows read-only access to information about the user, such as name and groups.
- **user:check-access** - Allows access to **self-localsubjectaccessreviews** and **self-subjectaccessreviews**. These are the variables where you pass an empty user and groups in your request object.
- **user:list-projects** - Allows read-only access to list the projects the user has access to.

15.1.2. Role scope

The role scope allows you to have the same level of access as a given role filtered by namespace.

- **role:<cluster-role name>:<namespace or * for all>** - Limits the scope to the rules specified by the cluster-role, but only in the specified namespace .

NOTE

Caveat: This prevents escalating access. Even if the role allows access to resources like secrets, rolebindings, and roles, this scope will deny access to those resources. This helps prevent unexpected escalations. Many people do not think of a role like **edit** as being an escalating role, but with access to a secret it is.

- **role:<cluster-role name>:<namespace or * for all>:!** - This is similar to the example above, except that including the bang causes this scope to allow escalating access.

15.2. ADDING UNAUTHENTICATED GROUPS TO CLUSTER ROLES

Grant unauthenticated users access to specific cluster roles to enable features that require cluster access without authentication, such as external webhooks or automated token management.

You can add unauthenticated users to the following cluster roles:

- **system:scope-impersonation**
- **system:webhook**
- **system:oauth-token-deleter**
- **self-access-reviewer**

IMPORTANT

Always verify compliance with your organization's security standards when modifying unauthenticated access.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Create a YAML file named **add-<cluster_role>-unauth.yaml** and add the following content:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 annotations:
   rbac.authorization.kubernetes.io/autoupdate: "true"
 name: <cluster_role>access-unauthenticated
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: <cluster_role>
subjects:
 - apiGroup: rbac.authorization.k8s.io
   kind: Group
   name: system:unauthenticated
```

2. Apply the configuration by running the following command:

```
$ oc apply -f add-<cluster_role>.yaml
```

CHAPTER 16. USING BOUND SERVICE ACCOUNT TOKENS

You can use bound service account tokens, which improve the ability to integrate with cloud provider identity access management (IAM) services such as OpenShift Container Platform on AWS IAM or Google Cloud IAM.

16.1. ABOUT BOUND SERVICE ACCOUNT TOKENS

You can use bound service account tokens to limit the scope of permissions for a given service account token.

Bound service account tokens are audience-bound and time-bound. This facilitates the authentication of a service account to an IAM role and the generation of temporary credentials mounted to a pod. You can request bound service account tokens by using volume projection and the TokenRequest API.

16.2. CONFIGURING BOUND SERVICE ACCOUNT TOKENS USING VOLUME PROJECTION

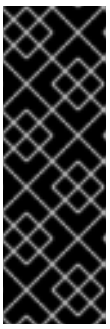
You can configure pods to request bound service account tokens by using volume projection.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have created a service account. This procedure assumes that the service account is named **build-robot**.

Procedure

1. Optional: Set the service account issuer.
   This step is typically not required if the bound tokens are used only within the cluster.



IMPORTANT

If you change the service account issuer to a custom one, the previous service account issuer is still trusted for the next 24 hours.

You can force all holders to request a new bound token either by manually restarting all pods in the cluster or by performing a rolling node restart. Before performing either action, wait for a new revision of the Kubernetes API server pods to roll out with your service account issuer changes.

   a. Edit the **cluster Authentication** object:

```
$ oc edit authentications cluster
```

   b. Set the **spec.serviceAccountIssuer** field to the desired service account issuer value:

```
spec:
  serviceAccountIssuer: https://test.default.svc
```

This value should be a URL from which the recipient of a bound token can source the public keys necessary to verify the signature of the token. The default is **https://kubernetes.default.svc**.

c. Save the file to apply the changes.

d. Wait for a new revision of the Kubernetes API server pods to roll out. It can take several minutes for all nodes to update to the new revision. Run the following command:

```
$ oc get kubeapiserver -o=jsonpath='{range .items[0].status.conditions[?
(@.type=="NodeInstallerProgressing")]}{.reason}{"\n"}{.message}{"\n"}'
```

Review the **NodeInstallerProgressing** status condition for the Kubernetes API server to verify that all nodes are at the latest revision. The output shows **AllNodesAtLatestRevision** upon successful update:

```
AllNodesAtLatestRevision
3 nodes are at revision 12
```

In this example, the latest revision number is **12**.

If the output shows a message similar to one of the following messages, the update is still in progress. Wait a few minutes and try again.

- **3 nodes are at revision 11; 0 nodes have achieved new revision 12**
- **2 nodes are at revision 11; 1 nodes are at revision 12**

e. Optional: Force the holder to request a new bound token either by performing a rolling node restart or by manually restarting all pods in the cluster.

- Perform a rolling node restart:

WARNING

It is not recommended to perform a rolling node restart if you have custom workloads running on your cluster, because it can cause a service interruption. Instead, manually restart all pods in the cluster.

Restart nodes sequentially. Wait for the node to become fully available before restarting the next node. See *Rebooting a node gracefully* for instructions on how to drain, restart, and mark a node as schedulable again.

- Manually restart all pods in the cluster:

WARNING

Be aware that running this command causes a service interruption, because it deletes every running pod in every namespace. These pods will automatically restart after they are deleted.

Run the following command:

```
$ for I in $(oc get ns -o jsonpath='{range .items[*]} {.metadata.name}{"\n"} {end}'); \
    do oc delete pods --all -n $I; \
    sleep 1; \
    done
```

2. Configure a pod to use a bound service account token by using volume projection.

   a. Create a file called **pod-projected-svc-token.yaml** with the following contents:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - image: nginx
    name: nginx
    volumeMounts:
    - mountPath: /var/run/secrets/tokens
      name: vault-token
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: [ALL]
  serviceAccountName: build-robot
  volumes:
  - name: vault-token
    projected:
      sources:
      - serviceAccountToken:
          path: vault-token
          expirationSeconds: 7200
          audience: vault
```

where:

spec.securityContext.runAsNonRoot

Specifies whether to restrict containers from running as root. When **true**, containers cannot run as root to minimize compromise risks.

spec.securityContext.seccompProfile.type

Specifies the seccomp profile to use. Set to **RuntimeDefault** to use the default seccomp profile, limiting to essential system calls, to reduce risks.

spec.serviceAccountName

Specifies an existing service account.

spec.volumes.projected.sources.serviceAccountToken.path

Specifies a path relative to the mount point of the file to project the token into.

spec.volumes.projected.sources.serviceAccountToken.expirationSeconds

Specifies the expiration of the service account token, in seconds. The default value is 3600 seconds (1 hour). This value must be at least 600 seconds (10 minutes). The kubelet starts trying to rotate the token if the token is older than 80 percent of its time to live or if the token is older than 24 hours. This parameter is optional.

spec.volumes.projected.sources.serviceAccountToken.audience

Specifies the intended audience of the token. The recipient of a token should verify that the recipient identity matches the audience claim of the token, and should otherwise reject the token. The audience defaults to the identifier of the API server. This parameter is optional.

NOTE

In order to prevent unexpected failure, OpenShift Container Platform overrides the **expirationSeconds** value to be one year from the initial token generation with the **--service-account-extend-token-expiration** default of **true**. You cannot change this setting.

b. Create the pod:

```
$ oc create -f pod-projected-svc-token.yaml
```

The kubelet requests and stores the token on behalf of the pod, makes the token available to the pod at a configurable file path, and refreshes the token as it approaches expiration.

3. The application that uses the bound token must handle reloading the token when it rotates. The kubelet rotates the token if it is older than 80 percent of its time to live, or if the token is older than 24 hours.

16.3. CREATING BOUND SERVICE ACCOUNT TOKENS OUTSIDE THE POD

You can create bound service tokens outside of the pod, if needed.

Prerequisites

- You have created a service account. This procedure assumes that the service account is named **build-robot**.

Procedure

- Create the bound service account token outside the pod by running the following command:

```
$ oc create token build-robot
```

Example output

```
eyJhbGciOiJSUzI1NiIsImtpZCI6IkY2M1N4MHRvc2xFNnFSQlA4eG9GYzVPdnN3NkhIV0tRW
mFrUDRNcWx4S0kifQ.eyJhdWQiOlsiaHR0cHM6Ly9pc3N1ZXIyLnRlc3QuY29tIiwiaHR0cHM6L
y9pc3N1ZXIxLnRlc3QuY29tIiwiaHR0cHM6Ly9rdWJlcm5ldGVzLmRlZmF1bHQuc3ZjIl0sImV4c
CI6MTY3OTU0MzgzMCwiaWF0IjoxNjc5NTQwMjMwLCJpc3MiOiJodHRwczovL2lzc3VlcjludGV
zdC5jb20iLCJrdWJlcm5ldGVzLmlvIjp7Im5hbWVzcGFjZSI6ImRlZmF1bHQiLCJzZXJ2aWNlYW
Njb3VudCI6eyJuYW1lIjoidGVzdC1zYSIsInVpZCI6ImM3ZjA4MjkwLWIzOTUtNGM4NC04NjI4L
TMzMTM1NTVhNWY1OSJ9fSwibmJmIjoxNjc5NTQwMjMwLCJzdWIiOiJzeXN0ZW06c2Vydmlj
ZWFjY291bnQ6ZGVmYXVsdDp0ZXN0LXNhIn0.WyAOPvh1BFMUl3LNhBCrQeaB5wSynbnCf
ojWuNNPSilT4YvFnKibxwREwmzHpV4LO1xOFZHSi6bXBOmG_o-
m0XNDYL3FrGHd65mymiFyluztxa2lgHVxjw5reIV5ZLgNSol3Y8bJqQqmNg3rtQQWRML2kpJB
XdDHNww0E5XOypmffYkfkadli8lN5QQD-
MhsCbiAF8waCYs8bj6V6Y7uUKTcxee8sCjiRMVtXKjQtooERKm-
CH_p57wxCljIBeM89VdaR51NJGued4hVV5lxvVrYZFu89lBEAq4oyQN_d6N1vBWGXQMyoihn
t_fQjn-NfnlJWk-3NSZDIluDJAv7e-MTEk3geDrHVQKNEzDei2-Un64hSzb-
n1g1M0Vn0885wQBQAePC9UlZm8YZlMNk1tq6wIUKQTMv3HPfi5HtBRqVc2eVs0EfMX4-x-
PHhPCasJ6qLJWyj6DvyQ08dP4DW_TWZVGvKlmId0hzwpg59TTcLR0iCklSEJgAVEEd13Aa_
M0-
faD11L3MhUGxw0qxgOsPczdXUsolSISbefs7OKymzFSIkTAn9sDQ8PHMOsuyxsK8vzfrR-
E0z7MAeguZ2kaIY7cZqbN6WFy0caWgx46hrKem9vCKALefElRYbCg3hcBmowBcRTOqaFHL
NnHghhU1LaRpoFzH7OUarqX9SGQ
```

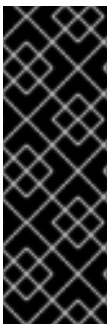
16.4. ADDITIONAL RESOURCES

- Rebooting a node gracefully
- Creating service accounts

CHAPTER 17. MANAGING SECURITY CONTEXT CONSTRAINTS

In OpenShift Container Platform, you can use security context constraints (SCCs) to control permissions for the pods in your cluster.

Default SCCs are created during installation and when you install some Operators or other components. As a cluster administrator, you can also create your own SCCs by using the OpenShift CLI (**oc**).



IMPORTANT

Do not modify the default SCCs. Customizing the default SCCs can lead to issues when some of the platform pods deploy or OpenShift Container Platform is upgraded. Additionally, the default SCC values are reset to the defaults during some cluster upgrades, which discards all customizations to those SCCs.

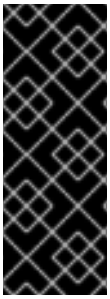
Instead of modifying the default SCCs, create and modify your own SCCs as needed. For detailed steps, see Creating security context constraints.

17.1. ABOUT SECURITY CONTEXT CONSTRAINTS

You can use security context constraints (SCCs) to control permissions for pods by defining what actions a pod can perform, what resources it can access, and what conditions it must meet to be accepted into the system.

Security context constraints allow an administrator to control:

- Whether a pod can run privileged containers with the **allowPrivilegedContainer** flag
- Whether a pod is constrained with the **allowPrivilegeEscalation** flag
- The capabilities that a container can request
- The use of host directories as volumes
- The SELinux context of the container
- The container user ID
- The use of host namespaces and networking
- The allocation of an **FSGroup** that owns the pod volumes
- The configuration of allowable supplemental groups
- Whether a container requires write access to its root file system
- The usage of volume types
- The configuration of allowable **seccomp** profiles

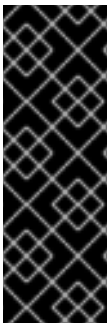


IMPORTANT

Do not set the **openshift.io/run-level** label on any namespaces in OpenShift Container Platform. This label is for use by internal OpenShift Container Platform components to manage the startup of major API groups, such as the Kubernetes API server and OpenShift API server. If the **openshift.io/run-level** label is set, no SCCs are applied to pods in that namespace, causing any workloads running in that namespace to be highly privileged.

17.1.1. Default security context constraints

The cluster contains several default security context constraints (SCCs) as described in the table below. Additional SCCs might be installed when you install Operators or other components to OpenShift Container Platform.



IMPORTANT

Do not modify the default SCCs. Customizing the default SCCs can lead to issues when some of the platform pods deploy or OpenShift Container Platform is upgraded. Additionally, the default SCC values are reset to the defaults during some cluster upgrades, which discards all customizations to those SCCs.

Instead of modifying the default SCCs, create and modify your own SCCs as needed. For detailed steps, see *Creating security context constraints*.

Table 17.1. Default security context constraints

Security context constraint	Description
anyuid	Provides all features of the **restricted** SCC, but allows users to run with any UID and any GID.
hostaccess	Allows access to all host namespaces but still requires pods to be run with a UID and SELinux context that are allocated to the namespace.  **WARNING** This SCC allows host access to namespaces, file systems, and PIDs. It should only be used by trusted pods. Grant with caution.

Security context constraint	Description
hostmount-anyuid	Provides all the features of the **restricted** SCC, but allows host mounts and running as any UID and any GID on the system. **WARNING** This SCC allows host file system access as any UID, including UID 0. Grant with caution.
hostnetwork	Allows using host networking and host ports but still requires pods to be run with a UID and SELinux context that are allocated to the namespace. **WARNING** If additional workloads are run on control plane hosts, use caution when providing access to **hostnetwork**. A workload that runs **hostnetwork** on a control plane host is effectively root on the cluster and must be trusted accordingly.
hostnetwork-v2	Like the **hostnetwork** SCC, but with the following differences: • **ALL** capabilities are dropped from containers. • The **NET_BIND_SERVICE** capability can be added explicitly. • **seccompProfile** is set to **runtime/default** by default. • **allowPrivilegeEscalation** must be unset or set to**false** in security contexts.

Security context constraint	Description
nested-container	Like the **restricted-v2** SCC, but with the following differences: • **seLinuxContext** is set to **MustRunAs** and **seLinuxOptions.type** is **container_engine_t**. • **runAsUser** is set to **MustRunAsRange**. • **requiredDropCapabilities** is set to **null**. • **userNamespaceLevel** is set to **RequirePodLevel**, which forces pods to be in a Linux user namespace (**hostUsers: false**). This SCC allows a user to run a container engine inside of an OpenShift Container Platform pod.
node-exporter	Used for the Prometheus node exporter. **WARNING** This SCC allows host file system access as any UID, including UID 0. Grant with caution.
nonroot	Provides all features of the **restricted** SCC, but allows users to run with any non-root UID. The user must specify the UID or it must be specified in the manifest of the container runtime.
nonroot-v2	Like the **nonroot** SCC, but with the following differences: • **ALL** capabilities are dropped from containers. • The **NET_BIND_SERVICE** capability can be added explicitly. • **seccompProfile** is set to **runtime/default** by default. • **allowPrivilegeEscalation** must be unset or set to**false** in security contexts.

Security context constraint	Description
privileged	Allows access to all privileged and host features and the ability to run as any user, any group, any FSGroup, and with any SELinux context. **WARNING** This is the most relaxed SCC and should be used only for cluster administration. Grant with caution. The **privileged** SCC allows: • Users to run privileged pods • Pods to mount host directories as volumes • Pods to run as any user • Pods to run with any MCS label • Pods to use the host's IPC namespace • Pods to use the host's PID namespace • Pods to use any FSGroup • Pods to use any supplemental group • Pods to use any seccomp profiles • Pods to request any capabilities **NOTE** Setting **privileged: true** in the pod specification does not necessarily select the **privileged** SCC. The SCC that has **allowPrivilegedContainer: true** and has the highest prioritization will be chosen if the user has the permissions to use it.

Security context constraint	Description
restricted	Denies access to all host features and requires pods to be run with a UID, and SELinux context that are allocated to the namespace. The **restricted** SCC: • Ensures that pods cannot run as privileged • Ensures that pods cannot mount host directory volumes • Requires that a pod is run as a user in a pre-allocated range of UIDs • Requires that a pod is run with a pre-allocated MCS label • Requires that a pod is run with a preallocated FSGroup • Allows pods to use any supplemental group In clusters that were upgraded from OpenShift Container Platform 4.10 or earlier, this SCC is available for use by any authenticated user. The **restricted** SCC is no longer available to users of new OpenShift Container Platform 4.11 or later installations, unless the access is explicitly granted.
restricted-v2	Like the **restricted** SCC, but with the following differences: • **ALL** capabilities are dropped from containers. • The **NET_BIND_SERVICE** capability can be added explicitly. • **seccompProfile** is set to **runtime/default** by default. • **allowPrivilegeEscalation** must be unset or set to**false** in security contexts. This SCC is used by default for authenticated users.
restricted-v3	Like the **restricted-v2** SCC, but with the following differences: • **UserNamespaceLevel** is set to **RequirePodLevel**, which forces pods to be in a Linux user namespace (**hostUsers: false**). This is the most restrictive SCC provided by a new installation and will be used by default for authenticated users.  NOTE The **restricted-v3** SCC is the most restrictive of the SCCs that is included by default with the system. However, you can create a custom SCC that is even more restrictive. For example, you can create an SCC that restricts **readOnlyRootFilesystem** to **true**.

17.1.2. Security context constraints settings

Security context constraints (SCCs) are composed of settings and strategies that control the security features a pod has access to. These settings fall into three categories:

Category	Description
Controlled by a boolean	Fields of this type default to the most restrictive value. For example, **AllowPrivilegedContainer** is always set to **false** if unspecified.
Controlled by an allowable set	Fields of this type are checked against the set to ensure their value is allowed.
Controlled by a strategy	Items that have a strategy to generate a value provide: • A mechanism to generate the value, and • A mechanism to ensure that a specified value falls into the set of allowable values.

CRI-O has the following default list of capabilities that are allowed for each container of a pod:

- **CHOWN**
- **DAC_OVERRIDE**
- **FSETID**
- **FOWNER**
- **SETGID**
- **SETUID**
- **SETPCAP**
- **NET_BIND_SERVICE**
- **KILL**

The containers use the capabilities from this default list, but pod manifest authors can alter the list by requesting additional capabilities or removing some of the default behaviors. Use the **allowedCapabilities**, **defaultAddCapabilities**, and **requiredDropCapabilities** parameters to control such requests from the pods. With these parameters you can specify which capabilities can be requested, which ones must be added to each container, and which ones must be forbidden, or dropped, from each container.

NOTE

You can drop all capabilites from containers by setting the **requiredDropCapabilities** parameter to **ALL**. This is what the **restricted-v2** SCC does.

17.1.3. Security context constraints strategies

RunAsUser

- **MustRunAs** - Requires a **runAsUser** to be configured. Uses the configured **runAsUser** as the default. Validates against the configured **runAsUser**.

  Example **MustRunAs** snippet

```
...
runAsUser:
  type: MustRunAs
  uid: <id>
...
```

- **MustRunAsRange** - Requires minimum and maximum values to be defined if not using pre-allocated values. Uses the minimum as the default. Validates against the entire allowable range.

  Example **MustRunAsRange** snippet

```
...
runAsUser:
  type: MustRunAsRange
  uidRangeMax: <maxvalue>
  uidRangeMin: <minvalue>
...
```

- **MustRunAsNonRoot** - Requires that the pod be submitted with a non-zero **runAsUser** or have the **USER** directive defined in the image. No default provided.

  Example **MustRunAsNonRoot** snippet

```
...
runAsUser:
  type: MustRunAsNonRoot
...
```

- **RunAsAny** - No default provided. Allows any **runAsUser** to be specified.

  Example **RunAsAny** snippet

```
...
runAsUser:
  type: RunAsAny
...
```

SELinuxContext

- **MustRunAs** - Requires **seLinuxOptions** to be configured if not using pre-allocated values. Uses **seLinuxOptions** as the default. Validates against **seLinuxOptions**.

- **RunAsAny** - No default provided. Allows any **seLinuxOptions** to be specified.

SupplementalGroups

- **MustRunAs** - Requires at least one range to be specified if not using pre-allocated values. Uses the minimum value of the first range as the default. Validates against all ranges.
- **RunAsAny** - No default provided. Allows any **supplementalGroups** to be specified.

FSGroup

- **MustRunAs** - Requires at least one range to be specified if not using pre-allocated values. Uses the minimum value of the first range as the default. Validates against the first ID in the first range.
- **RunAsAny** - No default provided. Allows any **fsGroup** ID to be specified.

17.1.4. Controlling volumes

The usage of specific volume types can be controlled by setting the **volumes** field of the SCC.

The allowable values of this field correspond to the volume sources that are defined when creating a volume:

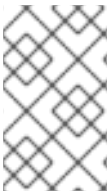
- **awsElasticBlockStore**
- **azureDisk**
- **azureFile**
- **cephFS**
- **cinder**
- **configMap**
- **csi**
- **downwardAPI**
- **emptyDir**
- **fc**
- **flexVolume**
- **flocker**
- **gcePersistentDisk**
- **ephemeral**
- **gitRepo**
- **glusterfs**
- **hostPath**
- **iscsi**

- **nfs**
- **persistentVolumeClaim**
- **photonPersistentDisk**
- **portworxVolume**
- **projected**
- **quobyte**
- **rbd**
- **scaleIO**
- **secret**
- **storageos**
- **vsphereVolume**
- ***** (A special value to allow the use of all volume types.)
- **none** (A special value to disallow the use of all volumes types. Exists only for backwards compatibility.)

The recommended minimum set of allowed volumes for new SCCs are **configMap**, **downwardAPI**, **emptyDir**, **persistentVolumeClaim**, **secret**, and **projected**.

NOTE

This list of allowable volume types is not exhaustive because new types are added with each release of OpenShift Container Platform.



NOTE

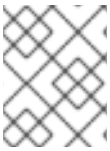
For backwards compatibility, the usage of **allowHostDirVolumePlugin** overrides settings in the **volumes** field. For example, if **allowHostDirVolumePlugin** is set to false but allowed in the **volumes** field, then the **hostPath** value will be removed from **volumes**.

17.1.5. Admission control

Admission control with SCCs allows for control over the creation of resources based on the capabilities granted to a user.

In terms of the SCCs, this means that an admission controller can inspect the user information made available in the context to retrieve an appropriate set of SCCs. Doing so ensures the pod is authorized to make requests about its operating environment or to generate a set of constraints to apply to the pod.

The set of SCCs that admission uses to authorize a pod are determined by the user identity and groups that the user belongs to. Additionally, if the pod specifies a service account, the set of allowable SCCs includes any constraints accessible to the service account.



NOTE

When you create a workload resource, such as deployment, only the service account is used to find the SCCs and admit the pods when they are created.

Admission uses the following approach to create the final security context for the pod:

1. Retrieve all SCCs available for use.
2. Generate field values for security context settings that were not specified on the request.
3. Validate the final settings against the available constraints.

If a matching set of constraints is found, then the pod is accepted. If the request cannot be matched to an SCC, the pod is rejected.

A pod must validate every field against the SCC. The following are examples for just two of the fields that must be validated:

NOTE

These examples are in the context of a strategy using the pre-allocated values.

An FSGroup SCC strategy of MustRunAs

If the pod defines a **fsGroup** ID, then that ID must equal the default **fsGroup** ID. Otherwise, the pod is not validated by that SCC and the next SCC is evaluated.

If the **SecurityContextConstraints.fsGroup** field has value **RunAsAny** and the pod specification omits the **Pod.spec.securityContext.fsGroup**, then this field is considered valid. Note that it is possible that during validation, other SCC settings will reject other pod fields and thus cause the pod to fail.

A SupplementalGroups SCC strategy of MustRunAs

If the pod specification defines one or more **supplementalGroups** IDs, then the pod's IDs must equal one of the IDs in the namespace's **openshift.io/sa.scc.supplemental-groups** annotation. Otherwise, the pod is not validated by that SCC and the next SCC is evaluated.

If the **SecurityContextConstraints.supplementalGroups** field has value **RunAsAny** and the pod specification omits the **Pod.spec.securityContext.supplementalGroups**, then this field is considered valid. Note that it is possible that during validation, other SCC settings will reject other pod fields and thus cause the pod to fail.

17.1.6. Security context constraints prioritization

Security context constraints (SCCs) have a priority field that affects the ordering when attempting to validate a request by the admission controller.

WARNING

Setting an SCC priority greater than 0 for the default OpenShift Container Platform SCCs can cause critical cluster instability.

A priority value of **0** is the lowest possible priority. A nil priority is considered a **0**, or lowest, priority. Higher priority SCCs are moved to the front of the set when sorting.

When the complete set of available SCCs is determined, the SCCs are ordered in the following manner:

1. The highest priority SCCs are ordered first.
2. If the priorities are equal, the SCCs are sorted from most restrictive to least restrictive.
3. If both the priorities and restrictions are equal, the SCCs are sorted by name.

By default, the **anyuid** SCC granted to cluster administrators is given priority in their SCC set. This allows cluster administrators to run pods as any user by specifying **RunAsUser** in the pod's **SecurityContext**.

17.2. ABOUT PRE-ALLOCATED SECURITY CONTEXT CONSTRAINTS VALUES

When your security context constraints (SCCs) do not define explicit ranges, the admission controller automatically populates them with pre-allocated values from the namespace before processing your pods.

Each SCC strategy is evaluated independently of other strategies, with the pre-allocated values, where allowed, for each policy aggregated with pod specification values to make the final values for the various IDs defined in the running pod.

The following SCCs cause the admission controller to look for pre-allocated values when no ranges are defined in the pod specification:

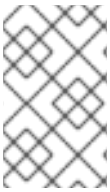
1. A **RunAsUser** strategy of **MustRunAsRange** with no minimum or maximum set. Admission looks for the **openshift.io/sa.scc.uid-range** annotation to populate range fields.
2. An **SELinuxContext** strategy of **MustRunAs** with no level set. Admission looks for the **openshift.io/sa.scc.mcs** annotation to populate the level.
3. A **FSGroup** strategy of **MustRunAs**. Admission looks for the **openshift.io/sa.scc.supplemental-groups** annotation.
4. A **SupplementalGroups** strategy of **MustRunAs**. Admission looks for the **openshift.io/sa.scc.supplemental-groups** annotation.

During the generation phase, the security context provider uses default values for any parameter values that are not specifically set in the pod. Default values are based on the selected strategy:

1. **RunAsAny** and **MustRunAsNonRoot** strategies do not provide default values. If the pod needs a parameter value, such as a group ID, you must define the value in the pod specification.

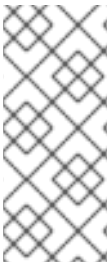
2. **MustRunAs** (single value) strategies provide a default value that is always used. For example, for group IDs, even if the pod specification defines its own ID value, the namespace's default parameter value also appears in the pod's groups.

3. **MustRunAsRange** and **MustRunAs** (range-based) strategies provide the minimum value of the range. As with a single value **MustRunAs** strategy, the namespace's default parameter value appears in the running pod. If a range-based strategy is configurable with multiple ranges, it provides the minimum value of the first configured range.



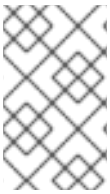
NOTE

FSGroup and **SupplementalGroups** strategies fall back to the **openshift.io/sa.scc.uid-range** annotation if the **openshift.io/sa.scc.supplemental-groups** annotation does not exist on the namespace. If neither exists, the SCC is not created.



NOTE

By default, the annotation-based **FSGroup** strategy configures itself with a single range based on the minimum value for the annotation. For example, if your annotation reads **1/3**, the **FSGroup** strategy configures itself with a minimum and maximum value of **1**. If you want to allow more groups to be accepted for the **FSGroup** field, you can configure a custom SCC that does not use the annotation.



NOTE

The **openshift.io/sa.scc.supplemental-groups** annotation accepts a comma-delimited list of blocks in the format of **<start>/<length** or **<start>-<end>**. The **openshift.io/sa.scc.uid-range** annotation accepts only a single block.

17.3. EXAMPLE SECURITY CONTEXT CONSTRAINTS

The following examples show how to define and use security context constraints (SCCs) in your cluster.

Annotated privileged SCC

```
allowHostDirVolumePlugin: true
allowHostIPC: true
allowHostNetwork: true
allowHostPID: true
allowHostPorts: true
allowPrivilegedContainer: true
allowedCapabilities:
- '*'
apiVersion: security.openshift.io/v1
defaultAddCapabilities: []
fsGroup:
  type: RunAsAny
groups:
- system:cluster-admins
- system:nodes
kind: SecurityContextConstraints
```

```
metadata:
  annotations:
    kubernetes.io/description: 'privileged allows access to all privileged and host
      features and the ability to run as any user, any group, any fsGroup, and with
      any SELinux context.  WARNING: this is the most relaxed SCC and should be used
      only for cluster administration. Grant with caution.'
  creationTimestamp: null
  name: privileged
priority: null
readOnlyRootFilesystem: false
requiredDropCapabilities: null
runAsUser:
  type: RunAsAny
seLinuxContext:
  type: RunAsAny
seccompProfiles:
- '*'
supplementalGroups:
  type: RunAsAny
users:
- system:serviceaccount:default:registry
- system:serviceaccount:default:router
- system:serviceaccount:openshift-infra:build-controller
volumes:
- '*'
```

where, **allowedCapabilities**

A list of capabilities that a pod can request. An empty list means that none of capabilities can be requested while the special symbol ***** allows any capabilities.

defaultAddCapabilities

A list of additional capabilities that are added to any pod.

fsGroup

The **FSGroup** strategy, which dictates the allowable values for the security context.

groups

The groups that can access this SCC.

requiredDropCapabilities

A list of capabilities to drop from a pod. Or, specify **ALL** to drop all capabilities.

runAsUser

The **runAsUser** strategy type, which dictates the allowable values for the security context.

seLinuxContext

The **seLinuxContext** strategy type, which dictates the allowable values for the security context.

supplementalGroups

The **supplementalGroups** strategy, which dictates the allowable supplemental groups for the security context.

users

The users who can access this SCC.

volumes

The allowable volume types for the security context. In the example, * allows the use of all volume types.

The **users** and **groups** fields on the SCC control which users can access the SCC. By default, cluster administrators, nodes, and the build controller are granted access to the privileged SCC. All authenticated users are granted access to the **restricted-v2** SCC.

NOTE

When verifying access to an SCC, be aware of the following command behaviors:

- The **oc adm policy who-can use scc <scc_name>** and **oc auth can-i use scc/<scc_name>** commands evaluate only RBAC policies (**RoleBinding** or **ClusterRoleBinding** resources). Their output does not include users or groups configured directly in the SCC **users** and **groups** fields.
- The **oc describe scc <scc_name>** command displays only the users and groups configured directly within the SCC object. Its output does not include access granted through RBAC policies.

Without explicit runAsUser setting

```
apiVersion: v1
kind: Pod
metadata:
  name: security-context-demo
spec:
  securityContext:
  containers:
  - name: sec-ctx-demo
    image: gcr.io/google-samples/node-hello:1.0
```

When a container or pod does not request a user ID under which it should be run,the effective UID depends on the SCC that emits this pod. Because the **restricted-v2** SCC is granted to all authenticated users by default, it will be available to all users and service accounts and used in most cases. The **restricted-v2** SCC uses **MustRunAsRange** strategy for constraining and defaulting the possible values of the **securityContext.runAsUser** field. The admission plugin will look for the **openshift.io/sa.scc.uid-range** annotation on the current project to populate range fields, as it does not provide this range. In the end, a container will have **runAsUser** equal to the first value of the range that is hard to predict because every project has different ranges.

With explicit runAsUser setting

```
apiVersion: v1
kind: Pod
metadata:
  name: security-context-demo
spec:
  securityContext:
    runAsUser: 1000
  containers:
    - name: sec-ctx-demo
      image: gcr.io/google-samples/node-hello:1.0
```

A container or pod that requests a specific user ID will be accepted by OpenShift Container Platform only when a service account or a user is granted access to a SCC that allows such a user ID. The SCC can allow arbitrary IDs, an ID that falls into a range, or the exact user ID specific to the request.

This configuration is valid for SELinux, fsGroup, and Supplemental Groups.

17.4. CREATING SECURITY CONTEXT CONSTRAINTS

If the default security context constraints (SCCs) do not satisfy your application workload requirements, you can create a custom SCC by using the OpenShift CLI (**oc**).

IMPORTANT

Creating and modifying your own SCCs are advanced operations that might cause instability to your cluster. If you have questions about using your own SCCs, contact Red Hat Support. For information about contacting Red Hat support, see *Getting support*.

WARNING

Setting an SCC priority greater than 0 for the default OpenShift Container Platform SCCs can cause critical cluster instability.

Prerequisites

- Install the OpenShift CLI (**oc**).
- Log in to the cluster as a user with the **cluster-admin** role.

Procedure

1. Define the SCC in a YAML file named **scc-admin.yaml**:

```
kind: SecurityContextConstraints
apiVersion: security.openshift.io/v1
metadata:
  name: scc-admin
allowPrivilegedContainer: true
runAsUser:
  type: RunAsAny
seLinuxContext:
  type: RunAsAny
fsGroup:
  type: RunAsAny
supplementalGroups:
  type: RunAsAny
users:
- my-admin-user
groups:
- my-admin-group
```

Optionally, you can drop specific capabilities for an SCC by setting the **requiredDropCapabilities** field with the desired values. Any specified capabilities are dropped from the container. To drop all capabilities, specify **ALL**. For example, to create an SCC that drops the **KILL**, **MKNOD**, and **SYS_CHROOT** capabilities, add the following to the SCC object:

```
requiredDropCapabilities:
- KILL
- MKNOD
- SYS_CHROOT
```

NOTE

You cannot list a capability in both **allowedCapabilities** and **requiredDropCapabilities**.

CRI-O supports the same list of capability values that are found in the Docker documentation.

2. Create the SCC by passing in the file:

```
$ oc create -f scc-admin.yaml
```

Example output

```
securitycontextconstraints "scc-admin" created
```

Verification

- Verify that the SCC was created:

```
$ oc get scc scc-admin
```

Example output

```
NAME      PRIV    CAPS    SELINUX    RUNASUSER    FSGROUP    SUPGROUP
PRIORITY    READONLYROOTFS    VOLUMES
scc-admin    true    []    RunAsAny    RunAsAny    RunAsAny    RunAsAny    <none>    false
[awsElasticBlockStore azureDisk azureFile cephFS cinder configMap downwardAPI
emptyDir fc flexVolume flocker gcePersistentDisk gitRepo glusterfs iscsi nfs
persistentVolumeClaim photonPersistentDisk quobyte rbd secret vsphere]
```

17.5. CONFIGURING A WORKLOAD TO REQUIRE A SPECIFIC SCC

You can configure a workload to require a certain security context constraint (SCC). This is useful in scenarios where you want to pin a specific SCC to the workload or if you want to prevent your required SCC from being preempted by another SCC in the cluster.

To require a specific SCC, set the **openshift.io/required-scc** annotation on your workload. You can set this annotation on any resource that can set a pod manifest template, such as a deployment or daemon set.

The SCC must exist in the cluster and must be applicable to the workload, otherwise pod admission fails. An SCC is considered applicable to the workload if the user creating the pod or the pod's service account has **use** permissions for the SCC in the pod's namespace.

WARNING

Do not change the **openshift.io/required-scc** annotation in the live pod's manifest, because doing so causes the pod admission to fail. To change the required SCC, update the annotation in the underlying pod template, which causes the pod to be deleted and re-created.

Prerequisites

- The SCC must exist in the cluster.

Procedure

1. Create a YAML file for the deployment and specify a required SCC by setting the **openshift.io/required-scc** annotation:

   Example deployment.yaml

   ```
   apiVersion: config.openshift.io/v1
   kind: Deployment
   apiVersion: apps/v1
   spec:
   # ...
     template:
       metadata:
         annotations:
           openshift.io/required-scc: "my-scc"
   # ...
   ```

2. Create the resource by running the following command:

   ```
   $ oc create -f deployment.yaml
   ```

Verification

- Verify that the deployment used the specified SCC:

  a. View the value of the pod's **openshift.io/scc** annotation by running the following command, replacing **<pod_name>** with the name of your deployment pod:

     ```
     $ oc get pod <pod_name> -o jsonpath='{.metadata.annotations.openshift\.io\/scc}{"\n"}'
     ```
     (1)

  b. Examine the output and confirm that the displayed SCC matches the SCC that you defined in the deployment:

Example output

```
my-scc
```

17.6. ROLE-BASED ACCESS TO SECURITY CONTEXT CONSTRAINTS

You can specify SCCs as resources that are handled by RBAC. This allows you to scope access to your SCCs to a certain project or to the entire cluster. Assigning users, groups, or service accounts directly to an SCC retains cluster-wide scope.

IMPORTANT

Do not run workloads in or share access to default projects. Default projects are reserved for running core cluster components.

The following default projects are considered highly privileged: **default**, **kube-public**, **kube-system**, **openshift**, **openshift-infra**, **openshift-node**, and other system-created projects that have the **openshift.io/run-level** label set to **0** or **1**. Functionality that relies on admission plugins, such as pod security admission, security context constraints, cluster resource quotas, and image reference resolution, does not work in highly privileged projects.

To include access to SCCs for your role, specify the **scc** resource when creating a role.

```
$ oc create role <role-name> --verb=use --resource=scc --resource-name=<scc-name> -n
<namespace>
```

This results in the following role definition:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
...
  name: role-name
  namespace: namespace
...
rules:
- apiGroups:
  - security.openshift.io
  resourceNames:
  - scc-name
  resources:
  - securitycontextconstraints
  verbs:
  - use
```

where, name

The name of the role.

namespace

The namespace of the defined role. Defaults to **default** if not specified.

apiGroups

The API group that includes the **SecurityContextConstraints** resource. Automatically defined when **scc** is specified as a resource.

resourceName

An example name for an SCC you want to have access.

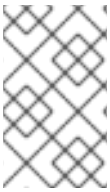
resources

The name of the resource group that allows users to specify SCC names in the **resourceNames** field.

verbs

A list of verbs to apply to the role.

A local or cluster role with such a rule allows the subjects that are bound to it with a role binding or a cluster role binding to use the user-defined SCC called **scc-name**.



NOTE

Because RBAC is designed to prevent escalation, even project administrators are unable to grant access to an SCC. By default, they are not allowed to use the verb **use** on SCC resources, including the **restricted-v2** SCC.

17.7. REFERENCE OF SECURITY CONTEXT CONSTRAINTS COMMANDS

You can manage security context constraints (SCCs) in your instance as normal API objects by using the OpenShift CLI (**oc**).

NOTE

You must have **cluster-admin** privileges to manage SCCs.

17.7.1. Listing security context constraints

To get a current list of SCCs:

```
$ oc get scc
```

Example output

```
NAME                              PRIV    CAPS                    SELINUX     RUNASUSER          FSGROUP
SUPGROUP    PRIORITY     READONLYROOTFS   VOLUMES
anyuid                            false   <no value>              MustRunAs   RunAsAny           RunAsAny
RunAsAny    10           false
["configMap","downwardAPI","emptyDir","persistentVolumeClaim","projected","secret"]
hostaccess                        false   <no value>              MustRunAs   MustRunAsRange     MustRunAs
RunAsAny    <no value>   false
["configMap","downwardAPI","emptyDir","hostPath","persistentVolumeClaim","projected","secret"]
hostmount-anyuid                  false   <no value>              MustRunAs   RunAsAny           RunAsAny
RunAsAny    <no value>   false
["configMap","downwardAPI","emptyDir","hostPath","nfs","persistentVolumeClaim","projected","secret"]

hostnetwork                       false   <no value>              MustRunAs   MustRunAsRange     MustRunAs
```

```
MustRunAs   <no value>   false
["configMap","downwardAPI","emptyDir","persistentVolumeClaim","projected","secret"]
hostnetwork-v2                false   ["NET_BIND_SERVICE"]   MustRunAs   MustRunAsRange
MustRunAs   MustRunAs   <no value>   false
["configMap","downwardAPI","emptyDir","persistentVolumeClaim","projected","secret"]
node-exporter                 true    <no value>          RunAsAny    RunAsAny          RunAsAny
RunAsAny    <no value>   false          ["*"]
nonroot                       false   <no value>          MustRunAs   MustRunAsNonRoot   RunAsAny
RunAsAny    <no value>   false
["configMap","downwardAPI","emptyDir","persistentVolumeClaim","projected","secret"]
nonroot-v2                    false   ["NET_BIND_SERVICE"]   MustRunAs   MustRunAsNonRoot
RunAsAny    RunAsAny    <no value>   false
["configMap","downwardAPI","emptyDir","persistentVolumeClaim","projected","secret"]
privileged                    true    ["*"]               RunAsAny    RunAsAny          RunAsAny    RunAsAny
<no value>   false          ["*"]
restricted                    false   <no value>          MustRunAs   MustRunAsRange     MustRunAs
RunAsAny    <no value>   false
["configMap","downwardAPI","emptyDir","persistentVolumeClaim","projected","secret"]
restricted-v2                 false   ["NET_BIND_SERVICE"]   MustRunAs   MustRunAsRange
MustRunAs   RunAsAny    <no value>   false
["configMap","downwardAPI","emptyDir","persistentVolumeClaim","projected","secret"]
```

17.7.2. Examining security context constraints

You can view information about a particular SCC, including which users, service accounts, and groups the SCC is applied to.

For example, to examine the **restricted** SCC:

```
$ oc describe scc restricted
```

Example output

```
Name:                           restricted
Priority:                       <none>
Access:
  Users:                        <none>
  Groups:                       <none>
Settings:
  Allow Privileged:             false
  Allow Privilege Escalation:       true
  Default Add Capabilities:         <none>
  Required Drop Capabilities:       KILL,MKNOD,SETUID,SETGID
  Allowed Capabilities:             <none>
  Allowed Seccomp Profiles:         <none>
  Allowed Volume Types:
configMap,downwardAPI,emptyDir,persistentVolumeClaim,projected,secret
  Allowed Flexvolumes:              <all>
  Allowed Unsafe Sysctls:           <none>
  Forbidden Sysctls:                <none>
  Allow Host Network:               false
  Allow Host Ports:                 false
  Allow Host PID:                   false
  Allow Host IPC:                   false
  Read Only Root Filesystem:        false
```

```
Run As User Strategy: MustRunAsRange
  UID:                    <none>
  UID Range Min:          <none>
  UID Range Max:          <none>
SELinux Context Strategy: MustRunAs
  User:                   <none>
  Role:                   <none>
  Type:                   <none>
  Level:                  <none>
FSGroup Strategy: MustRunAs
  Ranges:                 <none>
Supplemental Groups Strategy: RunAsAny
  Ranges:                 <none>
```

where, Users

Lists which users and service accounts the SCC is applied to.

Groups

Lists which groups the SCC is applied to.

NOTE

To preserve customized SCCs during upgrades, do not edit settings on the default SCCs.

17.7.3. Updating security context constraints

If your custom SCC no longer satisfies your application workloads requirements, you can update your SCC by using the OpenShift CLI (**oc**).

To update an existing SCC:

```
$ oc edit scc <scc_name>
```

IMPORTANT

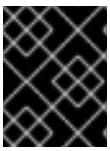
To preserve customized SCCs during upgrades, do not edit settings on the default SCCs.

17.7.4. Deleting security context constraints

If you no longer require your custom SCC, you can delete the SCC by using the OpenShift CLI (**oc**).

To delete an SCC:

```
$ oc delete scc <scc_name>
```



IMPORTANT

Do not delete default SCCs. If you delete a default SCC, it is regenerated by the Cluster Version Operator.

17.8. ADDITIONAL RESOURCES

- Getting support

CHAPTER 18. UNDERSTANDING AND MANAGING POD SECURITY ADMISSION

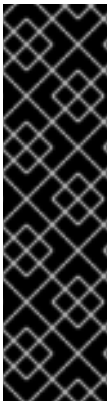
You can configure pod security admission to enforce the Kubernetes pod security standards. You can apply this enforcement at both the global and namespace levels.

18.1. ABOUT POD SECURITY ADMISSION

You can use pod security admission modes, such as **enforce**, **warn**, or **audit**, along with security profiles to restrict which pods run in your cluster. You can apply this control at both the global and namespace levels.

Globally, the **privileged** profile is enforced, and the **restricted** profile is used for warnings and audits.

You can also configure the pod security admission settings at the namespace level.



IMPORTANT

Do not run workloads in or share access to default projects. Default projects are reserved for running core cluster components.

The following default projects are considered highly privileged: **default**, **kube-public**, **kube-system**, **openshift**, **openshift-infra**, **openshift-node**, and other system-created projects that have the **openshift.io/run-level** label set to **0** or **1**. Functionality that relies on admission plugins, such as pod security admission, security context constraints, cluster resource quotas, and image reference resolution, does not work in highly privileged projects.

18.1.1. Pod security admission modes

You can configure the following pod security admission modes for a namespace:

Table 18.1. Pod security admission modes

Mode	Label	Description
enforce	**pod-security.kubernetes.io/enforce**	Rejects a pod from admission if it does not comply with the set profile
audit	**pod-security.kubernetes.io/audit**	Logs audit events if a pod does not comply with the set profile
warn	**pod-security.kubernetes.io/warn**	Displays warnings if a pod does not comply with the set profile

18.1.2. Pod security admission profiles

You can set each of the pod security admission modes to one of the following profiles:

Table 18.2. Pod security admission profiles

Profile	Description
privileged	Least restrictive policy; allows for known privilege escalation
baseline	Minimally restrictive policy; prevents known privilege escalations
restricted	Most restrictive policy; follows current pod hardening best practices

18.1.3. Privileged namespaces

The following system namespaces are always set to the **privileged** pod security admission profile:

- **default**
- **kube-public**
- **kube-system**

You cannot change the pod security profile for these privileged namespaces.

Example privileged namespace configuration

```
apiVersion: v1
kind: Namespace
metadata:
  labels:
    openshift.io/cluster-monitoring: "true"
    pod-security.kubernetes.io/enforce: privileged
    pod-security.kubernetes.io/audit: privileged
    pod-security.kubernetes.io/warn: privileged
  name: "<mig_namespace>"
# ...
```

18.1.4. Pod security admission and security context constraints

Pod security admission and security context constraints operate as two independent mechanisms in OpenShift Container Platform. You must ensure your workloads comply with both to avoid unexpected pod rejections.

The two controllers independently enforce security policies by using the following processes:

1. The security context constraint controller may mutate some security context fields per the pod's assigned SCC. For example, if the seccomp profile is empty or not set and if the pod's assigned SCC enforces **seccompProfiles** field to be **runtime/default**, the controller sets the default type to **RuntimeDefault**.

2. The security context constraint controller validates the pod's security context against the matching SCC.

3. The pod security admission controller validates the pod's security context against the pod security standard assigned to the namespace.

18.2. ABOUT POD SECURITY ADMISSION SYNCHRONIZATION

Pod security admission **warn** and **audit** labels are automatically synchronized on your namespaces. This synchronization maps security context constraints to pod security profiles based on the service account permissions in each namespace.

The controller examines **ServiceAccount** object permissions to use security context constraints in each namespace. Security context constraints (SCCs) are mapped to pod security profiles based on their field values; the controller uses these translated profiles. Pod security admission **warn** and **audit** labels are set to the most privileged pod security profile in the namespace to prevent displaying warnings and logging audit events when pods are created.

Namespace labeling is based on consideration of namespace-local service account privileges.

Applying pods directly might use the SCC privileges of the user who runs the pod. However, user privileges are not considered during automatic labeling.

18.2.1. Pod security admission synchronization namespace exclusions

If you use pod security admission synchronization, the system-created namespaces are permanently disabled from synchronization.

User-created **openshift-*** prefixed namespaces are also initially disabled, but you can enable synchronization on them later.

IMPORTANT

If a pod security admission label (**pod-security.kubernetes.io/<mode>**) is manually modified from the automatically labeled value on a label-synchronized namespace, synchronization is disabled for that label.

If necessary, you can enable synchronization again by using one of the following methods:

- By removing the modified pod security admission label from the namespace
- By setting the **security.openshift.io/scc.podSecurityLabelSync** label to **true**
  If you force synchronization by adding this label, then any modified pod security admission labels will be overwritten.

18.2.1.1. Permanently disabled namespaces

Namespaces that are defined as part of the cluster payload have pod security admission synchronization disabled permanently. The following namespaces are permanently disabled:

- **default**
- **kube-node-lease**
- **kube-system**
- **kube-public**
- **openshift**

- All system-created namespaces that are prefixed with **openshift-** , except for **openshift-operators**

18.2.1.2. Initially disabled namespaces

By default, all namespaces that have an **openshift-** prefix have pod security admission synchronization disabled initially. You can enable synchronization for user-created **openshift-*** namespaces and for the **openshift-operators** namespace.

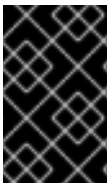
NOTE

You cannot enable synchronization for any system-created **openshift-*** namespaces, except for **openshift-operators**.

If an Operator is installed in a user-created **openshift-*** namespace, synchronization is enabled automatically after a cluster service version (CSV) is created in the namespace. The synchronized label is derived from the permissions of the service accounts in the namespace.

18.3. CONTROLLING POD SECURITY ADMISSION SYNCHRONIZATION

To customize which namespaces have their pod security admission labels automatically updated, you can enable or disable synchronization for most namespaces.



IMPORTANT

You cannot enable pod security admission synchronization on some system-created namespaces. For more information, see *Pod security admission synchronization namespace exclusions*.

Procedure

- For each namespace that you want to configure, set a value for the **security.openshift.io/scc.podSecurityLabelSync** label:
  - To disable pod security admission label synchronization in a namespace, set the value of the **security.openshift.io/scc.podSecurityLabelSync** label to **false**.
    Run the following command:

    ```
    $ oc label namespace <namespace>
    security.openshift.io/scc.podSecurityLabelSync=false
    ```

  - To enable pod security admission label synchronization in a namespace, set the value of the **security.openshift.io/scc.podSecurityLabelSync** label to **true**.
    Run the following command:

    ```
    $ oc label namespace <namespace>
    security.openshift.io/scc.podSecurityLabelSync=true
    ```

NOTE

Use the **--overwrite** flag to overwrite the value if this label is already set on the namespace.

Additional resources

- Pod security admission synchronization namespace exclusions

18.4. CONFIGURING POD SECURITY ADMISSION FOR A NAMESPACE

You can configure pod security admission modes and profiles at the namespace level to control the security standards that pods must meet in a specific namespace.

Procedure

- For each pod security admission mode that you want to set on a namespace, run the following command:

```
$ oc label namespace <namespace> \
    pod-security.kubernetes.io/<mode>=<profile> \
    --overwrite
```

where:

<namespace>

Specifies the namespace to configure.

<mode>

Specifies the pod security admission mode. Valid values are **enforce**, **warn**, or **audit**.

<profile>

Specifies the pod security profile. Valid values are **restricted**, **baseline**, or **privileged**.

18.5. ABOUT POD SECURITY ADMISSION ALERTS

If your pods violate the configured pod security standards, you receive a **PodSecurityViolation** alert. This alert persists for one day so that you can investigate and resolve compliance issues.

You can view the Kubernetes API server audit logs to investigate alerts that were triggered. As an example, a workload is likely to fail admission if global enforcement is set to the **restricted** pod security level.

To identify pod security admission violation audit events, see "Audit annotations" in the Kubernetes documentation.

18.5.1. Identifying pod security violations

To identify which workloads are causing pod security violations, you can review the Kubernetes API server audit logs by using the **must-gather** tool.

The **PodSecurityViolation** alert does not provide details on which workloads are causing pod security violations.

Prerequisites

- You have installed **jq**.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

1. To gather the audit logs, enter the following command:

```
$ oc adm must-gather -- /usr/bin/gather_audit_logs
```

2. To output the affected workload details, enter the following command:

```
$ zgrep -h pod-security.kubernetes.io/audit-violations must-gather.local.
<archive_id>/<image_digest_id>/audit_logs/kube-apiserver/*log.gz \
  | jq -r 'select((.annotations["pod-security.kubernetes.io/audit-violations"] != null) and
(.objectRef.resource=="pods")) | .objectRef.namespace + " " + .objectRef.name' \
  | sort | uniq -c
```

Replace **<archive_id>** and **<image_digest_id>** with the actual path names.

Example output

```
1 test-namespace my-pod
```

18.6. ADDITIONAL RESOURCES

- Pod Security Admission (Kubernetes documentation)
- Pod Security Standards (Kubernetes documentation)
- Audit Annotations (Kubernetes documentation)
- Viewing audit logs
- Managing security context constraints

CHAPTER 19. IMPERSONATING THE SYSTEM:ADMIN USER

You can configure API requests to impersonate users or groups to test permissions and troubleshoot access issues in OpenShift Container Platform.

19.1. API IMPERSONATION

You can configure API requests in OpenShift Container Platform to act as another user. Impersonation allows you to perform actions on behalf of another account without switching credentials.

19.2. IMPERSONATING THE SYSTEM:ADMIN USER

You can use the OpenShift web console to impersonate a user and select multiple group memberships at the same time to reproduce that user's effective permissions.

Procedure

- To grant a user permission to impersonate **system:admin**, run the following command:

```
$ oc create clusterrolebinding <any_valid_name> --clusterrole=sudoer --user=<username>
```

TIP

You can alternatively apply the following YAML to grant permission to impersonate **system:admin**:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: <any_valid_name>
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: sudoer
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: <username>
```

19.3. IMPERSONATING THE SYSTEM:ADMIN GROUP

To impersonate a user who has cluster administration privileges through group membership, you must specify both the user and the associated groups in the impersonation command.

Procedure

- To grant a user permission to impersonate a **system:admin** by impersonating the associated cluster administration groups, run the following command:

```
$ oc create clusterrolebinding <any_valid_name> --clusterrole=sudoer --as=<user> \
--as-group=<group1> --as-group=<group2>
```

19.4. IMPERSONATING A USER WITH MULTIPLE GROUP MEMBERSHIPS IN THE WEB CONSOLE

You can start user impersonation from multiple locations in the OpenShift Container Platform Console. Depending on where you start, you can impersonate a single user, a single group, or a user with one or more group memberships.

Prerequisites

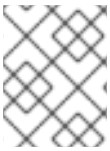
- You must be logged in to the OpenShift Container Platform web console as a user with permission to impersonate other users.
- The user or group that you want to impersonate must already exist.

NOTE

The impersonated user can belong to zero or more groups.

Procedure

1. From the **Overview** page in the OpenShift Container Platform console, click your user name and select **Impersonate User**.
2. In the **Username** field in the **Impersonate** dialog, enter the name of the user you want to impersonate.
3. Optional: In the **Groups** field, choose one or more groups that are associated with the user. The dialog displays a warning message explaining that impersonation applies the effective permissions of the specified user and any selected groups.
4. Click **Impersonate** to impersonate your selected user, groups, or both.



NOTE

Selecting one group uses the existing single-group impersonation behavior. Selecting no groups uses regular single-user impersonation.

19.5. STARTING IMPERSONATION FROM THE USERS OR GROUPS PAGES

You can start impersonation for users or groups from the **Users** or **Groups** pages in the OpenShift Container Platform Console.

Procedure

1. From the **Overview** page in the OpenShift Container Platform console, click **User Management → Users**.
2. Open the menu for the user you want to impersonate and select **Impersonate User**.
3. Optional: To impersonate a group, click **User Management → Groups**, click the menu for that group, and select **Impersonate Group**.

19.6. STOPPING IMPERSONATION

You can stop impersonating a user or group at any time from the OpenShift Container Platform Console.

Procedure

1. On any page in the OpenShift Container Platform console, click **Stop impersonating** at the top of the page.
2. Alternatively, click your user name and select **Stop impersonating**.

19.7. ADDING UNAUTHENTICATED GROUPS TO CLUSTER ROLES

Grant unauthenticated users access to specific cluster roles to enable features that require cluster access without authentication, such as external webhooks or automated token management.

You can add unauthenticated users to the following cluster roles:

- **system:scope-impersonation**
- **system:webhook**
- **system:oauth-token-deleter**
- **self-access-reviewer**

IMPORTANT

Always verify compliance with your organization's security standards when modifying unauthenticated access.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Create a YAML file named **add-<cluster_role>-unauth.yaml** and add the following content:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 annotations:
   rbac.authorization.kubernetes.io/autoupdate: "true"
 name: <cluster_role>access-unauthenticated
roleRef:
 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: <cluster_role>
subjects:
```

```
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: system:unauthenticated
```

2. Apply the configuration by running the following command:

```
$ oc apply -f add-<cluster_role>.yaml
```

19.8. ADDITIONAL RESOURCES

- User impersonation (Kubernetes documentation)

CHAPTER 20. SYNCING LDAP GROUPS

As an administrator, you can use groups to manage users, change their permissions, and enhance collaboration. Your organization may have already created user groups and stored them in an LDAP server. OpenShift Container Platform can sync those LDAP records with internal OpenShift Container Platform records, enabling you to manage your groups in one place. OpenShift Container Platform currently supports group sync with LDAP servers using three common schemas for defining group membership: RFC 2307, Active Directory, and augmented Active Directory.

For more information on configuring LDAP, see Configuring an LDAP identity provider.

NOTE

You must have **cluster-admin** privileges to sync groups.

20.1. ABOUT CONFIGURING LDAP SYNC

Before you can run LDAP sync, you need a sync configuration file. This file contains the following LDAP client configuration details:

- Configuration for connecting to your LDAP server.
- Sync configuration options that are dependent on the schema used in your LDAP server.
- An administrator-defined list of name mappings that maps OpenShift Container Platform group names to groups in your LDAP server.

The format of the configuration file depends upon the schema you are using: RFC 2307, Active Directory, or augmented Active Directory.

LDAP client configuration

The LDAP client configuration section of the configuration defines the connections to your LDAP server.

The LDAP client configuration section of the configuration defines the connections to your LDAP server.

LDAP client configuration

```
url: ldap://10.0.0.0:389 (1)
bindDN: cn=admin,dc=example,dc=com (2)
bindPassword: <password> (3)
insecure: false (4)
ca: my-ldap-ca-bundle.crt (5)
```

(1) (1) The connection protocol, IP address of the LDAP server hosting your database, and the port to connect to, formatted as **scheme://host:port**.

(2) Optional distinguished name (DN) to use as the Bind DN. OpenShift Container Platform uses this if elevated privilege is required to retrieve entries for the sync operation.

(3) Optional password to use to bind. OpenShift Container Platform uses this if elevated privilege is necessary to retrieve entries for the sync operation. This value may also be provided in an environment variable, external file, or encrypted file.

4 When **false**, secure LDAP (**ldaps://**) URLs connect using TLS, and insecure LDAP (**ldap://**) URLs are upgraded to TLS. When **true**, no TLS connection is made to the server and you cannot use

5 The certificate bundle to use for validating server certificates for the configured URL. If empty, OpenShift Container Platform uses system-trusted roots. This only applies if **insecure** is set to **false**.

LDAP query definition

Sync configurations consist of LDAP query definitions for the entries that are required for synchronization. The specific definition of an LDAP query depends on the schema used to store membership information in the LDAP server.

LDAP query definition

```
baseDN: ou=users,dc=example,dc=com 1
scope: sub 2
derefAliases: never 3
timeout: 0 4
filter: (objectClass=person) 5
pageSize: 0 6
```

1 The distinguished name (DN) of the branch of the directory where all searches will start from. It is required that you specify the top of your directory tree, but you can also specify a subtree in the directory.

2 The scope of the search. Valid values are **base**, **one**, or **sub**. If this is left undefined, then a scope of **sub** is assumed. Descriptions of the scope options can be found in the table below.

3 The behavior of the search with respect to aliases in the LDAP tree. Valid values are **never**, **search**, **base**, or **always**. If this is left undefined, then the default is to **always** dereference aliases. Descriptions of the dereferencing behaviors can be found in the table below.

4 The time limit allowed for the search by the client, in seconds. A value of **0** imposes no client-side limit.

5 A valid LDAP search filter. If this is left undefined, then the default is **(objectClass=*)**.

6 The optional maximum size of response pages from the server, measured in LDAP entries. If set to **0**, no size restrictions will be made on pages of responses. Setting paging sizes is necessary when queries return more entries than the client or server allow by default.

Table 20.1. LDAP search scope options

LDAP search scope	Description
base	Only consider the object specified by the base DN given for the query.
one	Consider all of the objects on the same level in the tree as the base DN for the query.

LDAP search scope	Description
sub	Consider the entire subtree rooted at the base DN given for the query.

Table 20.2. LDAP dereferencing behaviors

Dereferencing behavior	Description
never	Never dereference any aliases found in the LDAP tree.
search	Only dereference aliases found while searching.
base	Only dereference aliases while finding the base object.
always	Always dereference all aliases found in the LDAP tree.

User-defined name mapping

A user-defined name mapping explicitly maps the names of OpenShift Container Platform groups to unique identifiers that find groups on your LDAP server. The mapping uses normal YAML syntax. A user-defined mapping can contain an entry for every group in your LDAP server or only a subset of those groups. If there are groups on the LDAP server that do not have a user-defined name mapping, the default behavior during sync is to use the attribute specified as the OpenShift Container Platform group's name.

User-defined name mapping

```
groupUIDNameMapping:
  "cn=group1,ou=groups,dc=example,dc=com": firstgroup
  "cn=group2,ou=groups,dc=example,dc=com": secondgroup
  "cn=group3,ou=groups,dc=example,dc=com": thirdgroup
```

20.1.1. About the RFC 2307 configuration file

The RFC 2307 schema requires you to provide an LDAP query definition for both user and group entries, as well as the attributes with which to represent them in the internal OpenShift Container Platform records.

For clarity, the group you create in OpenShift Container Platform should use attributes other than the distinguished name whenever possible for user- or administrator-facing fields. For example, identify the users of an OpenShift Container Platform group by their e-mail, and use the name of the group as the common name. The following configuration file creates these relationships:

NOTE

If using user-defined name mappings, your configuration file will differ.

LDAP sync configuration that uses RFC 2307 schema: **rfc2307_config.yaml**

```
kind: LDAPSyncConfig
apiVersion: v1
url: ldap://LDAP_SERVICE_IP:389 (1)
insecure: false (2)
bindDN: cn=admin,dc=example,dc=com
bindPassword:
  file: "/etc/secrets/bindPassword"
rfc2307:
    groupsQuery:
        baseDN: "ou=groups,dc=example,dc=com"
        scope: sub
        derefAliases: never
        pageSize: 0
    groupUIDAttribute: dn (3)
    groupNameAttributes: [ cn ] (4)
    groupMembershipAttributes: [ member ] (5)
    usersQuery:
        baseDN: "ou=users,dc=example,dc=com"
        scope: sub
        derefAliases: never
        pageSize: 0
    userUIDAttribute: dn (6)
    userNameAttributes: [ mail ] (7)
    tolerateMemberNotFoundErrors: false
    tolerateMemberOutOfScopeErrors: false
```

(1) The IP address and host of the LDAP server where this group's record is stored.

(2) When **false**, secure LDAP (**ldaps://**) URLs connect using TLS, and insecure LDAP (**ldap://**) URLs are upgraded to TLS. When **true**, no TLS connection is made to the server and you cannot use **ldaps://** URL schemes.

(3) The attribute that uniquely identifies a group on the LDAP server. You cannot specify **groupsQuery** filters when using DN for **groupUIDAttribute**. For fine-grained filtering, use the whitelist / blacklist method.

(4) The attribute to use as the name of the group.

(5) The attribute on the group that stores the membership information.

(6) The attribute that uniquely identifies a user on the LDAP server. You cannot specify **usersQuery** filters when using DN for userUIDAttribute. For fine-grained filtering, use the whitelist / blacklist method.

(7) The attribute to use as the name of the user in the OpenShift Container Platform group record.

20.1.2. About the Active Directory configuration file

The Active Directory schema requires you to provide an LDAP query definition for user entries, as well as the attributes to represent them with in the internal OpenShift Container Platform group records.

For clarity, the group you create in OpenShift Container Platform should use attributes other than the distinguished name whenever possible for user- or administrator-facing fields. For example, identify the

users of an OpenShift Container Platform group by their e-mail, but define the name of the group by the name of the group on the LDAP server. The following configuration file creates these relationships:

LDAP sync configuration that uses Active Directory schema: active_directory_config.yaml

```
kind: LDAPSyncConfig
apiVersion: v1
url: ldap://LDAP_SERVICE_IP:389
activeDirectory:
    usersQuery:
        baseDN: "ou=users,dc=example,dc=com"
        scope: sub
        derefAliases: never
        filter: (objectclass=person)
        pageSize: 0
    userNameAttributes: [ mail ] 1
    groupMembershipAttributes: [ memberOf ] 2
```

1 The attribute to use as the name of the user in the OpenShift Container Platform group record.

2 The attribute on the user that stores the membership information.

20.1.3. About the augmented Active Directory configuration file

The augmented Active Directory schema requires you to provide an LDAP query definition for both user entries and group entries, as well as the attributes with which to represent them in the internal OpenShift Container Platform group records.

For clarity, the group you create in OpenShift Container Platform should use attributes other than the distinguished name whenever possible for user- or administrator-facing fields. For example, identify the users of an OpenShift Container Platform group by their e-mail, and use the name of the group as the common name. The following configuration file creates these relationships.

LDAP sync configuration that uses augmented Active Directory schema: augmented_active_directory_config.yaml

```
kind: LDAPSyncConfig
apiVersion: v1
url: ldap://LDAP_SERVICE_IP:389
augmentedActiveDirectory:
    groupsQuery:
        baseDN: "ou=groups,dc=example,dc=com"
        scope: sub
        derefAliases: never
        pageSize: 0
    groupUIDAttribute: dn 1
    groupNameAttributes: [ cn ] 2
    usersQuery:
        baseDN: "ou=users,dc=example,dc=com"
        scope: sub
        derefAliases: never
        filter: (objectclass=person)
```

```
    pageSize: 0
userNameAttributes: [ mail ] 3
groupMembershipAttributes: [ memberOf ] 4
```

1. The attribute that uniquely identifies a group on the LDAP server. You cannot specify **groupsQuery** filters when using DN for groupUIDAttribute. For fine-grained filtering, use the whitelist / blacklist method.
2. The attribute to use as the name of the group.
3. The attribute to use as the name of the user in the OpenShift Container Platform group record.
4. The attribute on the user that stores the membership information.

20.2. RUNNING LDAP SYNC

Once you have created a sync configuration file, you can begin to sync. OpenShift Container Platform allows administrators to perform a number of different sync types with the same server.

20.2.1. Syncing the LDAP server with OpenShift Container Platform

You can sync all groups from the LDAP server with OpenShift Container Platform.

Prerequisites

- Create a sync configuration file.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- To sync all groups from the LDAP server with OpenShift Container Platform:

```
$ oc adm groups sync --sync-config=config.yaml --confirm
```

NOTE

By default, all group synchronization operations are dry-run, so you must set the **--confirm** flag on the **oc adm groups sync** command to make changes to OpenShift Container Platform group records.

20.2.2. Syncing OpenShift Container Platform groups with the LDAP server

You can sync all groups already in OpenShift Container Platform that correspond to groups in the LDAP server specified in the configuration file.

Prerequisites

- Create a sync configuration file.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- To sync OpenShift Container Platform groups with the LDAP server:

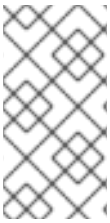
```
$ oc adm groups sync --type=openshift --sync-config=config.yaml --confirm
```

NOTE

By default, all group synchronization operations are dry-run, so you must set the **--confirm** flag on the **oc adm groups sync** command to make changes to OpenShift Container Platform group records.

20.2.3. Syncing subgroups from the LDAP server with OpenShift Container Platform

You can sync a subset of LDAP groups with OpenShift Container Platform using whitelist files, blacklist files, or both.



NOTE

You can use any combination of blacklist files, whitelist files, or whitelist literals. Whitelist and blacklist files must contain one unique group identifier per line, and you can include whitelist literals directly in the command itself. These guidelines apply to groups found on LDAP servers as well as groups already present in OpenShift Container Platform.

Prerequisites

- Create a sync configuration file.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- To sync a subset of LDAP groups with OpenShift Container Platform, use any the following commands:

```
$ oc adm groups sync --whitelist=<whitelist_file> \
                     --sync-config=config.yaml      \
                     --confirm
```

```
$ oc adm groups sync --blacklist=<blacklist_file> \
                     --sync-config=config.yaml      \
                     --confirm
```

```
$ oc adm groups sync <group_unique_identifier>    \
                     --sync-config=config.yaml      \
                     --confirm
```

```
$ oc adm groups sync <group_unique_identifier>  \
                     --whitelist=<whitelist_file> \
                     --blacklist=<blacklist_file> \
                     --sync-config=config.yaml    \
                     --confirm
```

```
$ oc adm groups sync --type=openshift           \
                --whitelist=<whitelist_file> \
                --sync-config=config.yaml    \
                --confirm
```

NOTE

By default, all group synchronization operations are dry-run, so you must set the **--confirm** flag on the **oc adm groups sync** command to make changes to OpenShift Container Platform group records.

20.3. RUNNING A GROUP PRUNING JOB

An administrator can also choose to remove groups from OpenShift Container Platform records if the records on the LDAP server that created them are no longer present. The prune job will accept the same sync configuration file and whitelists or blacklists as used for the sync job.

For example:

```
$ oc adm prune groups --sync-config=/path/to/ldap-sync-config.yaml --confirm
```

```
$ oc adm prune groups --whitelist=/path/to/whitelist.txt --sync-config=/path/to/ldap-sync-config.yaml --confirm
```

```
$ oc adm prune groups --blacklist=/path/to/blacklist.txt --sync-config=/path/to/ldap-sync-config.yaml --confirm
```

20.4. AUTOMATICALLY SYNCING LDAP GROUPS

You can automatically sync LDAP groups on a periodic basis by configuring a cron job.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have configured an LDAP identity provider (IDP).
  This procedure assumes that you created an LDAP secret named **ldap-secret** and a config map named **ca-config-map**.

Procedure

1. Create a project where the cron job will run:

```
$ oc new-project ldap-sync 1
```

1 This procedure uses a project called **ldap-sync**.

2. Locate the secret and config map that you created when configuring the LDAP identity provider and copy them to this new project.

The secret and config map exist in the **openshift-config** project and must be copied to the new **ldap-sync** project.

3. Define a service account:

Example ldap-sync-service-account.yaml

```
kind: ServiceAccount
apiVersion: v1
metadata:
  name: ldap-group-syncer
  namespace: ldap-sync
```

4. Create the service account:

```
$ oc create -f ldap-sync-service-account.yaml
```

5. Define a cluster role:

Example ldap-sync-cluster-role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: ldap-group-syncer
rules:
  - apiGroups:
      - user.openshift.io
    resources:
      - groups
    verbs:
      - get
      - list
      - create
      - update
```

6. Create the cluster role:

```
$ oc create -f ldap-sync-cluster-role.yaml
```

7. Define a cluster role binding to bind the cluster role to the service account:

Example ldap-sync-cluster-role-binding.yaml

```
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: ldap-group-syncer
subjects:
  - kind: ServiceAccount
    name: ldap-group-syncer                 (1)
    namespace: ldap-sync
roleRef:
```

```
apiGroup: rbac.authorization.k8s.io
kind: ClusterRole
name: ldap-group-syncer                    2
```

1. Reference to the service account created earlier in this procedure.
2. Reference to the cluster role created earlier in this procedure.

8. Create the cluster role binding:

```
$ oc create -f ldap-sync-cluster-role-binding.yaml
```

9. Define a config map that specifies the sync configuration file:

Example ldap-sync-config-map.yaml

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: ldap-group-syncer
  namespace: ldap-sync
data:
  sync.yaml: |                             1
    kind: LDAPSyncConfig
    apiVersion: v1
    url: ldaps://10.0.0.0:636              2
    insecure: false
    bindDN: cn=admin,dc=example,dc=com     3
    bindPassword:
      file: "/etc/secrets/bindPassword"
    ca: /etc/ldap-ca/ca.crt
    rfc2307:                               4
      groupsQuery:
        baseDN: "ou=groups,dc=example,dc=com"  5
        scope: sub
        filter: "(objectClass=groupOfMembers)"
        derefAliases: never
        pageSize: 0
      groupUIDAttribute: dn
      groupNameAttributes: [ cn ]
      groupMembershipAttributes: [ member ]
      usersQuery:
        baseDN: "ou=users,dc=example,dc=com"   6
        scope: sub
        derefAliases: never
        pageSize: 0
      userUIDAttribute: dn
      userNameAttributes: [ uid ]
      tolerateMemberNotFoundErrors: false
      tolerateMemberOutOfScopeErrors: false
```

1. Define the sync configuration file.

2 Specify the URL.

3 Specify the **bindDN**.

4 This example uses the RFC2307 schema; adjust values as necessary. You can also use a different schema.

5 Specify the **baseDN** for **groupsQuery**.

6 Specify the **baseDN** for **usersQuery**.

10. Create the config map:

```
$ oc create -f ldap-sync-config-map.yaml
```

11. Define a cron job:

Example ldap-sync-cron-job.yaml

```
kind: CronJob
apiVersion: batch/v1
metadata:
  name: ldap-group-syncer
  namespace: ldap-sync
spec:                                                                    1
  schedule: "*/30 * * * *"                                                 2
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      backoffLimit: 0
      ttlSecondsAfterFinished: 1800                                        3
      template:
        spec:
          containers:
            - name: ldap-group-sync
              image: "registry.redhat.io/openshift4/ose-cli:latest"
              command:
                - "/bin/bash"
                - "-c"
                - "oc adm groups sync --sync-config=/etc/config/sync.yaml --confirm"   4
              volumeMounts:
                - mountPath: "/etc/config"
                  name: "ldap-sync-volume"
                - mountPath: "/etc/secrets"
                  name: "ldap-bind-password"
                - mountPath: "/etc/ldap-ca"
                  name: "ldap-ca"
          volumes:
            - name: "ldap-sync-volume"
              configMap:
                name: "ldap-group-syncer"
            - name: "ldap-bind-password"
              secret:
                secretName: "ldap-secret"                                5
```

```
      - name: "ldap-ca"
        configMap:
          name: "ca-config-map"                                    6
    restartPolicy: "Never"
    terminationGracePeriodSeconds: 30
    activeDeadlineSeconds: 500
    dnsPolicy: "ClusterFirst"
    serviceAccountName: "ldap-group-syncer"
```

1. Configure the settings for the cron job. See "Creating cron jobs" for more information on cron job settings.
2. The schedule for the job specified in cron format. This example cron job runs every 30 minutes. Adjust the frequency as necessary, making sure to take into account how long the sync takes to run.
3. How long, in seconds, to keep finished jobs. This should match the period of the job schedule in order to clean old failed jobs and prevent unnecessary alerts. For more information, see TTL-after-finished Controller in the Kubernetes documentation.
4. The LDAP sync command for the cron job to run. Passes in the sync configuration file that was defined in the config map.
5. This secret was created when the LDAP IDP was configured.
6. This config map was created when the LDAP IDP was configured.

12. Create the cron job:

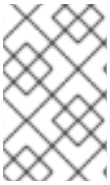
```
$ oc create -f ldap-sync-cron-job.yaml
```

Additional resources

- Configuring an LDAP identity provider
- Creating cron jobs

20.5. LDAP GROUP SYNC EXAMPLES

This section contains examples for the RFC 2307, Active Directory, and augmented Active Directory schemas.



NOTE

These examples assume that all users are direct members of their respective groups. Specifically, no groups have other groups as members. See the Nested Membership Sync Example for information on how to sync nested groups.

20.5.1. Syncing groups using the RFC 2307 schema

For the RFC 2307 schema, the following examples synchronize a group named **admins** that has two members: **Jane** and **Jim**. The examples explain:

- How the group and users are added to the LDAP server.
- What the resulting group record in OpenShift Container Platform will be after synchronization.

NOTE

These examples assume that all users are direct members of their respective groups. Specifically, no groups have other groups as members. See the Nested Membership Sync Example for information on how to sync nested groups.

In the RFC 2307 schema, both users (Jane and Jim) and groups exist on the LDAP server as first-class entries, and group membership is stored in attributes on the group. The following snippet of **ldif** defines the users and group for this schema:

LDAP entries that use RFC 2307 schema: rfc2307.ldif

```
dn: ou=users,dc=example,dc=com
objectClass: organizationalUnit
ou: users
dn: cn=Jane,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
cn: Jane
sn: Smith
displayName: Jane Smith
mail: jane.smith@example.com
dn: cn=Jim,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
cn: Jim
sn: Adams
displayName: Jim Adams
mail: jim.adams@example.com
dn: ou=groups,dc=example,dc=com
objectClass: organizationalUnit
ou: groups
dn: cn=admins,ou=groups,dc=example,dc=com (1)
objectClass: groupOfNames
cn: admins
owner: cn=admin,dc=example,dc=com
description: System Administrators
member: cn=Jane,ou=users,dc=example,dc=com (2)
member: cn=Jim,ou=users,dc=example,dc=com
```

1. The group is a first-class entry in the LDAP server.
2. Members of a group are listed with an identifying reference as attributes on the group.

Prerequisites

- Create the configuration file.

- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Run the sync with the **rfc2307_config.yaml** file:

```
$ oc adm groups sync --sync-config=rfc2307_config.yaml --confirm
```

OpenShift Container Platform creates the following group record as a result of the above sync operation:

OpenShift Container Platform group created by using the rfc2307_config.yaml file

```
apiVersion: user.openshift.io/v1
kind: Group
metadata:
  annotations:
    openshift.io/ldap.sync-time: 2015-10-13T10:08:38-0400 (1)
    openshift.io/ldap.uid: cn=admins,ou=groups,dc=example,dc=com (2)
    openshift.io/ldap.url: LDAP_SERVER_IP:389 (3)
  creationTimestamp:
  name: admins (4)
users: (5)
- jane.smith@example.com
- jim.adams@example.com
```

(1) The last time this OpenShift Container Platform group was synchronized with the LDAP server, in ISO 6801 format.

(2) The unique identifier for the group on the LDAP server.

(3) The IP address and host of the LDAP server where this group's record is stored.

(4) The name of the group as specified by the sync file.

(5) The users that are members of the group, named as specified by the sync file.

20.5.2. Syncing groups using the RFC2307 schema with user-defined name mappings

When syncing groups with user-defined name mappings, the configuration file changes to contain these mappings as shown below.

LDAP sync configuration that uses RFC 2307 schema with user-defined name mappings: rfc2307_config_user_defined.yaml

```
kind: LDAPSyncConfig
apiVersion: v1
groupUIDNameMapping:
  "cn=admins,ou=groups,dc=example,dc=com": Administrators (1)
rfc2307:
    groupsQuery:
```

```
    baseDN: "ou=groups,dc=example,dc=com"
    scope: sub
    derefAliases: never
    pageSize: 0
groupUIDAttribute: dn 2
groupNameAttributes: [ cn ] 3
groupMembershipAttributes: [ member ]
usersQuery:
    baseDN: "ou=users,dc=example,dc=com"
    scope: sub
    derefAliases: never
    pageSize: 0
userUIDAttribute: dn 4
userNameAttributes: [ mail ]
tolerateMemberNotFoundErrors: false
tolerateMemberOutOfScopeErrors: false
```

1. The user-defined name mapping.
2. The unique identifier attribute that is used for the keys in the user-defined name mapping. You cannot specify **groupsQuery** filters when using DN for groupUIDAttribute. For fine-grained filtering, use the whitelist / blacklist method.
3. The attribute to name OpenShift Container Platform groups with if their unique identifier is not in the user-defined name mapping.
4. The attribute that uniquely identifies a user on the LDAP server. You cannot specify **usersQuery** filters when using DN for userUIDAttribute. For fine-grained filtering, use the whitelist / blacklist method.

Prerequisites

- Create the configuration file.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Run the sync with the **rfc2307_config_user_defined.yaml** file:

```
$ oc adm groups sync --sync-config=rfc2307_config_user_defined.yaml --confirm
```

OpenShift Container Platform creates the following group record as a result of the above sync operation:

OpenShift Container Platform group created by using the rfc2307_config_user_defined.yaml file

```
apiVersion: user.openshift.io/v1
kind: Group
metadata:
  annotations:
    openshift.io/ldap.sync-time: 2015-10-13T10:08:38-0400
    openshift.io/ldap.uid: cn=admins,ou=groups,dc=example,dc=com
```

```
    openshift.io/ldap.url: LDAP_SERVER_IP:389
  creationTimestamp:
  name: Administrators 1
users:
- jane.smith@example.com
- jim.adams@example.com
```

1 The name of the group as specified by the user-defined name mapping.

20.5.3. Syncing groups using RFC 2307 with user-defined error tolerances

By default, if the groups being synced contain members whose entries are outside of the scope defined in the member query, the group sync fails with an error:

```
Error determining LDAP group membership for "<group>": membership lookup for user "<user>" in group "<group>" failed because of "search for entry with dn="<user-dn>" would search outside of the base dn specified (dn="<base-dn>")".
```

This often indicates a misconfigured **baseDN** in the **usersQuery** field. However, in cases where the **baseDN** intentionally does not contain some of the members of the group, setting **tolerateMemberOutOfScopeErrors: true** allows the group sync to continue. Out of scope members will be ignored.

Similarly, when the group sync process fails to locate a member for a group, it fails outright with errors:

```
Error determining LDAP group membership for "<group>": membership lookup for user "<user>" in group "<group>" failed because of "search for entry with base dn="<user-dn>" refers to a non-existent entry".
Error determining LDAP group membership for "<group>": membership lookup for user "<user>" in group "<group>" failed because of "search for entry with base dn="<user-dn>" and filter "<filter>" did not return any results".
```

This often indicates a misconfigured **usersQuery** field. However, in cases where the group contains member entries that are known to be missing, setting **tolerateMemberNotFoundErrors: true** allows the group sync to continue. Problematic members will be ignored.

WARNING

Enabling error tolerances for the LDAP group sync causes the sync process to ignore problematic member entries. If the LDAP group sync is not configured correctly, this could result in synced OpenShift Container Platform groups missing members.

LDAP entries that use RFC 2307 schema with problematic group membership: rfc2307_problematic_users.ldif

```
dn: ou=users,dc=example,dc=com
objectClass: organizationalUnit
```

```
ou: users
dn: cn=Jane,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
cn: Jane
sn: Smith
displayName: Jane Smith
mail: jane.smith@example.com
dn: cn=Jim,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
cn: Jim
sn: Adams
displayName: Jim Adams
mail: jim.adams@example.com
dn: ou=groups,dc=example,dc=com
objectClass: organizationalUnit
ou: groups
dn: cn=admins,ou=groups,dc=example,dc=com
objectClass: groupOfNames
cn: admins
owner: cn=admin,dc=example,dc=com
description: System Administrators
member: cn=Jane,ou=users,dc=example,dc=com
member: cn=Jim,ou=users,dc=example,dc=com
member: cn=INVALID,ou=users,dc=example,dc=com (1)
member: cn=Jim,ou=OUTOFSCOPE,dc=example,dc=com (2)
```

1. A member that does not exist on the LDAP server.
2. A member that may exist, but is not under the **baseDN** in the user query for the sync job.

To tolerate the errors in the above example, the following additions to your sync configuration file must be made:

LDAP sync configuration that uses RFC 2307 schema tolerating errors: **rfc2307_config_tolerating.yaml**

```
kind: LDAPSyncConfig
apiVersion: v1
url: ldap://LDAP_SERVICE_IP:389
rfc2307:
    groupsQuery:
        baseDN: "ou=groups,dc=example,dc=com"
        scope: sub
        derefAliases: never
    groupUIDAttribute: dn
    groupNameAttributes: [ cn ]
    groupMembershipAttributes: [ member ]
    usersQuery:
        baseDN: "ou=users,dc=example,dc=com"
        scope: sub
```

```
    derefAliases: never
  userUIDAttribute: dn (1)
  userNameAttributes: [ mail ]
  tolerateMemberNotFoundErrors: true (2)
  tolerateMemberOutOfScopeErrors: true (3)
```

1. The attribute that uniquely identifies a user on the LDAP server. You cannot specify **usersQuery** filters when using DN for userUIDAttribute. For fine-grained filtering, use the whitelist / blacklist method.
2. When **true**, the sync job tolerates groups for which some members were not found, and members whose LDAP entries are not found are ignored. The default behavior for the sync job is to fail if a member of a group is not found.
3. When **true**, the sync job tolerates groups for which some members are outside the user scope given in the **usersQuery** base DN, and members outside the member query scope are ignored. The default behavior for the sync job is to fail if a member of a group is out of scope.

Prerequisites

- Create the configuration file.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Run the sync with the **rfc2307_config_tolerating.yaml** file:

```
$ oc adm groups sync --sync-config=rfc2307_config_tolerating.yaml --confirm
```

OpenShift Container Platform creates the following group record as a result of the above sync operation:

OpenShift Container Platform group created by using the rfc2307_config.yaml file

```
apiVersion: user.openshift.io/v1
kind: Group
metadata:
  annotations:
    openshift.io/ldap.sync-time: 2015-10-13T10:08:38-0400
    openshift.io/ldap.uid: cn=admins,ou=groups,dc=example,dc=com
    openshift.io/ldap.url: LDAP_SERVER_IP:389
  creationTimestamp:
  name: admins
users: (1)
- jane.smith@example.com
- jim.adams@example.com
```

1. The users that are members of the group, as specified by the sync file. Members for which lookup encountered tolerated errors are absent.

20.5.4. Syncing groups using the Active Directory schema

In the Active Directory schema, both users (Jane and Jim) exist in the LDAP server as first-class entries, and group membership is stored in attributes on the user. The following snippet of **ldif** defines the users and group for this schema:

LDAP entries that use Active Directory schema: active_directory.ldif

```
dn: ou=users,dc=example,dc=com
objectClass: organizationalUnit
ou: users

dn: cn=Jane,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
objectClass: testPerson
cn: Jane
sn: Smith
displayName: Jane Smith
mail: jane.smith@example.com
memberOf: admins (1)

dn: cn=Jim,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
objectClass: testPerson
cn: Jim
sn: Adams
displayName: Jim Adams
mail: jim.adams@example.com
memberOf: admins
```

(1) The user's group memberships are listed as attributes on the user, and the group does not exist as an entry on the server. The **memberOf** attribute does not have to be a literal attribute on the user; in some LDAP servers, it is created during search and returned to the client, but not committed to the database.

Prerequisites

- Create the configuration file.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Run the sync with the **active_directory_config.yaml** file:

```
$ oc adm groups sync --sync-config=active_directory_config.yaml --confirm
```

OpenShift Container Platform creates the following group record as a result of the above sync operation:

OpenShift Container Platform group created by using the active_directory_config.yaml file

```
apiVersion: user.openshift.io/v1
kind: Group
metadata:
  annotations:
    openshift.io/ldap.sync-time: 2015-10-13T10:08:38-0400 (1)
    openshift.io/ldap.uid: admins (2)
    openshift.io/ldap.url: LDAP_SERVER_IP:389 (3)
  creationTimestamp:
  name: admins (4)
users: (5)
- jane.smith@example.com
- jim.adams@example.com
```

1. The last time this OpenShift Container Platform group was synchronized with the LDAP server, in ISO 6801 format.
2. The unique identifier for the group on the LDAP server.
3. The IP address and host of the LDAP server where this group's record is stored.
4. The name of the group as listed in the LDAP server.
5. The users that are members of the group, named as specified by the sync file.

20.5.5. Syncing groups using the augmented Active Directory schema

In the augmented Active Directory schema, both users (Jane and Jim) and groups exist in the LDAP server as first-class entries, and group membership is stored in attributes on the user. The following snippet of **ldif** defines the users and group for this schema:

LDAP entries that use augmented Active Directory schema: augmented_active_directory.ldif

```
dn: ou=users,dc=example,dc=com
objectClass: organizationalUnit
ou: users

dn: cn=Jane,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
objectClass: testPerson
cn: Jane
sn: Smith
displayName: Jane Smith
mail: jane.smith@example.com
memberOf: cn=admins,ou=groups,dc=example,dc=com (1)

dn: cn=Jim,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
```

```
objectClass: inetOrgPerson
objectClass: testPerson
cn: Jim
sn: Adams
displayName: Jim Adams
mail: jim.adams@example.com
memberOf: cn=admins,ou=groups,dc=example,dc=com

dn: ou=groups,dc=example,dc=com
objectClass: organizationalUnit
ou: groups

dn: cn=admins,ou=groups,dc=example,dc=com (2)
objectClass: groupOfNames
cn: admins
owner: cn=admin,dc=example,dc=com
description: System Administrators
member: cn=Jane,ou=users,dc=example,dc=com
member: cn=Jim,ou=users,dc=example,dc=com
```

1. The user's group memberships are listed as attributes on the user.
2. The group is a first-class entry on the LDAP server.

Prerequisites

- Create the configuration file.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Run the sync with the **augmented_active_directory_config.yaml** file:

```
$ oc adm groups sync --sync-config=augmented_active_directory_config.yaml --confirm
```

OpenShift Container Platform creates the following group record as a result of the above sync operation:

OpenShift Container Platform group created by using the augmented_active_directory_config.yaml file

```
apiVersion: user.openshift.io/v1
kind: Group
metadata:
  annotations:
    openshift.io/ldap.sync-time: 2015-10-13T10:08:38-0400 (1)
    openshift.io/ldap.uid: cn=admins,ou=groups,dc=example,dc=com (2)
    openshift.io/ldap.url: LDAP_SERVER_IP:389 (3)
  creationTimestamp:
  name: admins (4)
```

```
users: 5
- jane.smith@example.com
- jim.adams@example.com
```

1. The last time this OpenShift Container Platform group was synchronized with the LDAP server, in ISO 6801 format.
2. The unique identifier for the group on the LDAP server.
3. The IP address and host of the LDAP server where this group's record is stored.
4. The name of the group as specified by the sync file.
5. The users that are members of the group, named as specified by the sync file.

20.5.5.1. LDAP nested membership sync example

Groups in OpenShift Container Platform do not nest. The LDAP server must flatten group membership before the data can be consumed. Microsoft's Active Directory Server supports this feature via the **LDAP_MATCHING_RULE_IN_CHAIN** rule, which has the OID **1.2.840.113556.1.4.1941**. Furthermore, only explicitly whitelisted groups can be synced when using this matching rule.

This section has an example for the augmented Active Directory schema, which synchronizes a group named **admins** that has one user **Jane** and one group **otheradmins** as members. The **otheradmins** group has one user member: **Jim**. This example explains:

- How the group and users are added to the LDAP server.
- What the LDAP sync configuration file looks like.
- What the resulting group record in OpenShift Container Platform will be after synchronization.

In the augmented Active Directory schema, both users (**Jane** and **Jim**) and groups exist in the LDAP server as first-class entries, and group membership is stored in attributes on the user or the group. The following snippet of **ldif** defines the users and groups for this schema:

LDAP entries that use augmented Active Directory schema with nested members: augmented_active_directory_nested.ldif

```
dn: ou=users,dc=example,dc=com
objectClass: organizationalUnit
ou: users

dn: cn=Jane,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
objectClass: testPerson
cn: Jane
sn: Smith
displayName: Jane Smith
mail: jane.smith@example.com
memberOf: cn=admins,ou=groups,dc=example,dc=com 1
```

```
dn: cn=Jim,ou=users,dc=example,dc=com
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
objectClass: testPerson
cn: Jim
sn: Adams
displayName: Jim Adams
mail: jim.adams@example.com
memberOf: cn=otheradmins,ou=groups,dc=example,dc=com (2)

dn: ou=groups,dc=example,dc=com
objectClass: organizationalUnit
ou: groups

dn: cn=admins,ou=groups,dc=example,dc=com (3)
objectClass: group
cn: admins
owner: cn=admin,dc=example,dc=com
description: System Administrators
member: cn=Jane,ou=users,dc=example,dc=com
member: cn=otheradmins,ou=groups,dc=example,dc=com

dn: cn=otheradmins,ou=groups,dc=example,dc=com (4)
objectClass: group
cn: otheradmins
owner: cn=admin,dc=example,dc=com
description: Other System Administrators
memberOf: cn=admins,ou=groups,dc=example,dc=com (5) (6)
member: cn=Jim,ou=users,dc=example,dc=com
```

(1) (2) (5) The user's and group's memberships are listed as attributes on the object.

(3) (4) The groups are first-class entries on the LDAP server.

(6) The **otheradmins** group is a member of the **admins** group.

When syncing nested groups with Active Directory, you must provide an LDAP query definition for both user entries and group entries, as well as the attributes with which to represent them in the internal OpenShift Container Platform group records. Furthermore, certain changes are required in this configuration:

- The **oc adm groups sync** command must explicitly whitelist groups.
- The user's **groupMembershipAttributes** must include **"memberOf:1.2.840.113556.1.4.1941:"** to comply with the **LDAP_MATCHING_RULE_IN_CHAIN** rule.
- The **groupUIDAttribute** must be set to **dn**.
- The **groupsQuery**:
  - Must not set **filter**.
  - Must set a valid **derefAliases**.

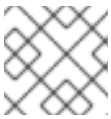
- Should not set **baseDN** as that value is ignored.
- Should not set **scope** as that value is ignored.

For clarity, the group you create in OpenShift Container Platform should use attributes other than the distinguished name whenever possible for user- or administrator-facing fields. For example, identify the users of an OpenShift Container Platform group by their e-mail, and use the name of the group as the common name. The following configuration file creates these relationships:

LDAP sync configuration that uses augmented Active Directory schema with nested members: augmented_active_directory_config_nested.yaml

```
kind: LDAPSyncConfig
apiVersion: v1
url: ldap://LDAP_SERVICE_IP:389
augmentedActiveDirectory:
  groupsQuery: 1
    derefAliases: never
    pageSize: 0
  groupUIDAttribute: dn 2
  groupNameAttributes: [ cn ] 3
  usersQuery:
    baseDN: "ou=users,dc=example,dc=com"
    scope: sub
    derefAliases: never
    filter: (objectclass=person)
    pageSize: 0
  userNameAttributes: [ mail ] 4
  groupMembershipAttributes: [ "memberOf:1.2.840.113556.1.4.1941:" ] 5
```

1. **groupsQuery** filters cannot be specified. The **groupsQuery** base DN and scope values are ignored. **groupsQuery** must set a valid **derefAliases**.
2. The attribute that uniquely identifies a group on the LDAP server. It must be set to **dn**.
3. The attribute to use as the name of the group.
4. The attribute to use as the name of the user in the OpenShift Container Platform group record.



NOTE

mail or **sAMAccountName** are preferred choices in most installations.

5. The attribute on the user that stores the membership information. Note the use of **LDAP_MATCHING_RULE_IN_CHAIN**.

Prerequisites

- Create the configuration file.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Run the sync with the **augmented_active_directory_config_nested.yaml** file:

```
$ oc adm groups sync \
    'cn=admins,ou=groups,dc=example,dc=com' \
    --sync-config=augmented_active_directory_config_nested.yaml \
    --confirm
```

NOTE

You must explicitly whitelist the **cn=admins,ou=groups,dc=example,dc=com** group.

OpenShift Container Platform creates the following group record as a result of the above sync operation:

OpenShift Container Platform group created by using the augmented_active_directory_config_nested.yaml file

```
apiVersion: user.openshift.io/v1
kind: Group
metadata:
  annotations:
    openshift.io/ldap.sync-time: 2015-10-13T10:08:38-0400 (1)
    openshift.io/ldap.uid: cn=admins,ou=groups,dc=example,dc=com (2)
    openshift.io/ldap.url: LDAP_SERVER_IP:389 (3)
  creationTimestamp:
  name: admins (4)
users: (5)
- jane.smith@example.com
- jim.adams@example.com
```

(1) The last time this OpenShift Container Platform group was synchronized with the LDAP server, in ISO 6801 format.

(2) The unique identifier for the group on the LDAP server.

(3) The IP address and host of the LDAP server where this group's record is stored.

(4) The name of the group as specified by the sync file.

(5) The users that are members of the group, named as specified by the sync file. Note that members of nested groups are included since the group membership was flattened by the Microsoft Active Directory Server.

20.6. LDAP SYNC CONFIGURATION SPECIFICATION

The object specification for the configuration file is below. Note that the different schema objects have different fields. For example, v1.ActiveDirectoryConfig has no **groupsQuery** field whereas v1.RFC2307Config and v1.AugmentedActiveDirectoryConfig both do.

IMPORTANT

There is no support for binary attributes. All attribute data coming from the LDAP server must be in the format of a UTF-8 encoded string. For example, never use a binary attribute, such as **objectGUID**, as an ID attribute. You must use string attributes, such as **sAMAccountName** or **userPrincipalName**, instead.

20.6.1. v1.LDAPSyncConfig

LDAPSyncConfig holds the necessary configuration options to define an LDAP group sync.

Name	Description	Schema
kind	String value representing the REST resource this object represents. Servers may infer this from the endpoint the client submits requests to. Cannot be updated. In CamelCase. More info: https://github.com/kubernetes/community/blob/master/contributors/devel/sig-architecture/api-conventions.md#types-kinds	string
apiVersion	Defines the versioned schema of this representation of an object. Servers should convert recognized schemas to the latest internal value, and may reject unrecognized values. More info: https://github.com/kubernetes/community/blob/master/contributors/devel/sig-architecture/api-conventions.md#resources	string
url	Host is the scheme, host and port of the LDAP server to connect to: **scheme://host:port**	string
bindDN	Optional DN to bind to the LDAP server with.	string
bindPassword	Optional password to bind with during the search phase.	v1.StringSource

Name	Description	Schema
insecure	If **true**, indicates the connection should not use TLS. If **false**, **ldaps://** URLs connect using TLS, and **ldap://** URLs are upgraded to a TLS connection using StartTLS as specified in https://tools.ietf.org/html/rfc2830. If you set **insecure** to **true**, you cannot use **ldaps://** URL schemes.	boolean
ca	Optional trusted certificate authority bundle to use when making requests to the server. If empty, the default system roots are used.	string
groupUIDNameMapping	Optional direct mapping of LDAP group UIDs to OpenShift Container Platform group names.	object
rfc2307	Holds the configuration for extracting data from an LDAP server set up in a fashion similar to RFC2307: first-class group and user entries, with group membership determined by a multi-valued attribute on the group entry listing its members.	v1.RFC2307Config
activeDirectory	Holds the configuration for extracting data from an LDAP server set up in a fashion similar to that used in Active Directory: first-class user entries, with group membership determined by a multi-valued attribute on members listing groups they are a member of.	v1.ActiveDirectoryConfig
augmentedActiveDirectory	Holds the configuration for extracting data from an LDAP server set up in a fashion similar to that used in Active Directory as described above, with one addition: first-class group entries exist and are used to hold metadata but not group membership.	v1.AugmentedActiveDirectoryConfig

20.6.2. v1.StringSource

StringSource allows specifying a string inline, or externally via environment variable or file. When it contains only a string value, it marshals to a simple JSON string.

Name	Description	Schema
value	Specifies the cleartext value, or an encrypted value if **keyFile** is specified.	string
env	Specifies an environment variable containing the cleartext value, or an encrypted value if the **keyFile** is specified.	string
file	References a file containing the cleartext value, or an encrypted value if a **keyFile** is specified.	string
keyFile	References a file containing the key to use to decrypt the value.	string

20.6.3. v1.LDAPQuery

LDAPQuery holds the options necessary to build an LDAP query.

Name	Description	Schema
baseDN	DN of the branch of the directory where all searches should start from.	string
scope	The optional scope of the search. Can be **base**: only the base object, **one**: all objects on the base level, **sub**: the entire subtree. Defaults to **sub** if not set.	string
derefAliases	The optional behavior of the search with regards to aliases. Can be **never**: never dereference aliases, **search**: only dereference in searching, **base**: only dereference in finding the base object, **always**: always dereference. Defaults to **always** if not set.	string

Name	Description	Schema
timeout	Holds the limit of time in seconds that any request to the server can remain outstanding before the wait for a response is given up. If this is **0**, no client-side limit is imposed.	integer
filter	A valid LDAP search filter that retrieves all relevant entries from the LDAP server with the base DN.	string
pageSize	Maximum preferred page size, measured in LDAP entries. A page size of **0** means no paging will be done.	integer

20.6.4. v1.RFC2307Config

RFC2307Config holds the necessary configuration options to define how an LDAP group sync interacts with an LDAP server using the RFC2307 schema.

Name	Description	Schema
groupsQuery	Holds the template for an LDAP query that returns group entries.	v1.LDAPQuery
groupUIDAttribute	Defines which attribute on an LDAP group entry will be interpreted as its unique identifier. (**ldapGroupUID**)	string
groupNameAttributes	Defines which attributes on an LDAP group entry will be interpreted as its name to use for an OpenShift Container Platform group.	string array
groupMembershipAttributes	Defines which attributes on an LDAP group entry will be interpreted as its members. The values contained in those attributes must be queryable by your **UserUIDAttribute**.	string array
usersQuery	Holds the template for an LDAP query that returns user entries.	v1.LDAPQuery

Name	Description	Schema
userUIDAttribute	Defines which attribute on an LDAP user entry will be interpreted as its unique identifier. It must correspond to values that will be found from the **GroupMembershipAttributes**.	string
userNameAttributes	Defines which attributes on an LDAP user entry will be used, in order, as its OpenShift Container Platform user name. The first attribute with a non-empty value is used. This should match your **PreferredUsername** setting for your **LDAPPasswordIdentityProvider**. The attribute to use as the name of the user in the OpenShift Container Platform group record. **mail** or **sAMAccountName** are preferred choices in most installations.	string array
tolerateMemberNotFoundErrors	Determines the behavior of the LDAP sync job when missing user entries are encountered. If **true**, an LDAP query for users that does not find any will be tolerated and an only and error will be logged. If **false**, the LDAP sync job will fail if a query for users does not find any. The default value is **false**. Misconfigured LDAP sync jobs with this flag set to **true** can cause group membership to be removed, so it is recommended to use this flag with caution.	boolean

Name	Description	Schema
tolerateMemberOutOfScopeErrors	Determines the behavior of the LDAP sync job when out-of-scope user entries are encountered. If **true**, an LDAP query for a user that falls outside of the base DN given for the all user query will be tolerated and only an error will be logged. If **false**, the LDAP sync job will fail if a user query would search outside of the base DN specified by the all user query. Misconfigured LDAP sync jobs with this flag set to **true** can result in groups missing users, so it is recommended to use this flag with caution.	boolean

20.6.5. v1.ActiveDirectoryConfig

ActiveDirectoryConfig holds the necessary configuration options to define how an LDAP group sync interacts with an LDAP server using the Active Directory schema.

Name	Description	Schema
usersQuery	Holds the template for an LDAP query that returns user entries.	v1.LDAPQuery
userNameAttributes	Defines which attributes on an LDAP user entry will be interpreted as its OpenShift Container Platform user name. The attribute to use as the name of the user in the OpenShift Container Platform group record. **mail** or **sAMAccountName** are preferred choices in most installations.	string array
groupMembershipAttributes	Defines which attributes on an LDAP user entry will be interpreted as the groups it is a member of.	string array

20.6.6. v1.AugmentedActiveDirectoryConfig

AugmentedActiveDirectoryConfig holds the necessary configuration options to define how an LDAP group sync interacts with an LDAP server using the augmented Active Directory schema.

Name	Description	Schema
usersQuery	Holds the template for an LDAP query that returns user entries.	v1.LDAPQuery
userNameAttributes	Defines which attributes on an LDAP user entry will be interpreted as its OpenShift Container Platform user name. The attribute to use as the name of the user in the OpenShift Container Platform group record. **mail** or **sAMAccountName** are preferred choices in most installations.	string array
groupMembershipAttributes	Defines which attributes on an LDAP user entry will be interpreted as the groups it is a member of.	string array
groupsQuery	Holds the template for an LDAP query that returns group entries.	v1.LDAPQuery
groupUIDAttribute	Defines which attribute on an LDAP group entry will be interpreted as its unique identifier. (**ldapGroupUID**)	string
groupNameAttributes	Defines which attributes on an LDAP group entry will be interpreted as its name to use for an OpenShift Container Platform group.	string array

CHAPTER 21. MANAGING CLOUD PROVIDER CREDENTIALS

21.1. ABOUT THE CLOUD CREDENTIAL OPERATOR

To allow OpenShift Container Platform components to request cloud provider credentials with the specific permissions that are required for the cluster to run, you can use the Cloud Credential Operator (CCO) to manage cloud provider credentials as custom resource definitions (CRDs).

You can configure the Cloud Credential Operator (CCO) to operate in several different modes. These options provide transparency and flexibility in how the CCO uses cloud credentials.

21.1.1. About Cloud Credential Operator modes

You can configure the Cloud Credential Operator (CCO) to operate in several different modes. These options provide transparency and flexibility in how the CCO uses cloud credentials to process **CredentialsRequest** CRs in the cluster to suit the security requirements of your organization.

By setting different values for the **credentialsMode** parameter in the **install-config.yaml** file, you can configure the CCO to operate in *mint*, *passthrough*, or *manual* mode.

- **Mint**: In mint mode, the CCO uses the provided admin-level cloud credential to create new credentials for components in the cluster with only the specific permissions that are required.
- **Passthrough**: In passthrough mode, the CCO passes the provided cloud credential to the components that request cloud credentials.
- **Manual mode with long-term credentials for components** In manual mode, you can manage long-term cloud credentials instead of the CCO.
- **Manual mode with short-term credentials for components** For some providers, you can use the CCO utility (**ccoctl**) during installation to implement short-term credentials for individual components. These credentials are created and managed outside the OpenShift Container Platform cluster.

If no mode is specified, or the **credentialsMode** parameter is set to an empty string (""), the CCO operates in its default mode.

Not all CCO modes are supported for all cloud providers, as described in the following table:

Table 21.1. CCO mode support matrix

Cloud provider	Mint	Passthrough	Manual with long-term credentials	Manual with short-term credentials
Amazon Web Services (AWS)	X	X	X	X
Global Microsoft Azure		X	X	X
Microsoft Azure Stack Hub			X	
Google Cloud	X	X	X	X

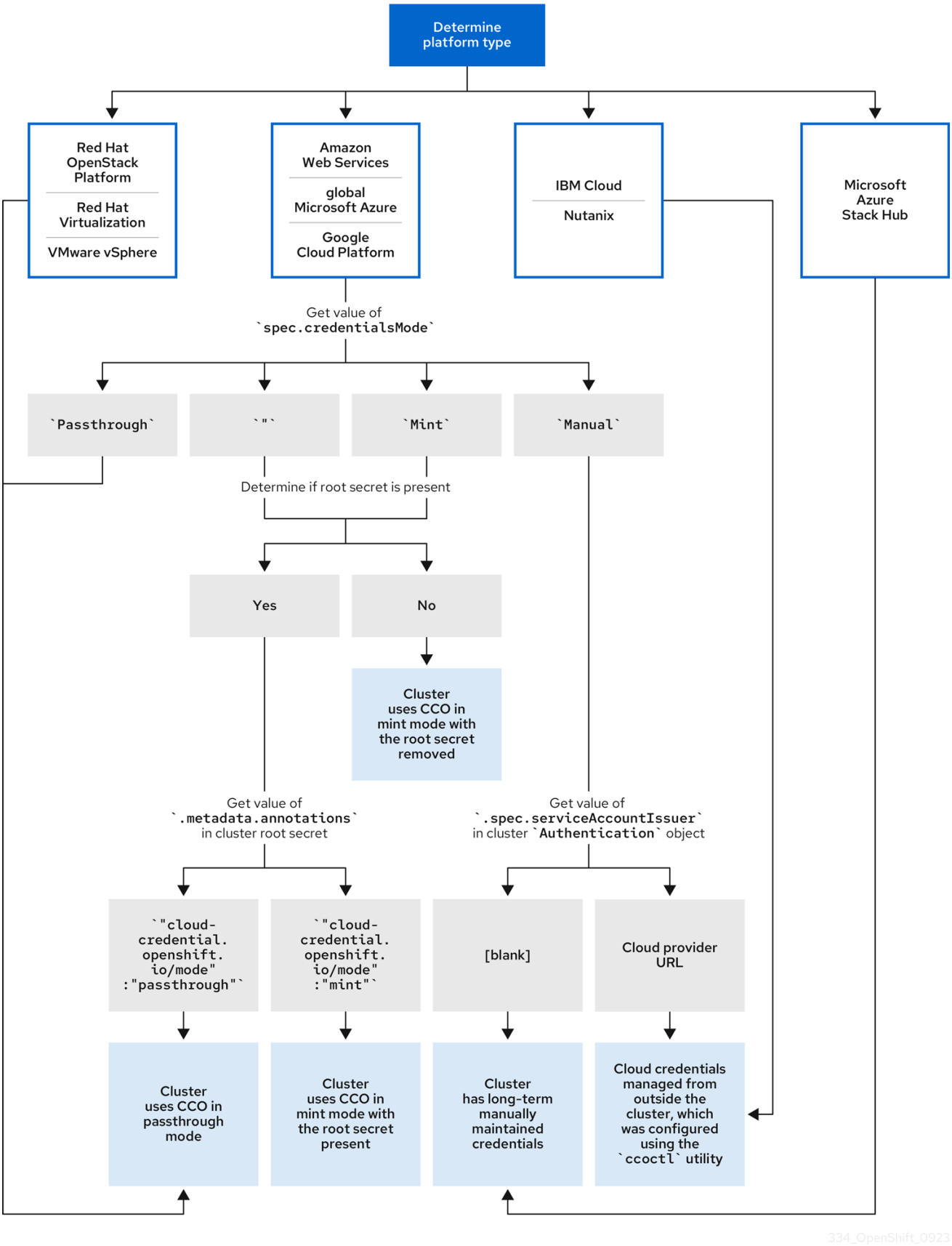
Cloud provider	Mint	Passthrough	Manual with long-term credentials	Manual with short-term credentials
IBM Cloud®			X [1]	
Nutanix			X [1]	
Red Hat OpenStack Platform (RHOSP)		X		
VMware vSphere		X		

1. This platform uses the **ccoctl** utility during installation to configure long-term credentials.

21.1.2. Determining the Cloud Credential Operator mode

For platforms that support using the CCO in multiple modes, you can determine what mode the CCO is configured to use by using the web console or the CLI.

Figure 21.1. Determining the CCO configuration



21.1.2.1. Determining the Cloud Credential Operator mode by using the web console

You can determine what mode the Cloud Credential Operator (CCO) is configured to use by using the web console.

Before you perform upgrades or troubleshoot, ensure you understand your cluster's credential management configuration.

NOTE

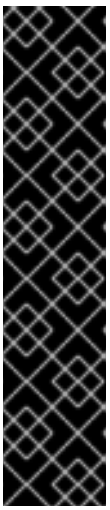
Only Amazon Web Services (AWS), global Microsoft Azure, and Google Cloud clusters support multiple CCO modes.

Prerequisites

- You have access to an OpenShift Container Platform account with cluster administrator permissions.

Procedure

1. Log in to the OpenShift Container Platform web console as a user with the **cluster-admin** role.
2. Navigate to **Administration → Cluster Settings**.
3. On the **Cluster Settings** page, select the **Configuration** tab.
4. Under **Configuration resource**, select **CloudCredential**.
5. On the **CloudCredential details** page, select the **YAML** tab.
6. In the YAML block, check the value of **spec.credentialsMode**. The following values are possible, though not all are supported on all platforms:
   - **''**: The CCO is operating in the default mode. In this configuration, the CCO operates in mint or passthrough mode, depending on the credentials provided during installation.
   - **Mint**: The CCO is operating in mint mode.
   - **Passthrough**: The CCO is operating in passthrough mode.
   - **Manual**: The CCO is operating in manual mode.



IMPORTANT

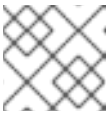
To determine the specific configuration of an AWS, Google Cloud, or global Microsoft Azure cluster that has a **spec.credentialsMode** of **''**, **Mint**, or **Manual**, you must investigate further.

AWS and Google Cloud clusters support using mint mode with the root secret deleted.

An AWS, Google Cloud, or global Microsoft Azure cluster that uses manual mode might be configured to create and manage cloud credentials from outside of the cluster with AWS STS, Google Cloud Workload Identity, or Microsoft Entra Workload ID. You can determine whether your cluster uses this strategy by examining the cluster **Authentication** object.

7. AWS or Google Cloud clusters that use the default (**''**) only: To determine whether the cluster is operating in mint or passthrough mode, inspect the annotations on the cluster root secret:

a. Navigate to **Workloads → Secrets** and look for the root secret for your cloud provider.



NOTE

Ensure that the **Project** dropdown is set to **All Projects**.

Platform	Secret name
AWS	**aws-creds**
Google Cloud	**gcp-credentials**

b. To view the CCO mode that the cluster is using, click **1 annotation** under **Annotations**, and check the value field. The following values are possible:

- **Mint**: The CCO is operating in mint mode.
- **Passthrough**: The CCO is operating in passthrough mode.

If your cluster uses mint mode, you can also determine whether the cluster is operating without the root secret.

8. AWS or Google Cloud clusters that use mint mode only: To determine whether the cluster is operating without the root secret, navigate to **Workloads → Secrets** and look for the root secret for your cloud provider.

NOTE

Ensure that the **Project** dropdown is set to **All Projects**.

Platform	Secret name
AWS	**aws-creds**
Google Cloud	**gcp-credentials**

- If you see one of these values, your cluster is using mint or passthrough mode with the root secret present.
- If you do not see these values, your cluster is using the CCO in mint mode with the root secret removed.

9. AWS, Google Cloud, or global Microsoft Azure clusters that use manual mode only: To determine whether the cluster is configured to create and manage cloud credentials from outside of the cluster, you must check the cluster **Authentication** object YAML values.

a. Navigate to **Administration → Cluster Settings**.

b. On the **Cluster Settings** page, select the **Configuration** tab.

c. Under **Configuration resource**, select **Authentication**.

d. On the **Authentication details** page, select the **YAML** tab.

e. In the YAML block, check the value of the **.spec.serviceAccountIssuer** parameter.

- A value that contains a URL that is associated with your cloud provider indicates that the CCO is using manual mode with short-term credentials for components. These clusters are configured using the **ccoctl** utility to create and manage cloud credentials from outside of the cluster.
- An empty value (**''**) indicates that the cluster is using the CCO in manual mode but was not configured using the **ccoctl** utility.

21.1.2.2. Determining the Cloud Credential Operator mode by using the CLI

You can determine what mode the Cloud Credential Operator (CCO) is configured to use by using the CLI.

Before you perform upgrades or troubleshoot, ensure you understand your cluster's credential management configuration.

NOTE

Only Amazon Web Services (AWS), global Microsoft Azure, and Google Cloud clusters support multiple CCO modes.

Prerequisites

- You have access to an OpenShift Container Platform account with cluster administrator permissions.
- You have installed the OpenShift CLI (**oc**).

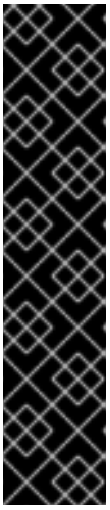
Procedure

1. Log in to **oc** on the cluster as a user with the **cluster-admin** role.
2. To determine the mode that the CCO is configured to use, enter the following command:

```
$ oc get cloudcredentials cluster \
  -o=jsonpath={.spec.credentialsMode}
```

The following output values are possible, though not all are supported on all platforms:

- **''**: The CCO is operating in the default mode. In this configuration, the CCO operates in mint or passthrough mode, depending on the credentials provided during installation.
- **Mint**: The CCO is operating in mint mode.
- **Passthrough**: The CCO is operating in passthrough mode.
- **Manual**: The CCO is operating in manual mode.



IMPORTANT

To determine the specific configuration of an AWS, Google Cloud, or global Microsoft Azure cluster that has a **spec.credentialsMode** of **''**, **Mint**, or **Manual**, you must investigate further.

AWS and Google Cloud clusters support using mint mode with the root secret deleted.

An AWS, Google Cloud, or global Microsoft Azure cluster that uses manual mode might be configured to create and manage cloud credentials from outside of the cluster with AWS STS, Google Cloud Workload Identity, or Microsoft Entra Workload ID. You can determine whether your cluster uses this strategy by examining the cluster **Authentication** object.

3. AWS or Google Cloud clusters that use the default (**''**) only: To determine whether the cluster is operating in mint or passthrough mode, run the following command:

```
$ oc get secret <secret_name> \
  -n kube-system \
  -o jsonpath \
  --template '{ .metadata.annotations }'
```

   where **<secret_name>** is **aws-creds** for AWS or **gcp-credentials** for Google Cloud.

   This command displays the value of the **.metadata.annotations** parameter in the cluster root secret object. The following output values are possible:

   - **Mint**: The CCO is operating in mint mode.
   - **Passthrough**: The CCO is operating in passthrough mode.

   If your cluster uses mint mode, you can also determine whether the cluster is operating without the root secret.

4. AWS or Google Cloud clusters that use mint mode only: To determine whether the cluster is operating without the root secret, run the following command:

```
$ oc get secret <secret_name> \
  -n=kube-system
```

   where **<secret_name>** is **aws-creds** for AWS or **gcp-credentials** for Google Cloud.

   If the root secret is present, the output of this command returns information about the secret. An error indicates that the root secret is not present on the cluster.

5. AWS, Google Cloud, or global Microsoft Azure clusters that use manual mode only: To determine whether the cluster is configured to create and manage cloud credentials from outside of the cluster, run the following command:

```
$ oc get authentication cluster \
  -o jsonpath \
  --template='{ .spec.serviceAccountIssuer }'
```

This command displays the value of the **.spec.serviceAccountIssuer** parameter in the cluster **Authentication** object.

- An output of a URL that is associated with your cloud provider indicates that the CCO is using manual mode with short-term credentials for components. These clusters are configured using the **ccoctl** utility to create and manage cloud credentials from outside of the cluster.
- An empty output indicates that the cluster is using the CCO in manual mode but was not configured using the **ccoctl** utility.

21.1.3. About the Cloud Credential Operator default behavior

To better manage cloud credentials, you should familiarize yourself with the default behaviors of the Cloud Credential Operator.

For platforms on which multiple modes are supported (AWS, Azure, and Google Cloud), when the CCO operates in its default mode, it checks the provided credentials dynamically to determine for which mode they are sufficient to process **CredentialsRequest** CRs.

By default, the CCO determines whether the credentials are sufficient for mint mode, which is the preferred mode of operation, and uses those credentials to create appropriate credentials for components in the cluster. If the credentials are not sufficient for mint mode, it determines whether they are sufficient for passthrough mode. If the credentials are not sufficient for passthrough mode, the CCO cannot adequately process **CredentialsRequest** CRs.

If the provided credentials are determined to be insufficient during installation, the installation fails. For AWS, the installation program fails early in the process and indicates which required permissions are missing. Other providers might not provide specific information about the cause of the error until errors are encountered.

If the credentials are changed after a successful installation and the CCO determines that the new credentials are insufficient, the CCO puts conditions on any new **CredentialsRequest** CRs to indicate that it cannot process them because of the insufficient credentials.

To resolve insufficient credentials issues, provide a credential with sufficient permissions. If an error occurred during installation, try installing again. For issues with new **CredentialsRequest** CRs, wait for the CCO to try to process the CR again. As an alternative, you can configure your cluster to use a different CCO mode that is supported for your cloud provider.

21.1.4. Additional resources

- Cluster Operators reference page for the Cloud Credential Operator
- About the Cloud Credential Operator in mint mode
- About the Cloud Credential Operator in passthrough mode
- About the Cloud Credential Operator in manual mode with long-term credentials for components
- About the Cloud Credential Operator in manual mode with short-term credentials for components

21.2. ABOUT THE CLOUD CREDENTIAL OPERATOR IN MINT MODE

You can use the Cloud Credential Operator (CCO) in mint mode to create and reconcile credentials for components in the cluster.

Mint mode is the default CCO credentials mode for OpenShift Container Platform on platforms that support it. Mint mode supports Amazon Web Services (AWS) and Google Cloud clusters.

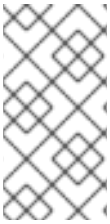
21.2.1. About mint mode credentials management

When using the Cloud Credential Operator (CCO) in mint mode, you should be familiar with how the CCO uses provider credentials.

For clusters that use the CCO in mint mode, the administrator-level credential is stored in the **kube-system** namespace. The CCO uses the **admin** credential to process the **CredentialsRequest** objects in the cluster and create users for components with limited permissions.

With mint mode, each cluster component has only the specific permissions it requires. Cloud credential reconciliation is automatic and continuous so that components can perform actions that require additional credentials or permissions.

For example, a minor version cluster update (such as updating from OpenShift Container Platform 4.21 to 4.22) might include an updated **CredentialsRequest** resource for a cluster component. The CCO, operating in mint mode, uses the **admin** credential to process the **CredentialsRequest** resource and create users with limited permissions to satisfy the updated authentication requirements.



NOTE

By default, mint mode requires storing the **admin** credential in the cluster **kube-system** namespace. If this approach does not meet the security requirements of your organization, you can remove the credential after installing the cluster. For more information, see "Removing cloud provider credentials".

21.2.1.1. About mint mode permissions requirements

When using the Cloud Credential Operator (CCO) in mint mode, ensure that the credential you provide meets the requirements of the cloud on which you are running or installing OpenShift Container Platform. If the provided credentials are not sufficient for mint mode, the CCO cannot create an IAM user.

21.2.1.1.1. Required AWS permissions

The credential you provide for mint mode in Amazon Web Services (AWS) must have the following permissions:

- **iam:CreateAccessKey**
- **iam:CreateUser**
- **iam:DeleteAccessKey**
- **iam:DeleteUser**
- **iam:DeleteUserPolicy**
- **iam:GetUser**

- **iam:GetUserPolicy**
- **iam:ListAccessKeys**
- **iam:PutUserPolicy**
- **iam:TagUser**
- **iam:SimulatePrincipalPolicy**

21.2.1.1.2. Required Google Cloud permissions

The credential you provide for mint mode in Google Cloud must have the following permissions:

- **resourcemanager.projects.get**
- **serviceusage.services.list**
- **iam.serviceAccountKeys.create**
- **iam.serviceAccountKeys.delete**
- **iam.serviceAccountKeys.list**
- **iam.serviceAccounts.create**
- **iam.serviceAccounts.delete**
- **iam.serviceAccounts.get**
- **iam.roles.create**
- **iam.roles.get**
- **iam.roles.list**
- **iam.roles.undelete**
- **iam.roles.update**
- **resourcemanager.projects.getIamPolicy**
- **resourcemanager.projects.setIamPolicy**

21.2.1.2. Admin credentials root secret format

The Cloud Credential Operator (CCO) creates a credentials root secret by minting new credentials with *mint mode* or by copying the credentials root secret with *passthrough mode*.

Each cloud provider uses a credentials root secret in the **kube-system** namespace by convention, which is then used to satisfy all credentials requests and create their respective secrets.

The format for the secret varies by cloud, and is also used for each **CredentialsRequest** secret.

Amazon Web Services (AWS) secret format

-

```
apiVersion: v1
kind: Secret
metadata:
  namespace: kube-system
  name: aws-creds
stringData:
  aws_access_key_id: <base64-encoded_access_key_id>
  aws_secret_access_key: <base64-encoded_secret_access_key>
```

Google Cloud secret format

```
apiVersion: v1
kind: Secret
metadata:
  namespace: kube-system
  name: gcp-credentials
stringData:
  service_account.json: <base64-encoded_service_account>
```

21.2.2. Maintaining cloud provider credentials

If your cloud provider credentials are changed for any reason, you must manually update the secret that the Cloud Credential Operator (CCO) uses to manage cloud provider credentials.

The process for rotating cloud credentials depends on the mode that the CCO is configured to use. After you rotate credentials for a cluster that is using mint mode, you must manually remove the component credentials that were created by the removed credential.

Prerequisites

- Your cluster is installed on a platform that supports rotating cloud credentials manually with the CCO mode that you are using:
  - For mint mode, Amazon Web Services (AWS) and Google Cloud are supported.
- You have changed the credentials that are used to interface with your cloud provider.
- The new credentials have sufficient permissions for the mode CCO is configured to use in your cluster.

Procedure

1. In the **Administrator** perspective of the web console, navigate to **Workloads → Secrets**.
2. In the table on the **Secrets** page, find the root secret for your cloud provider.

Platform	Secret name
AWS	**aws-creds**
Google Cloud	**gcp-credentials**

3. Click the Options menu  in the same row as the secret and select **Edit Secret**.

4. Record the contents of the **Value** field or fields. You can use this information to verify that the value is different after updating the credentials.

5. Update the text in the **Value** field or fields with the new authentication information for your cloud provider, and then click **Save**.

6. Delete each component secret that is referenced by the individual **CredentialsRequest** objects.

   a. Log in to the OpenShift Container Platform CLI as a user with the **cluster-admin** role.

   b. Get the names and namespaces of all referenced component secrets:

   ```
   $ oc -n openshift-cloud-credential-operator get CredentialsRequest \
     -o json | jq -r '.items[] | select (.spec.providerSpec.kind=="<provider_spec>") |
   .spec.secretRef'
   ```

   where **<provider_spec>** is the corresponding value for your cloud provider:

   - AWS: **AWSProviderSpec**
   - Google Cloud: **GCPProviderSpec**

   The following example is partial output for the command on an AWS cluster:

   ```
   {
     "name": "ebs-cloud-credentials",
     "namespace": "openshift-cluster-csi-drivers"
   }
   {
     "name": "cloud-credential-operator-iam-ro-creds",
     "namespace": "openshift-cloud-credential-operator"
   }
   ```

   c. Delete each of the referenced component secrets:

   ```
   $ oc delete secret <secret_name> \ 1
     -n <secret_namespace> 2
   ```

   where:

   <secret_name>

   Specifies the name of a secret.

   <secret_namespace>

   Specifies the namespace that contains the secret.

   The following example is a command to delete an AWS secret:

   ```
   $ oc delete secret ebs-cloud-credentials -n openshift-cluster-csi-drivers
   ```

You do not need to manually delete the credentials from your provider console. Deleting the referenced component secrets will cause the CCO to delete the existing credentials from the platform and create new ones.

Verification

1. In the **Administrator** perspective of the web console, navigate to **Workloads → Secrets**.
2. Verify that the contents of the **Value** field or fields have changed.

21.2.3. Additional resources

- Removing cloud provider credentials

21.3. ABOUT THE CLOUD CREDENTIAL OPERATOR IN PASSTHROUGH MODE

To allow the Cloud Credential Operator (CCO) to pass cloud credentials to the components that request them, you can configure the Cloud Credential Operator (CCO) to operate in passthrough mode.

The credential must have permissions to perform the installation and complete the operations that are required by components in the cluster, but does not need to be able to create new credentials. The CCO does not attempt to create additional limited-scoped credentials in passthrough mode.

Passthrough mode is supported for Amazon Web Services (AWS), Microsoft Azure, Google Cloud, Red Hat OpenStack Platform (RHOSP), and VMware vSphere.

NOTE

Manual mode is the only supported CCO configuration for Microsoft Azure Stack Hub.

21.3.1. Passthrough mode permissions requirements

When using the CCO in passthrough mode, ensure that the credential you provide meets the requirements of the cloud on which you are running or installing OpenShift Container Platform. If the provided credentials the CCO passes to a component that creates a **CredentialsRequest** CR are not sufficient, that component will report an error when it tries to call an API that it does not have permissions for.

Amazon Web Services (AWS) permissions

The credential you provide for passthrough mode in AWS must have all the requested permissions for all **CredentialsRequest** CRs that are required by the version of OpenShift Container Platform you are running or installing.
To locate the **CredentialsRequest** CRs that are required, see "Manually creating long-term credentials for AWS.

Microsoft Azure permissions

The credential you provide for passthrough mode in Azure must have all the requested permissions for all **CredentialsRequest** CRs that are required by the version of OpenShift Container Platform you are running or installing.
To locate the **CredentialsRequest** CRs that are required, see "Manually creating long-term credentials for Azure".

Google Cloud permissions

The credential you provide for passthrough mode in Google Cloud must have all the requested permissions for all **CredentialsRequest** CRs that are required by the version of OpenShift Container Platform you are running or installing.

To locate the **CredentialsRequest** CRs that are required, see "Manually creating long-term credentials for Google Cloud".

Red Hat OpenStack Platform (RHOSP) permissions

To install an OpenShift Container Platform cluster on RHOSP, the CCO requires a credential with the permissions of a **member** user role.

VMware vSphere permissions

To install an OpenShift Container Platform cluster on VMware vSphere, the CCO requires a credential with the following vSphere privileges:

Table 21.2. Required vSphere privileges

Category	Privileges
Datastore	*Allocate space*
Folder	*Create folder*, *Delete folder*
vSphere Tagging	All privileges
Network	*Assign network*
Resource	*Assign virtual machine to resource pool*
Profile-driven storage	All privileges
vApp	All privileges
Virtual machine	All privileges

If **CredentialsRequest** CRs change over time as the cluster is upgraded, you must manually update the passthrough mode credential to meet the requirements. To avoid credentials issues during an upgrade, check the **CredentialsRequest** CRs in the release image for the new version of OpenShift Container Platform before upgrading.

To locate the **CredentialsRequest** CRs that are required for AWS, Azure, or Google Cloud, see the *Manually creating long-term credentials* topic for your platform.

21.3.2. Admin credentials root secret format

The Cloud Credential Operator (CCO) creates a credentials root secret by minting new credentials with *mint mode* or by copying the credentials root secret with *passthrough mode*.

Each cloud provider uses a credentials root secret in the **kube-system** namespace by convention, which is then used to satisfy all credentials requests and create their respective secrets.

The format for the secret varies by cloud, and is also used for each **CredentialsRequest** secret.

Amazon Web Services (AWS) secret format

```
apiVersion: v1
kind: Secret
metadata:
  namespace: kube-system
  name: aws-creds
stringData:
  aws_access_key_id: <base64-encoded_access_key_id>
  aws_secret_access_key: <base64-encoded_secret_access_key>
```

Microsoft Azure secret format

```
apiVersion: v1
kind: Secret
metadata:
  namespace: kube-system
  name: azure-credentials
stringData:
  azure_subscription_id: <base64-encoded_subscription_id>
  azure_client_id: <base64-encoded_client_id>
  azure_client_secret: <base64-encoded_client_secret>
  azure_tenant_id: <base64-encoded_tenant_id>
  azure_resource_prefix: <base64-encoded_resource_prefix>
  azure_resourcegroup: <base64-encoded_resource_group>
  azure_region: <base64-encoded_region>
```

On Microsoft Azure, the credentials secret format includes two properties that must contain the cluster's infrastructure ID, generated randomly for each cluster installation. This value can be found after running create manifests:

```
$ cat .openshift_install_state.json | jq '."*installconfig.ClusterID".InfraID' -r
```

Example output

```
mycluster-2mpcn
```

This value would be used in the secret data as follows:

```
azure_resource_prefix: mycluster-2mpcn
azure_resourcegroup: mycluster-2mpcn-rg
```

Google Cloud secret format

```
apiVersion: v1
kind: Secret
metadata:
  namespace: kube-system
  name: gcp-credentials
stringData:
  service_account.json: <base64-encoded_service_account>
```

-

Red Hat OpenStack Platform (RHOSP) secret format

```
apiVersion: v1
kind: Secret
metadata:
  namespace: kube-system
  name: openstack-credentials
data:
  clouds.yaml: <base64-encoded_cloud_creds>
  clouds.conf: <base64-encoded_cloud_creds_init>
```

VMware vSphere secret format

```
apiVersion: v1
kind: Secret
metadata:
  namespace: kube-system
  name: vsphere-creds
data:
 vsphere.openshift.example.com.username: <base64-encoded_username>
 vsphere.openshift.example.com.password: <base64-encoded_password>
```

21.3.2.1. Maintaining cloud provider credentials

If your cloud provider credentials are changed for any reason, you must manually update the secret that the Cloud Credential Operator (CCO) uses to manage cloud provider credentials.

The process for rotating cloud credentials depends on the mode that the CCO is configured to use. After you rotate credentials for a cluster that is using mint mode, you must manually remove the component credentials that were created by the removed credential.

Prerequisites

- Your cluster is installed on a platform that supports rotating cloud credentials manually with the CCO mode that you are using:
  - For passthrough mode, Amazon Web Services (AWS), Microsoft Azure, Google Cloud, Red Hat OpenStack Platform (RHOSP), and VMware vSphere are supported.
- You have changed the credentials that are used to interface with your cloud provider.
- The new credentials have sufficient permissions for the mode CCO is configured to use in your cluster.

Procedure

1. In the **Administrator** perspective of the web console, navigate to **Workloads → Secrets**.
2. In the table on the **Secrets** page, find the root secret for your cloud provider.

Platform	Secret name
AWS	**aws-creds**

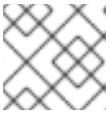
Platform	Secret name
Azure	**azure-credentials**
Google Cloud	**gcp-credentials**
RHOSP	**openstack-credentials**
VMware vSphere	**vsphere-creds**

3. Click the Options menu  in the same row as the secret and select **Edit Secret**.

4. Record the contents of the **Value** field or fields. You can use this information to verify that the value is different after updating the credentials.

5. Update the text in the **Value** field or fields with the new authentication information for your cloud provider, and then click **Save**.

6. If you are updating the credentials for a vSphere cluster that does not have the vSphere CSI Driver Operator enabled, you must force a rollout of the Kubernetes controller manager to apply the updated credentials.



NOTE

If the vSphere CSI Driver Operator is enabled, this step is not required.

To apply the updated vSphere credentials, log in to the OpenShift Container Platform CLI as a user with the **cluster-admin** role and run the following command:

```
$ oc patch kubecontrollermanager cluster \
  -p='{"spec": {"forceRedeploymentReason": "recovery-'"$( date )"'"}}' \
  --type=merge
```

While the credentials are rolling out, the status of the Kubernetes Controller Manager Operator reports **Progressing=true**. To view the status, run the following command:

```
$ oc get co kube-controller-manager
```

Verification

1. In the **Administrator** perspective of the web console, navigate to **Workloads → Secrets**.

2. Verify that the contents of the **Value** field or fields have changed.

Additional resources

- vSphere CSI Driver Operator

21.3.2.2. Reducing permissions after installation

When using passthrough mode, after installing you can reduce the installed permissions to only those permissions required to run the cluster.

In passthrough mode, each component has the same permissions used by all other components. If you do not reduce the permissions after installing, all components have the broad permissions that are required to run the installation program.

After installation, reduce the permissions on your credential to only those defined by the **CredentialsRequest** CRs in the release image for the version of OpenShift Container Platform that you are using.

To locate the **CredentialsRequest** CRs that are required for AWS, Azure, or Google Cloud and learn how to change the permissions the CCO uses, see the *Manually creating long-term credentials* topic for your platform.

21.3.3. Additional resources

- Manually creating long-term credentials for AWS
- Manually creating long-term credentials for Azure
- Manually creating long-term credentials for Google Cloud

21.4. ABOUT THE CLOUD CREDENTIAL OPERATOR IN MANUAL MODE WITH LONG-TERM CREDENTIALS FOR COMPONENTS

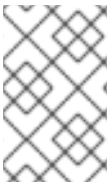
You can manage your cloud credentials instead of the Cloud Credential Operator (CCO) by setting the Operator to manual mode.

You can use manual mode with your Amazon Web Services (AWS), global Microsoft Azure, Microsoft Azure Stack Hub, Google Cloud, IBM Cloud®, or Nutanix cluster.

To use manual mode, you must examine the **CredentialsRequest** CRs in the release image for the version of OpenShift Container Platform that you are running or installing, create corresponding credentials in the underlying cloud provider, and create Kubernetes Secrets in the correct namespaces to satisfy all **CredentialsRequest** CRs for the cluster's cloud provider. Some platforms use the CCO utility (**ccoctl**) to facilitate this process during installation and updates.

Using manual mode with long-term credentials allows each cluster component to have only the permissions it requires, without storing an administrator-level credential in the cluster. This mode also does not require connectivity to services such as the AWS public IAM endpoint. However, you must manually reconcile permissions with new release images for every upgrade.

For information about configuring your cloud provider to use manual mode, see the manual credentials management options for your cloud provider.



NOTE

An AWS, global Azure, or Google Cloud cluster that uses manual mode can be configured to use short-term credentials for different components. For more information, see "Manual mode with short-term credentials for components".

21.4.1. Additional resources

- Manually creating long-term credentials for AWS
- Manually creating long-term credentials for Azure
- Manually creating long-term credentials for Google Cloud
- Configuring IAM for IBM Cloud®
- Configuring IAM for Nutanix
- About the Cloud Credential Operator in manual mode with short-term credentials for components
- Preparing to update a cluster with manually maintained credentials

21.5. ABOUT THE CLOUD CREDENTIAL OPERATOR IN MANUAL MODE WITH SHORT-TERM CREDENTIALS FOR COMPONENTS

During installation, you can configure the Cloud Credential Operator (CCO) to operate in manual mode and use the CCO utility (**ccoctl**) to implement short-term security credentials for individual components that are created and managed outside the OpenShift Container Platform cluster.

NOTE

This credentials strategy is supported for Amazon Web Services (AWS), Google Cloud, and global Microsoft Azure only.

For AWS and Google Cloud clusters, you must configure your cluster to use this strategy during installation of a new OpenShift Container Platform cluster. You cannot configure an existing AWS or Google Cloud cluster that uses a different credentials strategy to use this feature.

If you did not configure your Azure cluster to use Microsoft Entra Workload ID during installation, you can enable this authentication method on an existing cluster. For information, see "Enabling token-based authentication".

Cloud providers use different terms for their implementation of this authentication method.

Table 21.3. Short-term credentials provider terminology

Cloud provider	Provider nomenclature
Amazon Web Services (AWS)	AWS Security Token Service (STS)
Google Cloud	GCP Workload Identity
Global Microsoft Azure	Microsoft Entra Workload ID

21.5.1. About AWS Security Token Service

To assign IAM roles that provide short-term, limited-privilege security credentials to your cluster components, you can configure your cluster to use manual mode with Security Token Service (STS), allowing the individual OpenShift Container Platform components to use the AWS STS.

These credentials are associated with IAM roles that are specific to each component that makes AWS API calls.

Additional resources

- Configuring an AWS cluster to use short-term credentials

21.5.1.1. AWS Security Token Service authentication process

You should familiarize yourself with the process that the AWS Security Token Service (STS) and the **AssumeRole** API action perform to allow pods to retrieve access keys that are defined by an IAM role policy.

The OpenShift Container Platform cluster includes a Kubernetes service account signing service. This service uses a private key to sign service account JSON web tokens (JWT). A pod that requires a service account token requests one through the pod specification. When the pod is created and assigned to a node, the node retrieves a signed service account from the service account signing service and mounts it onto the pod.

Clusters that use STS contain an IAM role ID in their Kubernetes configuration secrets. Workloads assume the identity of this IAM role ID. The signed service account token issued to the workload aligns with the configuration in AWS, which allows AWS STS to grant access keys for the specified IAM role to the workload.

AWS STS grants access keys only for requests that include service account tokens that meet the following conditions:

- The token name and namespace match the service account name and namespace.
- The token is signed by a key that matches the public key.

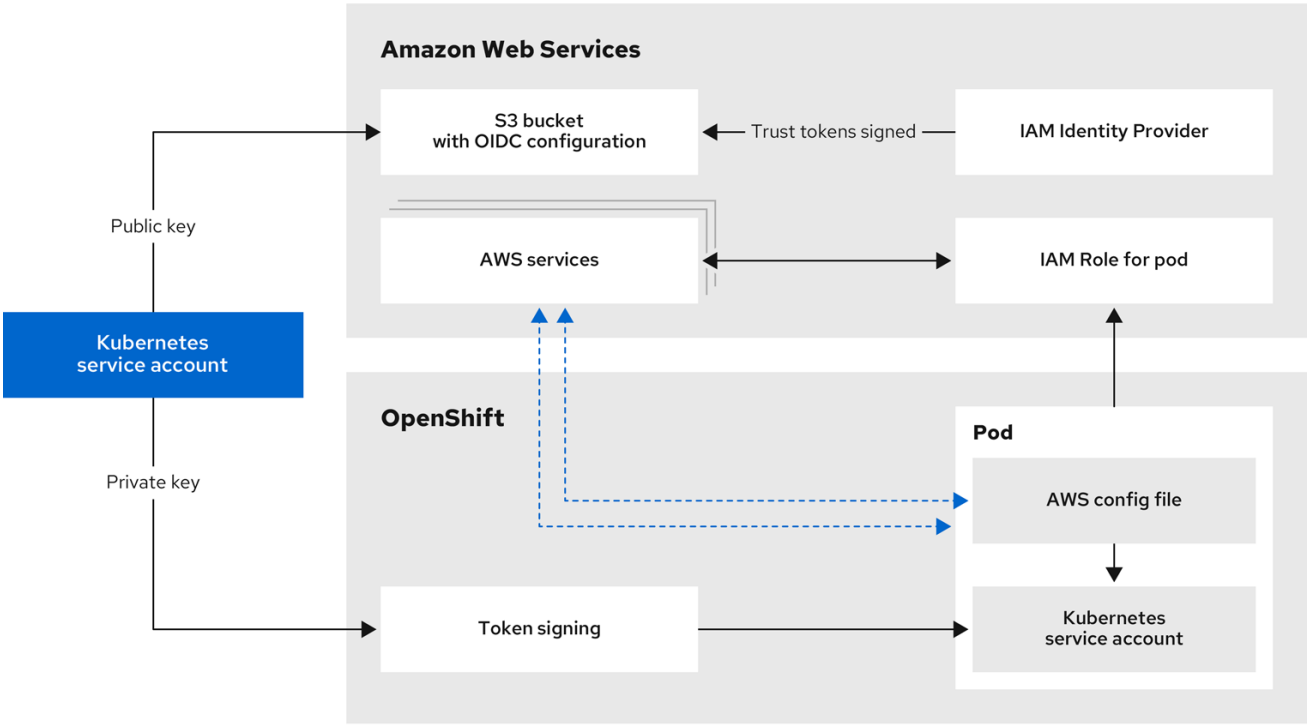
The public key pair for the service account signing key used by the cluster is stored in an AWS S3 bucket. AWS STS federation validates that the service account token signature aligns with the public key stored in the S3 bucket.

21.5.1.1.1. Authentication flow for AWS STS

The following diagram illustrates the authentication flow between AWS and the OpenShift Container Platform cluster when using AWS STS.

- *Token signing* is the Kubernetes service account signing service on the OpenShift Container Platform cluster.
- The *Kubernetes service account* in the pod is the signed service account token.

Figure 21.2. AWS Security Token Service authentication flow



Requests for new and refreshed credentials are automated by using an appropriately configured AWS IAM OpenID Connect (OIDC) identity provider combined with AWS IAM roles. Service account tokens that are trusted by AWS IAM are signed by OpenShift Container Platform and can be projected into a pod and used for authentication.

21.5.1.1.2. Token refreshing for AWS STS

The signed service account token that a pod uses expires after a period of time. For clusters that use AWS STS, this time period is 3600 seconds, or one hour.

The kubelet on the node that the pod is assigned to ensures that the token is refreshed. The kubelet attempts to rotate a token when it is older than 80 percent of its time to live.

21.5.1.1.3. OpenID Connect requirements for AWS STS

You can store the public portion of the encryption keys for your OIDC configuration in a public or private S3 bucket.

The OIDC spec requires the use of HTTPS. AWS services require a public endpoint to expose the OIDC documents in the form of JSON web key set (JWKS) public keys. This allows AWS services to validate the bound tokens signed by Kubernetes and determine whether to trust certificates. As a result, both S3 bucket options require a public HTTPS endpoint and private endpoints are not supported.

To use AWS STS, the public AWS backbone for the AWS STS service must be able to communicate with a public S3 bucket or a private S3 bucket with a public CloudFront endpoint. You can choose which type of bucket to use when you process **CredentialsRequest** objects during installation:

- By default, the CCO utility (**ccoctl**) stores the OIDC configuration files in a public S3 bucket and uses the S3 URL as the public OIDC endpoint.

- As an alternative, you can have the **ccoctl** utility store the OIDC configuration in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL.

21.5.1.2. AWS component secret formats

To change the content of the AWS credentials that are provided to individual OpenShift Container Platform components, you can use manual mode with the AWS Security Token Service (STS) .

Compare the following secret formats:

AWS secret format using long-term credentials

```
apiVersion: v1
kind: Secret
metadata:
  namespace: <target_namespace>
  name: <target_secret_name>
data:
  aws_access_key_id: <base64_encoded_access_key_id>
  aws_secret_access_key: <base64_encoded_secret_access_key>
```

where:

metadata.namespace

Specifies the namespace for the component.

metadata.name

Specifies the name of the component secret.

AWS secret format using AWS STS

```
apiVersion: v1
kind: Secret
metadata:
  namespace: <target_namespace>
  name: <target_secret_name>
stringData:
  credentials: |-
    [default]
    sts_regional_endpoints = regional
    role_name: <operator_role_name>
    web_identity_token_file: <path_to_token>
```

where:

metadata.namespace

Specifies the namespace for the component.

metadata.name

Specifies the name of the component secret.

stringData.credentials.role_name

Specifies the IAM role for the component.

stringData.credentials.web_identity_token_file

Specifies the path to the service account token inside the pod. By convention, this is **/var/run/secrets/openshift/serviceaccount/token** for OpenShift Container Platform components.

21.5.1.3. AWS component secret permissions requirements

You should familiarize yourself with the permissions required by the OpenShift Container Platform components. These values are in the **CredentialsRequest** custom resource (CR) for each component.

OpenShift Container Platform components require the following permissions:

NOTE

These permissions apply to all resources. Unless specified, there are no request conditions on these permissions.

Component	Custom resource	Required permissions for services
Cluster CAPI Operator	**openshift-cluster-api-aws**	EC2 • **ec2:CreateTags** • **ec2:DescribeAvailabilityZones** • **ec2:DescribeDhcpOptions** • **ec2:DescribeImages** • **ec2:DescribeInstances** • **ec2:DescribeInternetGateways** • **ec2:DescribeSecurityGroups** • **ec2:DescribeSubnets** • **ec2:DescribeVpcs** • **ec2:DescribeNetworkInterfaces** • **ec2:DescribeNetworkInterfaceAttribute** • **ec2:ModifyNetworkInterfaceAttribute** • **ec2:RunInstances** • **ec2:TerminateInstances**

Component	Custom resource	Required permissions for services
		Elastic load balancing • **elasticloadbalancing:DescribeLoadBalancers** • **elasticloadbalancing:DescribeTargetGroups** • **elasticloadbalancing:DescribeTargetHealth** • **elasticloadbalancing:RegisterInstancesWithLoadBalancer** • **elasticloadbalancing:RegisterTargets** • **elasticloadbalancing:DeregisterTargets** Identity and Access Management (IAM) • **iam:PassRole** • **iam:CreateServiceLinkedRole** Key Management Service (KMS) • **kms:Decrypt** • **kms:Encrypt** • **kms:GenerateDataKey** • **kms:GenerateDataKeyWithoutPlainText** • **kms:DescribeKey** • **kms:RevokeGrant**[1] • **kms:CreateGrant** [1] • **kms:ListGrants** [1]
Machine API Operator	**openshift-machine-api-aws**	EC2 • **ec2:CreateTags** • **ec2:DescribeAvailabilityZones**

Component	Custom resource	Required permissions for services
		• ec2:DescribeDhcpOp • ec2:DescribeImages • ec2:DescribeInstances • ec2:DescribeInstanceTypes • ec2:DescribeInternetGateways • ec2:DescribeSecurityGroups • ec2:DescribeRegions • ec2:DescribeSubnets • ec2:DescribeVpcs • ec2:RunInstances • ec2:TerminateInstances Elastic load balancing • elasticloadbalancing:DescribeLoadBalancers • elasticloadbalancing:DescribeTargetGroups • elasticloadbalancing:DescribeTargetHealth • elasticloadbalancing:RegisterInstancesWithLoadBalancer • elasticloadbalancing:RegisterTargets • elasticloadbalancing:DeregisterTargets Identity and Access Management (IAM) • iam:PassRole • iam:CreateServiceLinkedRole Key Management Service (KMS)

Component	Custom resource	Required permissions for services
		• **kms:Decrypt** • **kms:Encrypt** • **kms:GenerateDataKey** • **kms:GenerateDataKeyWithoutPlainText** • **kms:DescribeKey** • **kms:RevokeGrant**[1] • **kms:CreateGrant** [1] • **kms:ListGrants** [1]
Cloud Credential Operator	**cloud-credential-operator-iam-ro**	Identity and Access Management (IAM) • **iam:GetUser** • **iam:GetUserPolicy** • **iam:ListAccessKeys**

Component	Custom resource	Required permissions for services
Cluster Image Registry Operator	**openshift-image-registry**	S3 • **s3:CreateBucket** • **s3:DeleteBucket** • **s3:PutBucketTagging** • **s3:GetBucketTagging** • **s3:PutBucketPublicAccessBlock** • **s3:GetBucketPublicAccessBlock** • **s3:PutEncryptionConfiguration** • **s3:GetEncryptionConfiguration** • **s3:PutLifecycleConfiguration** • **s3:GetLifecycleConfiguration** • **s3:GetBucketLocation** • **s3:ListBucket** • **s3:GetObject** • **s3:PutObject** • **s3:DeleteObject** • **s3:ListBucketMultipartUploads** • **s3:AbortMultipartUpload** • **s3:ListMultipartUploadParts**

Component	Custom resource	Required permissions for services
Ingress Operator	**openshift-ingress**	Elastic load balancing • **elasticloadbalancing:DescribeLoadBalancers** Route 53 • **route53:ListHostedZones** • **route53:ListTagsForResources** • **route53:ChangeResourceRecordSets** Tag • **tag:GetResources** Security Token Service (STS) • **sts:AssumeRole**
Cluster Network Operator	**openshift-cloud-network-config-controller-aws**	EC2 • **ec2:DescribeInstances** • **ec2:DescribeInstanceStatus** • **ec2:DescribeInstanceTypes** • **ec2:UnassignPrivateIpAddresses** • **ec2:AssignPrivateIpAddresses** • **ec2:UnassignIpv6Addresses** • **ec2:AssignIpv6Addresses** • **ec2:DescribeSubnets** • **ec2:DescribeNetworkInterfaces**

Component	Custom resource	Required permissions for services
AWS Elastic Block Store CSI Driver Operator	**aws-ebs-csi-driver-operator**	EC2 **ec2:AttachVolume** • **ec2:CreateSnapshot** • **ec2:CreateTags** • **ec2:CreateVolume** • **ec2:DeleteSnapshot** • **ec2:DeleteTags** • **ec2:DeleteVolume** • **ec2:DescribeInstances** • **ec2:DescribeSnapshots** • **ec2:DescribeTags** • **ec2:DescribeVolumes** • **ec2:DescribeVolumesModifications** • **ec2:DetachVolume** • **ec2:ModifyVolume** • **ec2:DescribeAvailabilityZones** • **ec2:EnableFastSnapshotRestores** Key Management Service (KMS) • **kms:ReEncrypt*** • **kms:Decrypt** • **kms:Encrypt** • **kms:GenerateDataKey** • **kms:GenerateDataKeyWithoutPlainText** • **kms:DescribeKey** • **kms:RevokeGrant**[1] • **kms:CreateGrant** [1] • **kms:ListGrants** [1]

Component	Custom resource	Required permissions for services
		1. Request condition: **kms:GrantIsForAWSResource: true**

21.5.1.4. OLM-managed Operator support for authentication with AWS STS

To allow certain Operators that are managed by the Operator Lifecycle Manager (OLM) on AWS clusters to authenticate with limited-privilege, short-term credentials that are managed outside the cluster, you can use manual mode with STS.

To determine if an Operator supports authentication with AWS STS, see the Operator description in the software catalog.

Additional resources

- CCO-based workflow for OLM-managed Operators with AWS STS

21.5.2. About GCP Workload Identity

To allow components to use the Google Cloud Platform Workload Identity to impersonate Google Cloud service accounts using short-term, limited-privilege credentials, you can configure your cluster to use manual mode with GCP Workload Identity.

Additional resources

- Configuring a Google Cloud cluster to use short-term credentials

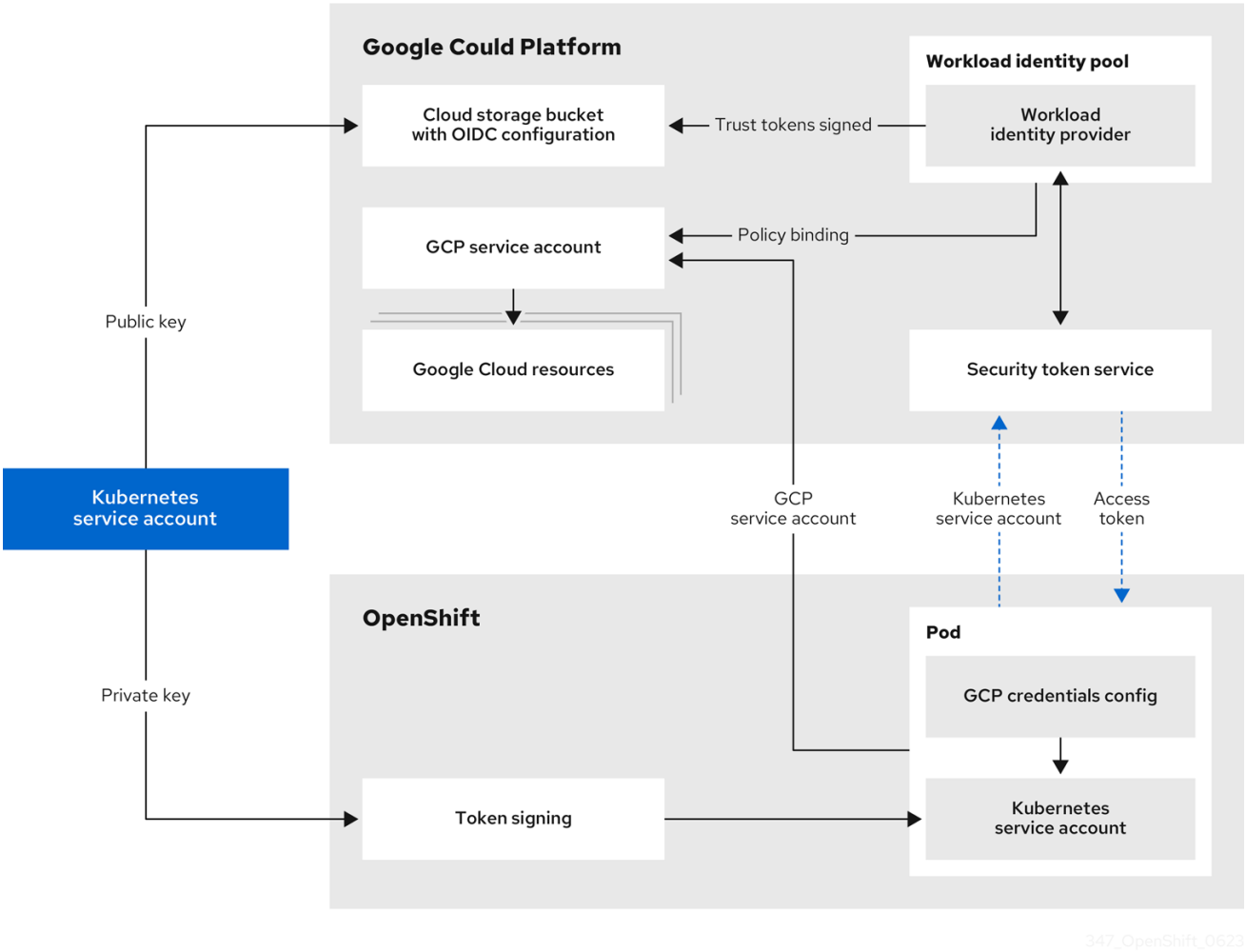
21.5.2.1. Google Cloud Workload Identity authentication process

You should familiarize yourself with the Google Cloud Workload Identity authentication process.

Requests for new and refreshed credentials are automated by using an appropriately configured OpenID Connect (OIDC) identity provider combined with IAM service accounts. Service account tokens that are trusted by Google Cloud are signed by OpenShift Container Platform and can be projected into a pod and used for authentication. Tokens are refreshed after one hour.

The following diagram details the authentication flow between Google Cloud and the OpenShift Container Platform cluster when using Google Cloud Workload Identity.

Figure 21.3. Google Cloud Workload Identity authentication flow



21.5.2.2. Google Cloud component secret formats

To change the content of the Google Cloud credentials that are provided to individual OpenShift Container Platform components, you can use manual mode with Google Cloud Workload Identity.

Compare the following secret content:

Google Cloud secret format

```
apiVersion: v1
kind: Secret
metadata:
  namespace: <target_namespace>
  name: <target_secret_name>
data:
  service_account.json: <service_account>
```

where:

metadata.namespace

Specifies the namespace for the component.

metadata.name

Specifies the name of the component secret.

data.service_account.json

Specifies the Base64 encoded service account.

Content of the Base64 encoded service_account.json file using long-term credentials

```
{
  "type": "service_account",
  "project_id": "<project_id>",
  "private_key_id": "<private_key_id>",
  "private_key": "<private_key>",
  "client_email": "<client_email_address>",
  "client_id": "<client_id>",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
  "client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/<client_email_address>"
}
```

where:

type

Specifies the credential type, in this example the type is **service_account**.

private_key

Specifies the private RSA key that is used to authenticate to Google Cloud. This key must be kept secure and is not rotated.

Content of the Base64 encoded service_account.json file using Google Cloud Workload Identity

```
{
  "type": "external_account",
  "audience": "//iam.googleapis.com/projects/123456789/locations/global/workloadIdentityPools/test-
pool/providers/test-provider",
  "subject_token_type": "urn:ietf:params:oauth:token-type:jwt",
  "token_url": "https://sts.googleapis.com/v1/token",
  "service_account_impersonation_url": "https://iamcredentials.googleapis.com/v1/projects/-
/serviceAccounts/<client_email_address>:generateAccessToken",
  "credential_source": {
    "file": "<path_to_token>",
    "format": {
      "type": "text"
    }
  }
}
```

where:

type

Specifies the credential type, in this example the type is **external_account**.

audience

Specifies the target audience is the Google Cloud Workload Identity provider.

service_account_impersonation_url

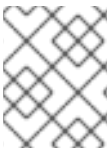
Specifies the resource URL of the service account that can be impersonated with these credentials.

credential_source.file

Specifies the path to the service account token inside the pod. By convention, this is **/var/run/secrets/openshift/serviceaccount/token** for OpenShift Container Platform components.

21.5.2.3. Google Cloud component secret permissions requirements

OpenShift Container Platform components require the following permissions. These values are in the **CredentialsRequest** custom resource (CR) for each component.



NOTE

These permissions apply to all resources. Unless specified, there are no request conditions on these permissions.

Component	Custom resource	Required permissions for services
Cloud Controller Manager Operator	**openshift-gcp-ccm**	Compute Engine • **compute.addresses.create** • **compute.addresses.delete** • **compute.addresses.get** • **compute.addresses.list** • **compute.firewalls.create** • **compute.firewalls.delete** • **compute.firewalls.get** • **compute.firewalls.update** • **compute.forwardingRules.create** • **compute.forwardingRules.delete** • **compute.forwardingRules.get** • **compute.healthChecks.create**

Component	Custom resource	Required permissions for services
		• compute.healthChecks.get • compute.healthChecks.update • compute.httpHealthChecks.create • compute.httpHealthChecks.delete • compute.httpHealthChecks.get • compute.httpHealthChecks.update • compute.instanceGroups.create • compute.instanceGroups.delete • compute.instanceGroups.get • compute.instanceGroups.update • compute.instances.get • compute.instances.use • compute.regionBackendServices.create • compute.regionBackendServices.delete • compute.regionBackendServices.get • compute.regionBackendServices.update • compute.targetPools.addInstance • compute.targetPools.create • compute.targetPools.delete • compute.targetPools.get

Component	Custom resource	Required permissions for services
		• **compute.targetPools.addInstance** • **compute.zones.list**
Cloud Credential Operator	**cloud-credential-operator-gcp-ro-creds**	Identity and Access Management (IAM) • **iam.roles.get** • **iam.serviceAccountKeys.list** • **iam.serviceAccounts.get** Resource Manager • **resourcemanager.projects.get** • **resourcemanager.projects.getIamPolicy** Service Usage • **serviceusage.services.list**

Component	Custom resource	Required permissions for services
Cluster Image Registry Operator	**openshift-image-registry-gcs**	Cloud Storage • **storage.buckets.create** • **storage.buckets.createTagBinding** • **storage.buckets.delete** • **storage.buckets.get** • **storage.buckets.list** • **storage.buckets.listEffectiveTags** • **storage.objects.create** • **storage.objects.delete** • **storage.objects.get** • **storage.objects.list** Resource Manager • **resourcemanager.tagValueBindings.create** • **resourcemanager.tagValues.get** • **resourcemanager.tagValues.list**
Cluster Ingress Operator	**openshift-ingress-gcp**	Cloud DNS • **dns.changes.create** • **dns.resourceRecordSets.create** • **dns.resourceRecordSets.delete** • **dns.resourceRecordSets.list** • **dns.resourceRecordSets.update**

Component	Custom resource	Required permissions for services
Cluster Network Operator	**openshift-cloud-network-config-controller-gcp**	Compute Engine • **compute.instances.get** • **compute.instances.updateNetworkInterface** • **compute.subnetworks.get** • **compute.subnetworks.use** • **compute.zoneOperations.get**
Cluster Storage Operator	**openshift-gcp-pd-csi-driver-operator**	Compute Engine • **compute.instances.attachDisk** • **compute.instances.detachDisk** • **compute.instances.get** This component also requires the following Google Cloud predefined roles: • **roles/compute.storageAdmin** • **roles/iam.serviceAccountUser** • **roles/resourcemanager.tagUser**
Machine API Operator	**openshift-machine-api-gcp**	Compute Engine • **compute.acceleratorTypes.get** • **compute.acceleratorTypes.list** • **compute.disks.create** • **compute.disks.createTagBinding**

Component	Custom resource	Required permissions for services
		• compute.disks.setLabels • compute.globalOperations.get • compute.globalOperations.list • compute.healthChecks.useReadOnly • compute.images.get • compute.images.getFromFamily • compute.images.useReadOnly • compute.instanceGroups.create • compute.instanceGroups.delete • compute.instanceGroups.get • compute.instanceGroups.list • compute.instanceGroups.update • compute.instances.create • compute.instances.createTagBinding • compute.instances.delete • compute.instances.get • compute.instances.list • compute.instances.setLabels • compute.instances.setMetadata • compute.instances.setServiceAccount • compute.instances.setTags

Component	Custom resource	Required permissions for services
		• **compute.instances.update** • **compute.instances.use** • **compute.machineTypes.get** • **compute.machineTypes.list** • **compute.projects.get** • **compute.regionBackendServices.create** • **compute.regionBackendServices.get** • **compute.regionBackendServices.update** • **compute.regions.get** • **compute.regions.list** • **compute.subnetworks.use** • **compute.subnetworks.useExternalIp** • **compute.targetPools.addInstance** • **compute.targetPools.delete** • **compute.targetPools.get** • **compute.targetPools.removeInstance** • **compute.zoneOperations.get** • **compute.zoneOperations.list** • **compute.zones.get** • **compute.zones.list** Identity and Access Management (IAM) • **iam.serviceAccounts.actAs**

Component	Custom resource	Required permissions for services
		• iam.serviceAccounts.get • iam.serviceAccounts.list Resource Manager • resourcemanager.tagValues.get • resourcemanager.tagValues.list Service Usage • serviceusage.quotas.get • serviceusage.services.get • serviceusage.services.list

21.5.2.4. OLM-managed Operator support for authentication with GCP Workload Identity

To allow certain Operators that are managed by the Operator Lifecycle Manager (OLM) on Google Cloud clusters to authenticate with limited-privilege, short-term credentials that are managed outside the cluster, you can use manual mode with GCP Workload Identity.

To determine if an Operator supports authentication with GCP Workload Identity, see the Operator description in the software catalog.

Additional resources

- CCO-based workflow for OLM-managed Operators with Google Cloud Platform Workload Identity

21.5.2.5. Application support for GCP Workload Identity service account tokens

You can use GCP Workload Identity authentication with applications in customer workloads on OpenShift Container Platform clusters that use Google Cloud Platform Workload Identity.

To use this authentication method with your applications, you must complete configuration steps on the cloud provider console and your OpenShift Container Platform cluster.

Additional resources

- Configuring GCP Workload Identity authentication for applications on Google Cloud

21.5.3. About Microsoft Entra Workload ID

When you configure your cluster to use manual mode with Microsoft Entra Workload ID, the individual OpenShift Container Platform cluster components use the Workload ID provider to assign to components short-term security credentials.

Additional resources

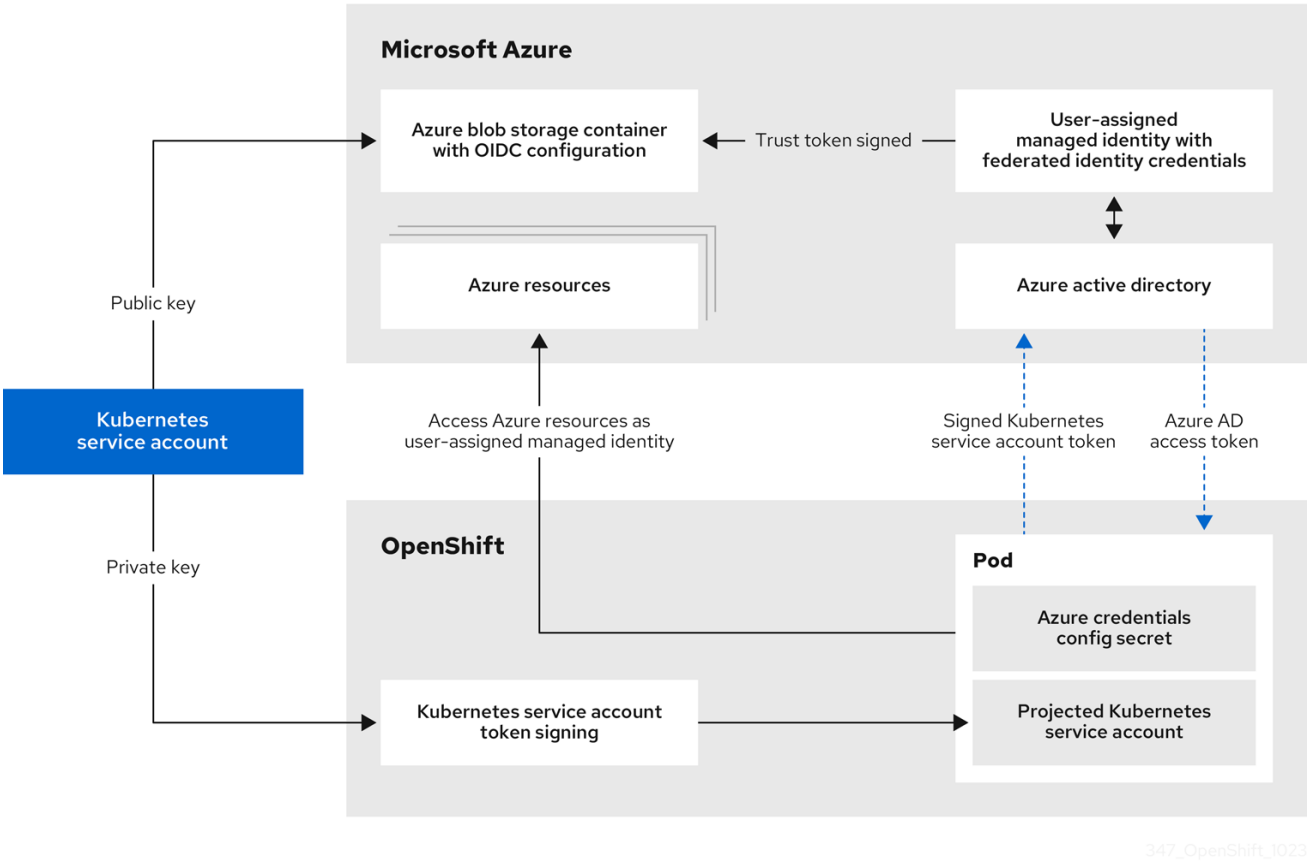
- Configuring a global Microsoft Azure cluster to use short-term credentials

21.5.3.1. Microsoft Entra Workload ID authentication process

You should familiarize yourself with the Microsoft Entra Workload ID authentication flow.

The following diagram details the authentication flow between Microsoft Azure and the OpenShift Container Platform cluster when using Workload ID.

Figure 21.4. Workload ID authentication flow



21.5.3.2. Azure component secret formats

To change the content of the Azure credentials that are provided to individual OpenShift Container Platform components, you can use manual mode with with Microsoft Entra Workload ID.

Compare the following secret formats:

Azure secret format using long-term credentials

```
apiVersion: v1
kind: Secret
metadata:
  namespace: <target_namespace>
  name: <target_secret_name>
data:
  azure_client_id: <client_id>
  azure_client_secret: <client_secret>
```

```
  azure_region: <region>
  azure_resource_prefix: <resource_group_prefix>
  azure_resourcegroup: <resource_group_prefix>-rg
  azure_subscription_id: <subscription_id>
  azure_tenant_id: <tenant_id>
type: Opaque
```

where:

metadata.namespace

Specifies the namespace for the component.

metadata.name

Specifies the name of the component secret.

data.azure_client_id

Specifies the client ID of the Microsoft Entra ID identity that the component uses to authenticate.

data.azure_client_secret

Specifies the component secret that is used to authenticate with Microsoft Entra ID for the **<client_id>** identity.

data.azure_resource_prefix

Specifies the resource group prefix.

data.azure_resourcegroup

Specifies the resource group. This value is formed by the **<resource_group_prefix>** and the suffix **-rg**.

Azure secret format using Microsoft Entra Workload ID

```
apiVersion: v1
kind: Secret
metadata:
  namespace: <target_namespace>
  name: <target_secret_name>
data:
  azure_client_id: <client_id>
  azure_federated_token_file: <path_to_token_file>
  azure_region: <region>
  azure_subscription_id: <subscription_id>
  azure_tenant_id: <tenant_id>
type: Opaque
```

where:

metadata.namespace

Specifies the namespace for the component.

metadata.name

Specifies the name of the component secret.

data.azure_client_id

Specifies the client ID of the user-assigned managed identity that the component uses to authenticate.

data.azure_federated_token_file

Specifies the path to the mounted service account token file.

21.5.3.3. Azure component secret permissions requirements

You should familiarize yourself with the permissions required by the OpenShift Container Platform components. These values are in the **CredentialsRequest** custom resource (CR) for each component.

Component	Custom resource	Required permissions for services
Cloud Controller Manager Operator	**openshift-azure-cloud-controller-manager**	• **Microsoft.Compute/virtualMachines/read** • **Microsoft.Network/loadBalancers/read** • **Microsoft.Network/loadBalancers/write** • **Microsoft.Network/networkInterfaces/read** • **Microsoft.Network/networkSecurityGroups/read** • **Microsoft.Network/networkSecurityGroups/write** • **Microsoft.Network/publicIPAddresses/join/action** • **Microsoft.Network/publicIPAddresses/read** • **Microsoft.Network/publicIPAddresses/write**
Cluster CAPI Operator	**openshift-cluster-api-azure**	role: **Contributor** [1]
Machine API Operator	**openshift-machine-api-azure**	• **Microsoft.Compute/availabilitySets/delete** • **Microsoft.Compute/availabilitySets/read** • **Microsoft.Compute/availabilitySets/write** • **Microsoft.Compute/diskEncryptionSets/read**

Component	Custom resource	Required permissions for services
		• Microsoft.Compute/disks/write • Microsoft.Compute/galleries/images/versions/read • Microsoft.Compute/skus/read • Microsoft.Compute/virtualMachines/delete • Microsoft.Compute/virtualMachines/extensions/delete • Microsoft.Compute/virtualMachines/extensions/read • Microsoft.Compute/virtualMachines/extensions/write • Microsoft.Compute/virtualMachines/read • Microsoft.Compute/virtualMachines/write • Microsoft.ManagedIdentity/userAssignedIdentities/assign/action • Microsoft.Network/applicationSecurityGroups/read • Microsoft.Network/loadBalancers/backendAddressPools/join/action • Microsoft.Network/loadBalancers/read • Microsoft.Network/loadBalancers/write • Microsoft.Network/networkInterfaces/delete • Microsoft.Network/networkInterfaces/join/action • Microsoft.Network/networkInterfaces/loadBalancers/read

Component	Custom resource	Required permissions for services
		• **Microsoft.Network/networkInterfaces/read** • **Microsoft.Network/networkInterfaces/write** • **Microsoft.Network/networkSecurityGroups/read** • **Microsoft.Network/networkSecurityGroups/write** • **Microsoft.Network/publicIPAddresses/delete** • **Microsoft.Network/publicIPAddresses/join/action** • **Microsoft.Network/publicIPAddresses/read** • **Microsoft.Network/publicIPAddresses/write** • **Microsoft.Network/routeTables/read** • **Microsoft.Network/virtualNetworks/delete** • **Microsoft.Network/virtualNetworks/read** • **Microsoft.Network/virtualNetworks/subnets/join/action** • **Microsoft.Network/virtualNetworks/subnets/read** • **Microsoft.Resources/subscriptions/resourceGroups/read**
Cluster Image Registry Operator	**openshift-image-registry-azure**	Data permissions • **Microsoft.Storage/storageAccounts/blobServices/containers/blobs/delete** • **Microsoft.Storage/storageAccounts/blobServices/containers/**

Component	Custom resource	Required permissions for services
		blobs/write Microsoft.Storage/storageAccounts/blobServices/containers/blobs/read • Microsoft.Storage/storageAccounts/blobServices/containers/blobs/add/action • Microsoft.Storage/storageAccounts/blobServices/containers/blobs/move/action General permissions • Microsoft.Storage/storageAccounts/blobServices/read • Microsoft.Storage/storageAccounts/blobServices/containers/read • Microsoft.Storage/storageAccounts/blobServices/containers/write • Microsoft.Storage/storageAccounts/blobServices/generateUserDelegationKey/action • Microsoft.Storage/storageAccounts/read • Microsoft.Storage/storageAccounts/write • Microsoft.Storage/storageAccounts/delete • Microsoft.Storage/storageAccounts/listKeys/action • Microsoft.Resources/tags/write

Component	Custom resource	Required permissions for services
Ingress Operator	**openshift-ingress-azure**	• **Microsoft.Network/dnsZones/A/delete** • **Microsoft.Network/dnsZones/A/write** • **Microsoft.Network/privateDnsZones/A/delete** • **Microsoft.Network/privateDnsZones/A/write**
Cluster Network Operator	**openshift-cloud-network-config-controller-azure**	• **Microsoft.Network/networkInterfaces/read** • **Microsoft.Network/networkInterfaces/write** • **Microsoft.Compute/virtualMachines/read** • **Microsoft.Network/virtualNetworks/read** • **Microsoft.Network/virtualNetworks/subnets/join/action** • **Microsoft.Network/loadBalancers/backendAddressPools/join/action**

Component	Custom resource	Required permissions for services
Azure File CSI Driver Operator	**azure-file-csi-driver-operator**	• **Microsoft.Network/networkSecurityGroups/join/action** • **Microsoft.Network/virtualNetworks/subnets/read** • **Microsoft.Network/virtualNetworks/subnets/write** • **Microsoft.Storage/storageAccounts/delete** • **Microsoft.Storage/storageAccounts/fileServices/read** • **Microsoft.Storage/storageAccounts/fileServices/shares/delete** • **Microsoft.Storage/storageAccounts/fileServices/shares/read** • **Microsoft.Storage/storageAccounts/fileServices/shares/write** • **Microsoft.Storage/storageAccounts/listKeys/action** • **Microsoft.Storage/storageAccounts/read** • **Microsoft.Storage/storageAccounts/write**

Component	Custom resource	Required permissions for services
Azure Disk CSI Driver Operator	**azure-disk-csi-driver-operator**	• **Microsoft.Compute/disks/*** • **Microsoft.Compute/snapshots/*** • **Microsoft.Compute/virtualMachineScaleSets/*/read** • **Microsoft.Compute/virtualMachineScaleSets/read** • **Microsoft.Compute/virtualMachineScaleSets/virtualMachines/write** • **Microsoft.Compute/virtualMachines/*/read** • **Microsoft.Compute/virtualMachines/write** • **Microsoft.Resources/subscriptions/resourceGroups/read**

1. This component requires a role rather than a set of permissions.

21.5.3.4. OLM-managed Operator support for authentication with Microsoft Entra Workload ID

To allow certain Operators that are managed by the Operator Lifecycle Manager (OLM) on Azure clusters to authenticate with limited-privilege, short-term credentials that are managed outside the cluster, you can use manual mode with Microsoft Entra Workload ID.

To determine if an Operator supports authentication with Workload ID, see the Operator description in the software catalog.

Additional resources

- CCO-based workflow for OLM-managed Operators with Microsoft Entra Workload ID

21.5.4. Additional resources

- Enabling token-based authentication
- Configuring an AWS cluster to use short-term credentials

- Configuring a Google Cloud cluster to use short-term credentials
- Configuring a global Microsoft Azure cluster to use short-term credentials
- Preparing to update a cluster with manually maintained credentials